

中华人民共和国通信行业标准

YD/T [×××××]—[××××]
[代替 YD/T]

基于 LTE 的车联网无线通信技术 安全证书
管理系统技术要求

LTE-based vehicular communication — Technical requirement of security
certificate management system

[点击此处添加与国际标准一致性程度的标识]

(报批稿)

[点击此处添加本稿完成日期]

[××××]-[××]-[××]发布

[××××]-[××]-[××]实施

中华人民共和国工业和信息化部 发布

目 次

前 言..... III

1 范围..... 1

2 规范性引用文件..... 1

3 术语、定义和缩略语 1

 3.1 术语和定义..... 1

 3.2 缩略语..... 2

4 概述..... 3

 4.1 V2X 通信安全系统构成..... 3

 4.2 V2X 通信安全服务架构..... 4

5 LTE-V2X 证书管理安全需求 7

 5.1 概述..... 7

 5.2 LTE-V2X 消息安全需求 7

6 LTE-V2X 通信安全认证机制总体技术要求 8

 6.1 LTE-V2X 证书管理系统架构 8

 6.2 LTE-V2X 安全证书..... 16

 6.3 基本元素说明..... 20

 6.4 安全协议数据单元..... 21

 6.5 数字证书和证书管理数据格式..... 32

7 LTE-V2X 通信安全认证交互流程及接口技术要求 44

 7.1 注册证书管理流程..... 44

 7.2 假名证书申请流程..... 50

 7.3 应用证书和身份证书管理流程..... 57

 7.4 证书撤销列表管理流程..... 62

 7.5 机构证书管理流程..... 71

 7.6 异常行为检测上报流程..... 71

 7.7 LA 管理架构和流程..... 72

8 LTE-V2X 通信安全认证 PKI 互信技术要求 75

 8.1 概述..... 75

 8.2 PKI 互信架构..... 75

 8.3 PKI 互信管理过程..... 78

 8.4 PKI 互信认证过程..... 79

 8.5 可信根证书列表管理策略..... 80

 8.6 可信域 CA 证书列表管理策略..... 80

8.7 可信域的异常行为检查 80

附 录 A（资料性附录）车联网通信安全基本应用模式 81

附 录 B（资料性附录）基于 OAUTH 的 token 授权机制 83

附 录 C（规范性附录）ASN.1 模板 86

附 录 D（规范性附录）密码算法的输入与输出 107

附 录 E（规范性附录）V2X 设备与安全证书管理系统接口的数据格式 111

附 录 F（规范性附录）GBA 机制应用层会话密钥产生及使用方法 146

附 录 G（资料性附录）证书生命周期及更新场景 147

附 录 H（资料性附录）密钥衍生流程的一种算法建议 150

附 录 I（规范性附录）链接值相关定义 153

附 录 J（规范性附录）可信 CA 证书列表及互信认证流程 155

附 录 K（资料性附录）算法编码示例 158

前 言

本文件按照GB/T 1.1—2020《标准化工作导则 第1部分：标准化文件的结构和起草规则》的规定起草。

请注意本文件的某些内容可能涉及专利。本文件的发布机构不承担识别这些专利的责任。

本文件由中国通信标准化协会提出并归口。

本文件起草单位：大唐电信科技产业集团（电信科学技术研究院）、中国信息通信研究院、中国移动通信集团有限公司、国汽（北京）智能网联汽车研究院有限公司、华为技术有限公司、高通无线通信技术（中国）有限公司、东软集团股份有限公司、郑州信大捷安信息技术股份有限公司、大众汽车（中国）投资有限公司、宝马（中国）服务有限公司、通用汽车（中国）投资有限公司、中国汽车工程研究院股份有限公司、北京数字认证股份有限公司、北京仁信证科技有限公司、深圳奥联信息安全技术有限公司、上海汽车集团股份有限公司、北京奇虎科技有限公司、深圳市腾讯计算机系统有限公司、北京信安世纪科技股份有限公司。

本文件主要起草人：徐晖、周巍、于润东、葛雨明、栗栗、田野、刘建行、罗璎珞、吴锦荣、杜志敏、潘凯、朱锦涛、康亮、周吉祥、李向锋、刘帅、李志明、梁承志、程朝辉、张丽佳、苏赓、郑雪松、张存玺、雷艺学、张永强、孔勇、张屹、张庆勇、高吉、郑军、刘鹏、陆玮瑾、张元生、金枫。

基于LTE的车联网无线通信技术 安全证书管理系统技术要求

1 范围

本文件规定了基于LTE的车联网安全证书管理系统技术要求，主要包括安全证书管理系统架构和相关的显式证书格式及交互流程。

本文件适用于LTE-V2X设备和安全证书管理系统。

2 规范性引用文件

下列文件中的内容通过文中的规范性引用而构成本文件必不可少的条款。其中，注日期的引用文件，仅该日期对应的版本适用于本文件；不注日期的引用文件，其最新版本（包括所有的修改单）适用于本文件。

GB/T 16262（所有部分） 信息技术抽象语法记法一（ASN.1）

GB/T 25056 信息安全技术 证书认证 系统密码及其相关安全技术规范

GB/T 32905 信息安全技术 SM3密码杂凑算法

GB/T 32907 信息安全技术 SM4分组密钥算法

GB/T 32918（所有部分） 信息安全技术 SM2椭圆曲线公钥算法

GB/T 36624 信息技术 安全技术 可鉴别的加密机制

YD/T 3707-2020 基于LTE的车联网无线通信技术 网络层技术要求

3GPP TS 33.220 通用认证架构：通用引导架构（Generic Authentication Architecture (GAA)；Generic Bootstrapping Architecture (GBA)）

ISO/IEC 8825-7 信息技术抽象语法记法一（ASN.1）编码规则 第7部分 八位字节编码规则（OER）（Information technology -- ASN.1 encoding rules: Specification of Octet Encoding Rules (OER)）

3 术语、定义和缩略语

3.1 术语和定义

下列术语和定义适用于本文件。

3.1.1

V2X 设备 V2X equipment

车载设备（OBU）、路侧设备（RSU）和服务提供商（VSP）的安全设备。

3.1.2

V2X 通信证书 V2X communication certificate

证书机构签发给车联网设备的各种与V2X通信相关的证书，例如注册证书、假名证书、应用证书、身份证书等。

3.2 缩略语

下列缩略语适用于本文件。

AAA	认证授权机构	Authentication and Authorization Authority
AC	应用证书	Application Certificate
ACA	应用证书机构	Application Certificate Authority
AID	应用标识	Application Identifier
API	应用程序接口	Application Programming Interface
ARA	应用证书注册机构	Application Certificate Registration Authority
ASN.1	抽象记法 1	Abstract Syntax Notation dot one
BSF	引导服务功能	Bootstrapping Server Function
BSM	主动安全消息	Basic Safety Message
CA	证书机构	Certificate Authority
CRA	证书撤销机构	Certificate Revocation Authority
CRL	证书撤销列表	Certificate Revocation List
CTL	证书可信列表	Certificate Trust List
DCM	设备配置管理	Device Configuration Manager
EC	注册证书	Enrolment Certificate
ECA	注册证书机构	Enrolment Certificate Authority
GBA	通用引导架构	Generic Bootstrapping Architecture
HTTP	超文本传输协议	Hyper Text Transfer Protocol
HTTPS	超文本传输安全协议	Hyper Text Transfer Protocol Secure
ICA	中间证书机构	Intermediate CA
ID	身份标识	Identity
LA	链接机构	Linkage Authority
LTE	长期演进	Long Term Evolution
LTE-V2X	基于长期演进的车联网通信技术	LTE- Vehicle to Everything
Is	链接种子	Linkage Seed
Iv	链接值	Linkage Value
MA	异常行为管理机构	Misbehavior Authority
NAF	网络应用功能	Network Application Function
OBUE	车载设备	On Board Unit
PC	假名证书	Pseudonym Certificate
PCA	假名证书机构	Pseudonym Certificate Authority
PDU	协议数据单元	Protocol Data Unit
PKI	公钥基础设施	Public Key Infrastructure
PRA	假名证书注册机构	Pseudonym Certificate Registration Authority
RA	注册机构	Registration Authority
Rscm	安全凭证管理参考点	Security Credential Management Reference point
Rsde	安全数据交换参考点	Secure Data Exchange Reference point
RSU	路侧设备	Road Side Unit
SCME	安全凭证管理实体	Security Credential Management Entity
SCMF	安全凭证管理功能	Security Credential Management Function
SDPF	安全数据处理功能	Secure Data Processing Function
SPDU	安全协议数据单元	Secured Protocol Data Unit
SSF	安全服务功能	Security Service Function

SSP	服务特定许可	Service Specific Permission
TCMF	可信证书管理功能	Trusted Certificate Management Function
TDCL	可信域 CA 证书列表	Trusted Domain CA Certificates List
TLS	传输层安全性协议	Transport Layer Security
TRCL	可信根证书列表	Trusted Root Certificate List
TRCLA	可信根证书列表管理机构	Trusted Root Certificate List Authority
USIM	全球用户识别模块	Universal Subscriber Identity Module
V2X	车联万物	Vehicle to Everything
VSP	车联网服务提供商	V2X Service Provider

4 概述

4.1 V2X 通信安全系统构成

车联网通信安全实体关系参考模型如图1所示，图中实线表示V2X设备之间的通信关系，虚线表示实体之间的授权关系。该参考模型由如下实体构成：

- 车载设备（On Board Unit, OBU）：安装在车辆上，负责 V2X 通信的实体。数据发送时，OBU 使用与 CA 签发给它的数字证书对应的私钥对其播发的信息进行数字签名和/或使用数据接收方证书对数据进行加密；数据接收时，OBU 使用发送方的公钥对消息进行验证和/或使用本地私钥对加密消息进行解密。
- 路侧设备（Road Side Unit, RSU）：安装在路侧交通控制设备和交通信息发布设备中，负责 V2X 通信的实体。数据发送时，RSU 使用与 CA 签发给它的数字证书对应的私钥对其播发的信息进行数字签名和/或使用数据接收方证书对数据进行加密；数据接收时，RSU 使用发送方的公钥对消息进行验证和/或使用本地私钥对加密消息进行解密。
- 服务提供商（V2X Service Provider, VSP）：负责道路交通的管理机构和在车联网系统里提供某种商业服务的服务机构。数据发送时，VSP 使用与 CA 签发给它的数字证书对应的私钥对其播发的信息进行数字签名和/或使用数据接收方证书对数据进行加密；数据接收时，VSP 使用发送方的公钥对消息进行验证和/或使用本地私钥对加密消息进行解密。VSP 需要通过具有转发能力的路侧设备进行安全消息的发送和接收。
- 证书机构（Certificate Authority, CA）：负责向车联网设备（OBU, RSU, VSP）签发各种通信证书或签发证书撤销列表（Certificate Revocation List, CRL），例如注册 CA、假名 CA、应用 CA、证书撤销机构（Certificate Revocation Authority, CRA）等。

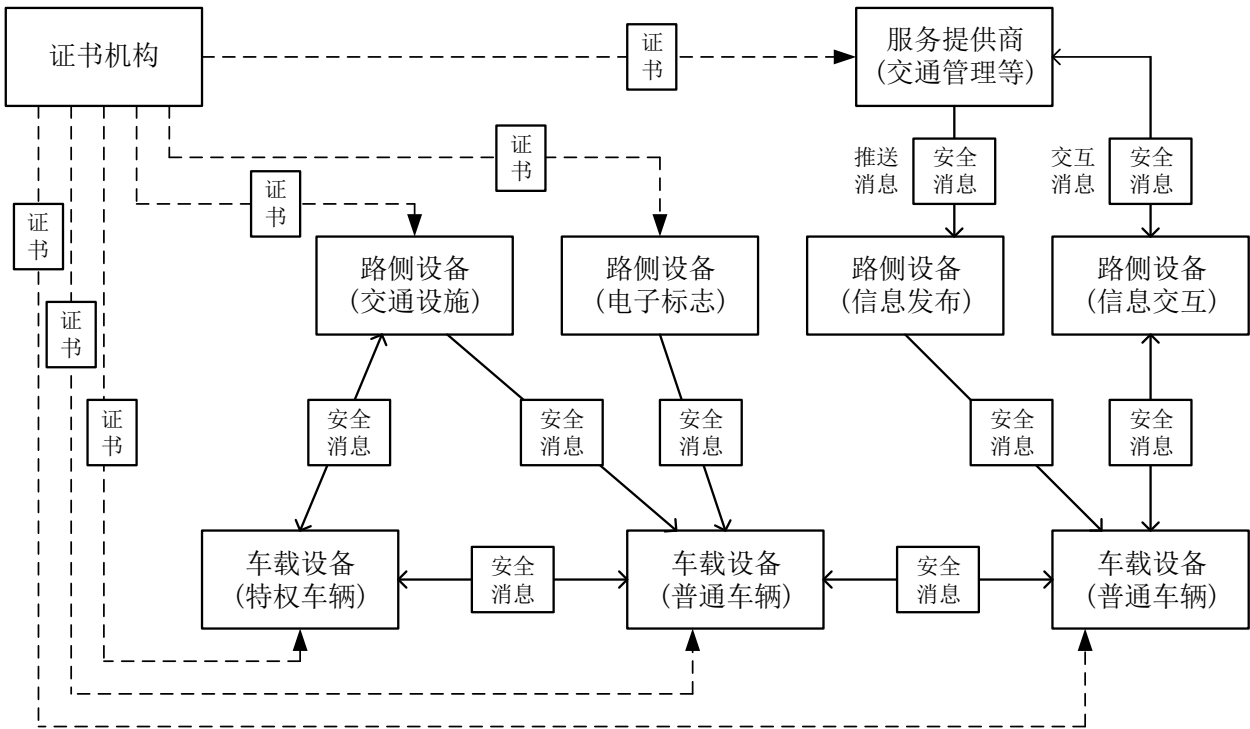


图1 车联网通信安全实体关系参考模型

证书机构车联网通信安全系统的基本应用模式参见附录A。

4.2 V2X 通信安全服务架构

4.2.1 安全功能和安全实体

4.2.1.1 概述

车联网安全通信服务架构如图2所示，其由如下功能或实体构成：

- 安全数据处理功能（Secure Data Processing Function，SDPF）
- 安全凭证管理功能（Security Credential Management Function，SCMF）
- 安全服务功能（Security Service Function，SSF）
- 安全凭证管理实体（Security Credential Management Entity，SCME）

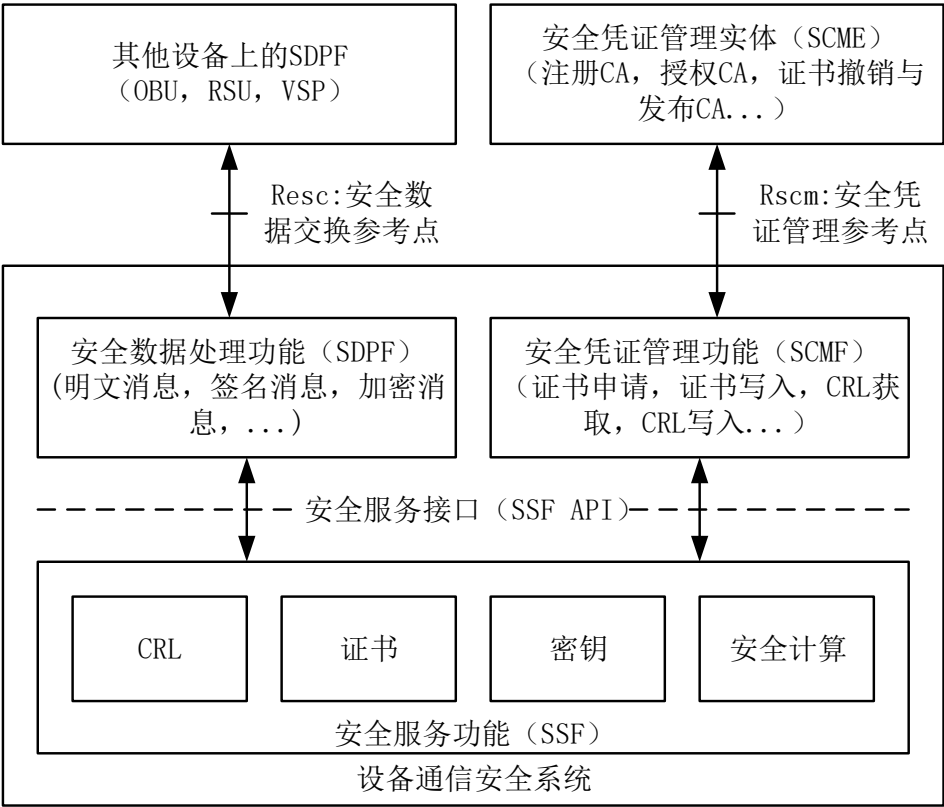


图2 车联网安全通信服务架构

4.2.1.2 安全数据处理功能

安全数据处理功能（SDPF）位于车联网设备中的某个车联网应用服务实体中，负责处理与特定应用相关的安全性操作，也即基于特定的车联网应用逻辑生成与处理安全消息。例如，在车联网主动安全应用中，车辆主动安全系统基于相应的应用逻辑生成签名消息或验证签名消息并获得消息明文。SDPF在执行安全数据处理过程中可能需要调用SSF提供的基本安全服务，例如数字签名、数字签名验证、数据加密、数据解密等。

4.2.1.3 安全凭证管理功能

安全凭证管理功能（SCMF）在车联网设备中负责与车联网安全凭证管理实体交互获得相关的公钥证书和CRL等安全凭证或数据。SCMF需要与SSF交互以完成安全凭证和数据（例如公钥证书和CRL）的导入和导出，以及密钥的生成和导出等操作。

4.2.1.4 安全服务功能

安全服务功能（SSF）位于车联网设备中，负责提供安全凭证和安全数据的存储和密码运算服务，例如，公钥证书和CRL的存储，密钥的生成和存储，签名、验签、加密、解密和哈希运算等密码运算。SSF通过SSF应用程序接口（Application Programming Interface, API）对外提供安全服务。

SSF负责处理本标准定义的安全协议数据单元（Secured Protocol Data Units, SPDUs）。SSF通过生成和处理SPDU为车联网设备提供签名、验签、加密和解密等安全服务。

4.2.1.5 安全凭证管理实体

安全凭证管理实体（SCME）负责向车辆联网设备签发各种用于通信安全的公钥证书，以及管理和发布证书撤销列表。

4.2.2 通信安全协议数据单元

在车联网系统中，为通信数据提供安全保护的基本数据单位称为安全协议数据单元（Secured Protocol Data Unit, SPDU）。安全协议数据单元由安全服务功能（SSF）创建和处理。

本标准定义了如下类型的安全协议数据单元：

- 明文安全协议数据单元（Unsecured SPDU）：数据未经数字签名或数据加密保护。该数据单元的使用者对数据单元中携带的数据不需要进行进一步的安全处理即可以使用。
- 签名安全协议数据单元（Signed SPDU）：数据为经与公钥证书对应的私钥进行数字签名后的数据。该数据单元的使用者需要对数据单元中携带的数据的完整性进行验证、对来源进行确认后才能使用。
- 加密安全协议数据单元（Encrypted SPDU）：数据为经对称密钥或非对称密钥加密后的数据。安全协议数据单元可嵌套使用，具体的应用顺序依据应用逻辑确定。

4.2.3 通信安全消息

在车联网通信中，由某个实体发送给其他实体的包含有安全协议数据单元（SPDU）和/或处理安全协议数据单元所需数据的消息称为安全消息（Secure Message）。安全消息由安全数据处理功能（SDPF）依据具体的应用逻辑创建和处理。

4.2.4 通信安全参考点

车联网实体之间通过安全通信参考点进行安全的数据交换操作。本标准定义了2个与通信安全相关的参考点：

- 安全数据交换参考点（Secure Data Exchange Reference point, Rsde）：车联网设备之间通过 Rsde 参考点来安全地交换数据。
- 安全凭证管理参考点（Security Credential Management Reference point, Rscm）：车联网设备通过 Rscm 参考点与安全凭证签发实体进行交互，以申请证书和获得 CRL 等安全数据。

4.2.5 通信安全过程

车联网通信安全的一般过程如图3所示。具体步骤如下：

步骤1：发送方的SDPF依据某个车联网应用逻辑生成明文数据，并向本地SSF发送安全服务请求，以便获得后者为其提供数字签名或数据加密等安全服务。服务请求中包含有明文数据和/或处理安全协议数据单元所需的数据，例如公钥证书等。

步骤2：发送方的SSF根据安全服务请求执行相应的安全操作，例如数据签名或数据加密等。SSF将安全操作的结果封装在SPDU中，然后将该SPDU通过安全服务响应返回给SDPF。

步骤3：发送方的SDPF依据SSF生成的SPDU和应用逻辑生成应用特定格式的安全消息，例如主动安全消息。

步骤4：发送方的SDPF将生成的安全消息广播出去。

步骤5：接收方的SDPF依据应用逻辑从接收到的安全消息中获得SPDU。

步骤6：接收方SDPF向本地SSF发送安全服务请求，以便获得后者为其提供签名验证或数据解密等安全服务。服务请求中包含有SPDU和/或处理安全协议数据单元所需的数据，例如公钥证书等。

步骤7：接收方的SSF根据安全服务请求执行相应的安全操作，例如验证签名或解密数据等，然后将安全服务结果通过安全服务响应返回给SDPF。

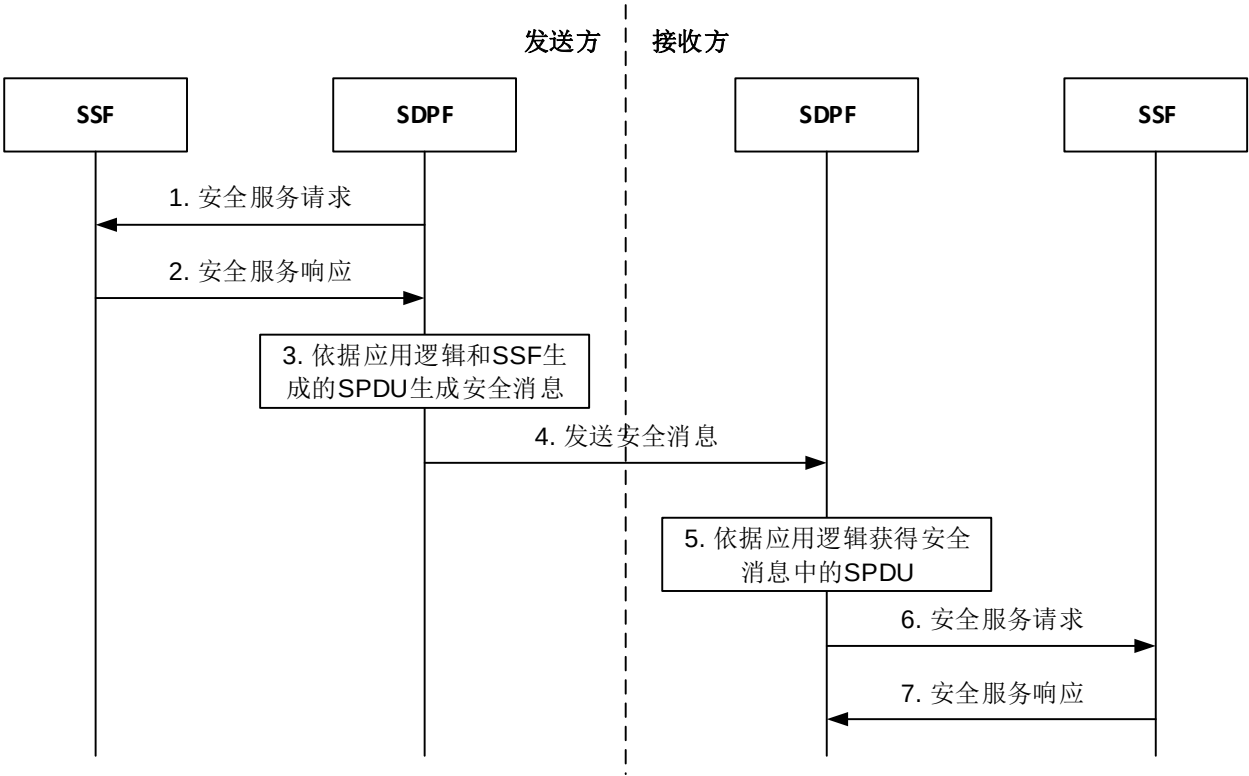


图3 车联网通信安全消息处理流程

5 LTE-V2X 证书管理安全需求

5.1 概述

LTE-V2X通信安全认证的内容包括但不限于：机密性、完整性、可认证性，要求可抵御重放攻击，可检测网络嗅探、DDoS攻击等行为，要求对V2X直连通信的各方尤其是OBU进行隐私保护。

5.2 LTE-V2X 消息安全需求

5.2.1 机密性要求

OBU、RSU和VSP接入CA系统时，根据需要可支持对消息的机密性保护，保证消息在传输时不被窃听，防止用户私密信息泄露。

各CA之间、CA内部各子系统之间通信时，应采用密码技术和安全协议，保证重要数据在传输过程中的机密性。

CA应保证重要数据在存储过程中的机密性。

5.2.2 完整性要求

OBU、RSU和VSP接入CA系统时，应支持对消息的完整性保护，防止消息被伪造、篡改。

CA之间、CA内部各子系统之间通信时，应采用密码技术和安全协议，保证重要数据在传输过程中的完整性。

CA应保证重要数据在存储过程中的完整性。

5.2.3 认证要求

OBU、RSU和VSP接入CA系统过程中，应支持数据源认证，保证数据源头的合法性，防止假冒或伪造的数据信息。

各CA之间、CA内部各子系统之间通信时，应执行双向认证，确认对方身份的合法性。

5.2.4 抗重放攻击要求

OBU、RSU和VSP接入CA系统过程中，应支持对消息的抗重放保护。

CA系统网络边界应部署边界防护设备，能够检测网络嗅探、DDoS攻击等行为。

5.2.5 隐私保护要求

CA系统应确保假名证书与非假名证书之间的无关联性，防止用户隐私泄露。

CA系统应支持对OBU真实身份标识的隐藏，防止内部或外部攻击者窃取用户隐私。

5.2.6 CA 系统安全要求

CA系统的安全性应符合GB/T 25056的相关要求。

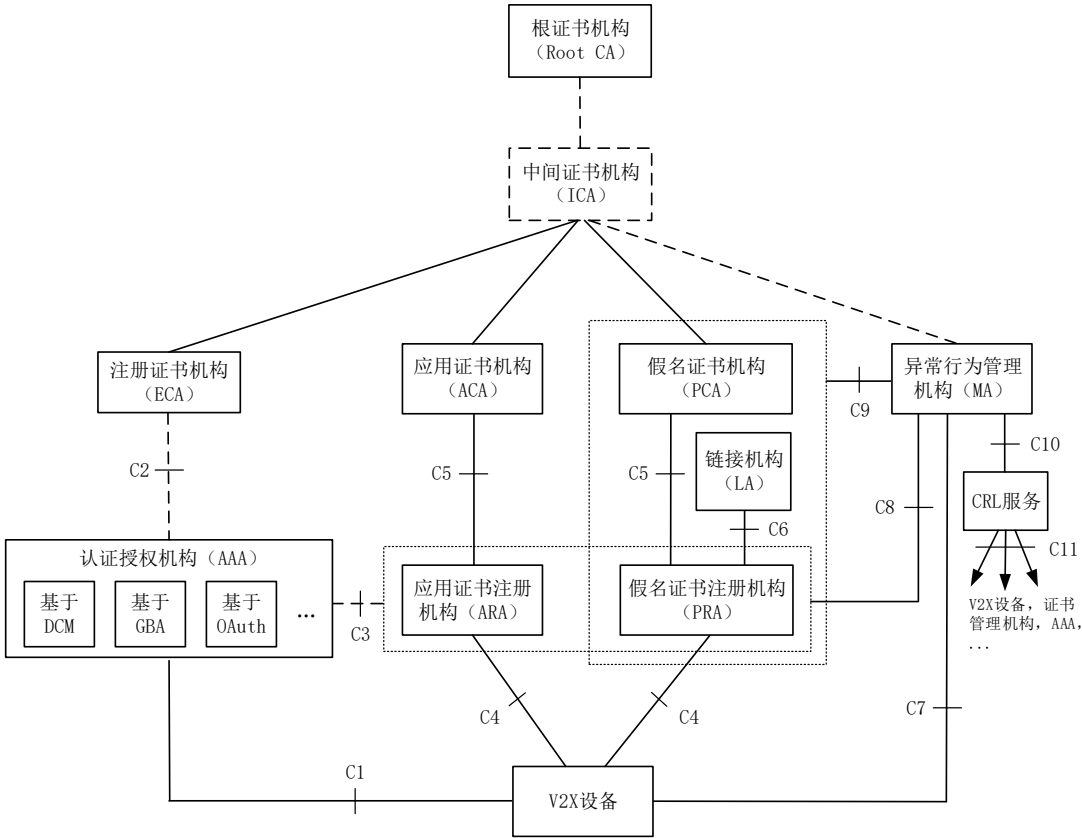
6 LTE-V2X 通信安全认证机制总体技术要求

6.1 LTE-V2X 证书管理系统架构

6.1.1 系统架构

6.1.1.1 概述

本标准中LTE-V2X证书管理系统基于公钥基础设施（Public Key Infrastructure, PKI）实现，其架构如图4所示，主要包括根证书机构、LTE-V2X证书机构、认证授权机构和证书申请主体四部分。实际应用时，图中各逻辑实体可以根据实际设备开发及部署需要合设或者分设，并可以根据政策法规，行业监管要求和业务运营需要，由不同机构分层分级部署、管理和运营。



注：虚线表示基于实际部署方式可能存在也可能不存在。

图4 LTE-V2X 证书管理系统架构

- LTE-V2X证书管理系统包括：
- 根证书机构（Root CA）；
 - 中间证书机构（Intermediate CA，ICA）；
 - LTE-V2X 证书机构；
 - 链接机构（Linkage Authority，LA）；
 - 异常行为管理机构（Misbehavior Authority，MA）；
 - 认证授权机构（Authentication and Authorization Authority，AAA）；
 - 车联网设备（V2X Equipment）。

6.1.1.2 根证书机构

根证书机构Root CA是LTE-V2X证书管理系统的信任根，负责系统根证书的管理与维护并对LTE-V2X证书机构进行注册审批。在确认LTE-V2X证书机构的合法性之后，根证书机构为其签发管理机构的数字证书，使其成为系统内的有效实体。

6.1.1.3 中间证书机构

LTE-V2X证书管理系统可根据PKI部署的实际需要，在根证书机构与LTE-V2X证书机构之间部署中间证书机构，以支持多层级CA部署方式。

6.1.1.4 LTE-V2X 证书机构

LTE-V2X证书机构负责管理LTE-V2X安全通信应用相关数字证书，负责审核证书申请主体的合法性，签发、撤销证书申请主体的数字证书。

LTE-V2X证书机构是LTE-V2X证书管理系统中各种证书机构的统称。根据LTE-V2X安全通信应用的证书类型及用途不同，LTE-V2X证书机构可分为：注册证书机构（Enrolment Certificate Authority, ECA）、假名证书机构（Pseudonym Certificate Authority, PCA）、应用证书机构（Application Certificate Authority, ACA）。

注册、假名和应用证书机构由注册机构（Registration Authority, RA）和证书机构（Certificate Authority, CA）两部分组成。注册机构负责证书申请主体的注册审批管理，证书机构负责数字证书的发行管理。

6.1.1.5 链接机构

链接机构为假名证书生成链接值，以支持假名证书的批量撤销。

6.1.1.6 异常行为管理机构

异常行为管理机构能够识别潜在的异常行为或故障，确定需要撤销的证书，生成证书撤销列表（Certificate Revocation List, CRL）。

6.1.1.7 认证授权机构

认证授权机构负责证书申请主体的身份认证和授权。在设备初始化阶段，为证书申请主体签发注册数字证书或其他类型的安全凭证，使其能够凭借获得的安全凭证与LTE-V2X证书机构安全交互并获取相应的证书。认证授权机构还可以对证书申请主体向LTE-V2X证书机构发起的证书请求进行授权。

根据应用场景的不同，认证授权系统可基于设备配置管理（Device Configuration Manager, DCM）服务系统，LTE网络通用引导架构（General Bootstrapping Architecture, GBA）认证授权系统或者OAuth授权服务系统等多种方式实现。

6.1.1.8 车联网设备

V2X设备为证书申请主体，其向LTE-V2X证书机构申请获取相关数字证书。参与LTE-V2X安全通信的实体，包括车载设备（OnBoard Unit, OBU）、路侧设备（Road Side Unit, RSU）及其他形态的实体。

6.1.2 基于 DCM 的 PKI 架构

V2X设备通过DCM服务系统与注册证书机构交互，以获取注册证书。DCM为被V2X设备和CA系统信任的设备。为此，DCM应预置与V2X设备和CA系统建立安全关联所需的安全凭证。相应地，V2X设备和CA系统也应预置与DCM设备建立安全关联所需的安全凭证。当V2X设备申请注册证书时，其应向DCM获取后续申请注册证书的相关信息，如PKI中各证书签发机构的证书。通过DCM申请V2X设备注册证书时，应确保V2X设备与DCM服务系统之间连接的安全性。例如，可通过基于物理环境安全、TLS安全协议或专用的端到端安全连接实现相关操作。

6.1.3 基于 GBA 的 PKI 架构

基于GBA的PKI架构支持有ECA的系统架构和无ECA的系统架构。GBA机制遵从3GPP TS 33.220实现，GBA认证授权系统包括3GPP标准定义的引导服务功能（Bootstrapping Server Function, BSF）网元和网络应用功能（Network Application Function, NAF）网元，负责OBU、RSU等证书申请主体的身份认

证、业务应用的授权及GBA共享会话密钥的提供。为了确保随机数、密钥等敏感参数信息在终端侧操作处理时的安全性，系统优先采用GBA_U方式实现。

基于GBA的PKI架构可以分为以下两种：

a) 采用 ECA 的系统架构

在采用ECA的系统架构下，证书申请主体首先通过GBA认证授权系统申请获取EC注册证书，之后基于EC注册证书申请LTE-V2X安全通信相关的其他应用数字证书（如PC假名证书，AC应用证书）。此架构下，GBA认证授权系统需要预先与EC注册证书机构建立起安全的通信通道，以确保两者之间数据交互的安全性。

在申请EC注册证书时，GBA认证授权系统基于USIM中的用户标识及根密钥与OBU、RSU终端进行双向身份认证，成功后为ECA提供与终端建立安全关联的GBA共享会话密钥。凭借GBA共享会话密钥，OBU、RSU终端可与ECA安全交互，在线实现EC证书申请、更新等业务处理。

在申请PC，AC等其他应用证书时，PCA、ACA等证书机构通过访问GBA认证授权系统来对OBU、RSU终端进行身份认证并对终端的业务请求获取授权。认证授权成功后，GBA认证授权系统向证书机构提供GBA共享会话密钥。凭借GBA共享会话密钥，OBU、RSU终端安全接入证书机构，并使用EC注册证书实现PC，AC等应用证书的申请、更新等操作。

b) 省略 ECA 的系统架构

在省略ECA的系统架构下，证书申请主体首先接入GBA认证授权系统为业务请求申请获取业务令牌，之后基于业务令牌申请PC、AC等其他应用证书。

在申请业务令牌时，GBA认证授权系统基于USIM中的用户标识及根密钥与OBU、RSU终端进行双向身份认证，并根据策略对终端的业务请求授权。授权成功后，GBA认证授权系统向终端签发业务令牌，并向应用证书机构提供GBA共享会话密钥。凭借GBA共享会话密钥，OBU、RSU终端安全接入证书机构，并使用业务令牌实现PC，AC等应用证书的申请、更新等操作。

6.1.4 基于 OAuth 的 PKI 架构

基于OAuth 2.0授权服务器的PKI架构为无ECA的系统架构。当认证授权服务器是OAuth授权服务器时，OBU通过OAuth机制授权访问PRA服务。这个过程基于IETF标准“RFC6749: OAuth 2.0授权架构”定义的OAuth2.0机制实现。OBU使用RFC 6749中定义的客户端凭证向OAuth授权服务器请求对PRA服务的授权访问。OAuth授权服务器根据策略接受或者拒绝这个请求。如果接受，则向OBU发行Access Token。OBU根据Access Token向服务提供方PRA申请假名证书，OBU与PRA之间建立TLS安全通道，PRA验证Access Token，验证成功后提供对假名证书的访问。基于OAuth的Token授权机制参见附录B。

6.1.5 证书管理系统实体间逻辑接口关系

6.1.5.1 V2X 设备-AAA 接口（C1）

V2X设备通过此接口与DCM、GBA和OAuth等认证授权系统进行交互，以获得向CA申请V2X证书所需的安全凭证。C1接口随实现AAA功能的方式而不同。

6.1.5.2 AAA-ECA 接口（C2）

认证授权系统通过此接口与ECA进行交互，以便向ECA提供与V2X设备建立安全关联所需的安全凭证。C2接口随实现AAA功能的方式而不同。

在V2X设备获得初始安全凭证后，V2X设备与ECA通过经由AAA建立的安全关联进行交互，以便实现注册证书的申请与签发过程。

6.1.5.3 RA-AAA 接口 (C3)

证书签发系统的注册机构通过此接口与AAA交互获得其与V2X设备建立安全关联的安全凭证，或者从AAA获得验证安全凭证所需要的安全材料（例如密钥，算法等），或者从AAA获得向V2X设备签发证书的授权。C3接口随实现AAA功能的方式而不同。

6.1.5.4 V2X 设备-RA 接口 (C4)

V2X设备通过此接口向证书签发系统申请V2X通信证书，也即假名证书、身份证书或应用证书。

6.1.5.5 RA-CA 接口 (C5)

证书签发系统的注册机构基于V2X设备的证书申请请求，生成相应的证书生成请求，将该请求通过此接口提供给CA，并通过此接口获得CA签发的V2X证书。

6.1.5.6 RA-LA 接口 (C6)

在V2X设备假名证书申请过程中，RA通过此接口从LA获得其为V2X设备生成的一组假名证书链接值。

6.1.5.7 V2X 设备-MA 接口 (C7)

V2X设备通过此接口上报其收集到的V2X设备异常行为报告。

6.1.5.8 MA-RA 接口 (C8)

MA通过此接口将被撤销的证书通知给证书签发系统，以便证书签发系统将相关联的注册证书放入黑名单中。

6.1.5.9 MA-LA 接口 (C9)

在V2X设备假名证书撤销过程中，MA通过此接口与假名证书签发系统交互，以便获得撤销一组V2X设备假名证书的数据。

6.1.5.10 MA-CRL 服务接口 (C10)

MA通过此接口将证书撤销列表提供给CRL服务实体。

6.1.5.11 V2X 设备-CRL 服务接口 (C11)

V2X设备或证书管理实体通过此接口从CRL服务实体下载或检查证书撤销列表。

6.1.6 支持多 PKI 系统互认

当车联网安全系统由多个独立PKI系统构成时，这些PKI系统之间可以根据需要构建可信关系，以便实现证书互认。实现多PKI体系可信关系的原理如图5所示。多个车联网PKI系统之间的可信关系是通过一个“可信根证书列表 (Trusted Root Certificate List, TRCL)”实现的。该可信列表由可信根证书列表管理机构 (Trusted Root Certificate List Authority, TRCLA) 签发。

可信根证书列表的存在与否不会影响各个独立PKI系统的运行，但会影响不同PKI系统证书之间是否能够互认。

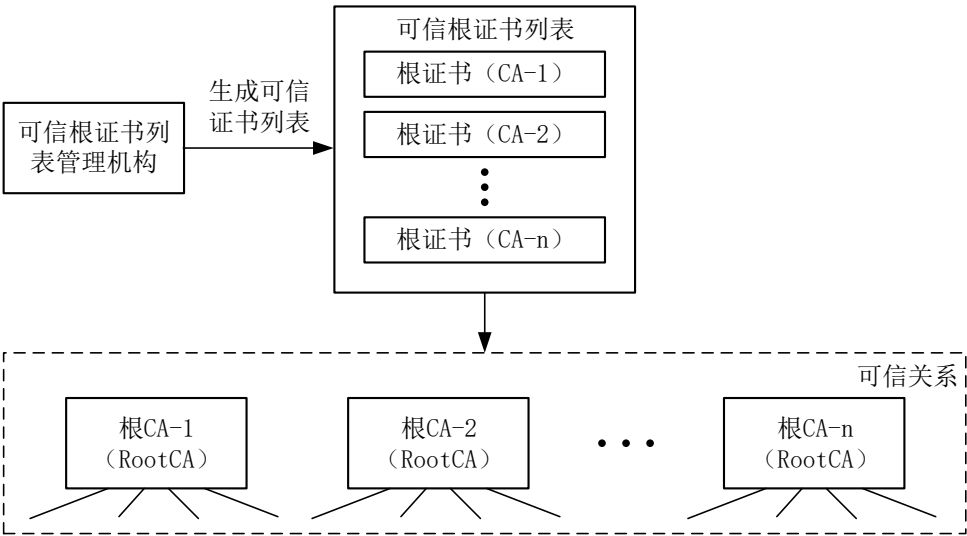


图5 构建多个 PKI 系统之间的可信关系

6.1.7 证书管理实体类型

6.1.7.1 根 CA

根CA（Root CA）是车联网安全体系中某个PKI系统中最高级别的CA。根CA首先需要向自身签发一个自签名证书，该自签名证书又称为根证书（Root Certificate）。根证书是一个PKI系统中所有证书链的终结点，即该PKI系统的信任锚点（trust anchor）。根CA可以根据需要向下级CA签发子CA证书，例如注册CA证书、假名CA证书、应用CA证书等。

6.1.7.2 注册 CA

注册CA（Enrollment CA，ECA）负责向OBU、RSU和VSP等车联网设备签发注册证书。为了加入某个车联网应用，OBU、RSU和VSP等车联网设备可以通过注册机构的认证并获注册证书，然后利用注册证书向应用授权机构（假名CA、应用CA等）申请实际用于生成V2X安全消息的V2X通信证书。

在一个车联网PKI系统中，可以根据不同的应用领域和管理范围设置不同的注册CA，以管理相关应用领域的设备准入。

6.1.7.3 假名 CA

假名CA（Pseudonym CA，PCA）负责向OBU签发假名证书。OBU使用假名证书对其播发的主动安全消息（Basic Safety Message，BSM）进行数字签名。为避免泄露车辆行驶路径，PCA应向OBU签发多个有效期相同的假名证书；OBU依据假名证书使用策略，定期更换用于消息签名的证书。

6.1.7.4 应用 CA

应用CA（Application CA，ACA）：又称应用授权CA（application authorization CA），负责向OBU签发身份证书（Identity Certificate）、向RSU和VSP等车联网设备签发应用证书（Application Certificate）。

6.1.7.5 证书撤销机构

证书撤销机构（Certificate Revocation Authority, CRA）：负责签发已发布证书的证书撤销列表（Certificate Revocation List, CRL）。证书撤销机构可与注册CA、假名CA、应用CA、异常行为管理机构MA等合并设置。

6.1.7.6 认证授权机构

认证授权机构（Authentication and Authorization Authority, AAA）：负责V2X设备的认证和授权。AAA可向V2X设备签发安全凭证，以便V2X设备使用该安全凭证向证书机构申请数字证书，或者根据证书机构的授权请求提供V2X设备授权安全凭证或授权服务。

6.1.7.7 假名注册机构 PRA

假名证书注册机构PRA（Pseudonym Registration Authority, PRA）：接收OBU的假名证书申请，对OBU提供的假名证书种子密钥进行扩展，从LA获取相应的证书链接值；基于扩展的密钥和链接值生成假名证书生成请求并发送给假名证书CA；从假名CA获取OBU的假名证书并将其发送给OBU。

6.1.7.8 链接机构 LA

链接机构（Linkage Authority, LA）实现链接值供应功能，包括个体链接值和组链接值。接收PRA的链接值请求，验证请求的有效性，并生成链接值；接受MA的链接种子查询，验证请求的有效性，并返回MA所需的链接种子。在证书撤销前，链接值不会泄露多个假名证书属于同一个证书申请实体的信息。本标准使用单LA架构。链接值支持高效撤销、简化撤销流程、优化撤销性能，并为OBU提供可信的防跟踪能力。为保证链接值的不可链接性，可基于多机构共同管理的机制保障LA的可信。

6.1.7.9 中间 CA

中间CA（Intermediate CA, ICA）：位于根CA与签发注册证书、各种授权证书CA之间，用于扩展PKI体系的层级，实现CA的多级部署和多级管理。ICA所签发证书的种类取决于ICA证书的权限或由一个PKI系统的证书管理策略确定。

6.1.7.10 异常行为管理机构 MA

异常行为管理机构（Misbehavior Authority, MA）：基于V2X设备上报的异常行为检测报告，判断需要撤销的假名证书、应用证书和身份证书，并签发相应的证书撤销列表。

6.1.8 证书类型

6.1.8.1 概述

本标准所描述的数字证书统称为V2X通信证书，简称V2X证书。

基于V2X证书的用途，V2X证书可分为注册证书、假名证书、应用证书和身份证书。注册证书用于申请假名证书、应用证书和身份证书。假名证书、应用证书和身份证书用于V2X安全通信，统称为V2X授权证书。

6.1.8.2 注册证书

注册证书（Enrollment Certificate）由注册CA签发给OBU、RSU和VSP。OBU、RSU或VSP被注册机构认证后，注册CA会为其签发注册证书。注册证书与设备唯一对应。V2X设备需要使用注册证书从假名CA申请假名证书或从各应用授权机构（ACA）申请应用证书和身份证书。可选地，设备也可以使用认证授权机构签发的授权凭证从假名CA申请假名证书或从各应用授权机构（ACA）申请应用证书和身份证书。

6.1.8.3 假名证书

假名证书（Pseudonym Certificate）由假名CA签发给OBU。OBU使用假名证书签发其播发的主动安全消息（Basic Safety Message, BSM）。为保护用户隐私，需要使用密码技术对用户的身份信息进行加密；为避免泄露车辆行驶轨迹，OBU可拥有多个假名证书，用于定期切换使用。

6.1.8.4 应用证书

应用证书（Application Certificate）由应用CA签发给RSU和VSP，用于特定的车联网应用。RSU和VSP使用应用证书签发其播发的某种应用消息，例如交通信号灯状态、交通信息、商业服务消息等。针对某个特定的车联网应用，RSU或VSP只能拥有一个应用证书。

6.1.8.5 身份证书

身份证书（Identity Certificate）由应用CA签发给OBU，用于特定的车联网应用。OBU使用身份证书向RSU或VSP证明其身份，以获得后者提供的某种车联网应用服务，例如警车与红绿灯进行交互，并控制后者的状态。针对某个特定的车联网应用，OBU只能拥有一个身份证书。

6.1.9 证书一致性检查要求

6.1.9.1 概述

证书的许可描述证书所能签发的消息或签发的证书，具体可描述如下信息：

- 证书有效期：规定证书适用的时间范围。
- 证书适用地理区域：规定证书适用的地理范围。
- 应用数据签名权限：规定一个证书所能够签发的应用消息。
- CA证书签发权限：规定一个CA证书所能签发的证书种类和证书权限。
- 证书申请权限：规定一个证书能够申请的证书种类和证书权限。

证书许可权限由应用标识（Application Identifier, AID）和服务特定许可（Service Specific Permission, SSP）描述。服务提供方标识（AID）描述证书适用的应用领域。服务特定许可（SSP）描述证书在某特定AID下所拥有的具体权限或许可。AID的定义见YD/T 3707-2020。SSP的语法和语义特定于具体的AID，不属于本标准范围。

6.1.9.2 证书签发过程中一致性检查要求

证书申请方应执行证书一致性检查，只有通过证书一致性检查后才能向CA发送证书申请。证书申请方应至少执行如下证书一致性检查：

- 证书有效性检查：应检查证书撤销列表，确认CA证书没有被撤销；若使用注册证书申请其他证书，则应检查证书撤销列表，确认注册证书没有被撤销。
- 申请证书有效期检查：所申请证书的有效期不应超出CA证书的有效期。
- 申请证书地理区域检查：若申请注册证书，且CA证书包含有地理区域描述，则所申请证书的地理区域不应超出CA证书的地理区域描述；若使用注册证书申请其他证书，且该注册证书包含有地理区域描述，则所申请证书的地理区域描述不应超出注册证书的地理区域描述。
- 申请证书权限描述：若申请注册证书，则所申请证书的权限描述不应超出CA证书的权限描述；若使用注册证书申请其他证书，则所申请证书的权限描述不应超出注册证书的权限描述。

证书签发方应执行证书一致性检查,只有通过证书一致性检查后才能向证书申请方签发证书。证书签发方应至少执行如下证书一致性检查:

- 注册证书有效性检查:若证书申请方使用注册证书申请证书,则应检查证书撤销列表,确认该注册证书没有被撤销。
- 授权令牌有效性检查:若证书申请方使用授权令牌申请证书,则应检查该令牌是否有效。
- 申请证书有效期检查:所申请证书的有效期不应超出CA证书的有效期。
- 申请证书地理区域检查:若CA证书包含有地理区域描述,则所申请证书的地理区域不应超出CA证书的地理区域描述;若证书申请方使用注册证书申请证书,且该注册证书包含有地理区域描述,则所申请证书的地理区域描述不应超出注册证书的地理区域描述。
- 申请证书权限描述:所申请证书的权限描述不应超出CA证书的权限描述;若证书申请方使用注册证书申请证书,则所申请证书的权限描述不应超出注册证书的权限描述。

6.1.9.3 应用消息签名和验签过程中一致性检查要求

消息签名方应执行证书一致性检查,只有通过证书一致性检查后才能执行消息签名操作。消息签名方应至少进行如下证书一致性检查:

- 证书撤销列表检查:检查证书撤销列表,确定签名证书未被撤销。
- 签名证书有效期检查:检查签名证书仍在有效期内。
- 签名证书地理区域检查:若签名证书包含有地理区域描述,则当前的地理区域不应超出签名证书的地理区域描述。
- 签名证书权限描述:检查签名证书的权限描述,确定所签名消息未超出签名证书的权限描述。应用消息签名过程中的权限一致性检查由安全数据处理功能(SDPF)执行。

消息签名验证方接收到包含数字签名和签名证书的消息时,应针对签名证书,构建验证路径,构建方法可参考GB/T 16264.8,并对验证路径上的每个证书执行证书一致性检查,未能通过证书一致性检查的证书不能用于验证消息签名操作。消息签名验证方应至少进行如下证书一致性检查:

- 签名证书合法性检查:利用签发签名证书的CA证书验证签名证书的合法性,确定该签名证书由可信的CA签发。
- 证书撤销列表检查:检查证书撤销列表,确定签名证书未被撤销。
- 签名证书有效期检查:检查签名证书仍在有效期内。
- 签名证书地理区域检查:若签名证书包含有地理区域描述,则签名消息的地理区域不应超出签名证书的地理区域描述。
- 签名证书权限描述:检查签名证书的权限描述,确定所签名消息未超出签名证书的权限描述。应用消息验签过程中的权限一致性检查由安全数据处理功能(SDPF)执行。

6.2 LTE-V2X 安全证书

6.2.1 证书基本结构

在基于LTE的车联网中,V2X设备使用基于数字证书的应用层安全机制,发送方对V2X消息进行完整性和抗重放攻击保护、消息接收方对所接收到的V2X消息进行验签认证。表1为证书基本结构。

表1 证书基本结构

数据域 1	数据域 2	数据域 3	是否必选	说明
-------	-------	-------	------	----

版本		version	是	证书结构版本，本标准对应的版本号为 3
类型		type	是	证书结构类型 •显式证书 •其他结构的证书
签发者		issuer	是	自签证书或签发此证书的 CA 证书的 HashedId8 值
签名数据	toBeSigned	id	是	证书标识
		cracald	是	CRL-CA 标识 HashedId3，若不使用设置为全零
		crlSeries	是	CRL 序列号，若不使用设置为全零
		validityPeriod	是	有效期
		region	否	有效地理范围
		assuranceLevel	否	信任级别
		appPermissions	是	应用数据签名权限（例如 OBU/RSU 签名的应用消息类型）
		certIssuePermissions	否	适用于 CA 证书，描述可签发的证书种类和权限范围
		certRequestPermissions	否	适用于注册证书，描述可申请的证书种类、权限范围
		canRequestRollover	否	是否能够用于请求同等权限的证书。
		encryptionKey	否	加密公钥
		verifyKeyIndicator	是	验证公钥，采用其他结构的证书结构时，可以是相关数据。
签名值		signature	否	证书结构类型为显式证书时，此字段为必填字段，用于存储证书的签名值。

6.2.2 根 CA 证书

表2描述了根CA证书的结构和使用。

表2 根 CA 证书结构

数据域 1	数据域 2	数据域 3	是否必选	说明
版本		version	是	证书结构版本，本标准对应的版本号为 3
类型		type	是	证书结构类型：显式证书
签发者		issuer	是	自签证书
签名数据	toBeSigned	id	是	证书标识
		cracald	是	CRL-CA 标识 HashedId3，若不使用设置为全零
		crlSeries	是	CRL 序列号，若不使用设置为全零
		validityPeriod	是	有效期
		region	否	有效地理范围
		assuranceLevel	否	信任级别
		appPermissions	是	应用数据签名权限（例如 OBU/RSU 签名的应用消息类型）

		certIssuePermissions	是	适用于 CA 证书，描述可签发的证书种类和权限范围
		certRequestPermissions	否	适用于注册证书，描述可申请的证书种类、权限范围
		canRequestRollover	否	是否能够用于请求同等权限的证书
		encryptionKey	否	加密公钥
		verifyKeyIndicator	是	验证公钥
签名值		signature	是	签名值。

6.2.3 注册证书

表3描述注册证书的结构和使用。

表3 注册证书结构

数据域 1	数据域 2	数据域 3	是否必选	说明
版本		version	是	证书结构版本，本标准对应的版本号为 3
类型		type	是	证书结构类型：显式证书
签发者		issuer	是	签发此证书的 CA 证书的 HashedId8 值
签名数据	toBeSigned	id	是	证书标识
		cracald	是	CRL-CA 标识 HashedId3，若不使用设置为全零
		crlSeries	是	CRL 序列号，若不使用设置为全零
		validityPeriod	是	有效期
		region	否	有效地理范围
		assuranceLevel	否	信任级别
		appPermissions	是	应用数据签名权限（例如 OBU/RSU 签名的应用消息类型）
		certIssuePermissions	否	适用于 CA 证书，描述可签发的证书种类和权限范围
		certRequestPermissions	是	适用于注册证书，描述可申请的证书种类、权限范围
		canRequestRollover	否	是否能够用于请求同等权限的证书。
		encryptionKey	否	加密公钥
		verifyKeyIndicator	是	验证公钥
签名值		signature	是	签名值。

6.2.4 假名证书

表4描述假名证书的结构和使用。

表4 假名证书结构

数据域 1	数据域 2	数据域 3	是否必选	说明
版本		version	是	证书结构版本，本标准对应的版本号为 3
类型		type	是	证书结构类型：

				•显式证书 •其他结构的证书
签发者		issuer	是	签发此证书的 CA 证书的 HashedId8 值
签名数据	toBeSigned	id	是	证书标识
		cracald	是	CRL-CA 标识 HashedId3, 若不使用设置为全零
		crlSeries	是	CRL 序列号, 若不使用设置为全零
		validityPeriod	是	有效期
		region	否	有效地理范围
		assuranceLevel	否	信任级别
		appPermissions	是	应用数据签名权限（例如 OBU/RSU 签名的应用消息类型）
		certIssuePermissions	否	适用于 CA 证书, 描述可签发的证书种类和权限范围
		certRequestPermissions	否	适用于注册证书, 描述可申请的证书种类、权限范围
		canRequestRollover	否	是否能够用于请求同等权限的证书。
		encryptionKey	否	加密公钥
		verifyKeyIndicator	是	验证公钥
签名值		signature	否	证书结构类型为显式证书时, 此字段为必填字段, 用于存储证书的签名值。

6.2.5 应用证书和身份证书

表5描述应用证书和身份证书的结构和使用。

表5 应用证书结构

数据域 1	数据域 2	数据域 3	是否必选	说明
版本		version	是	证书结构版本, 本标准对应的版本号为 3
类型		type	是	证书结构类型: •显式证书 •其他结构的证书
签发者		issuer	是	签发此证书的 CA 证书的 HashedId8 值
签名数据	toBeSigned	id	是	证书标识
		cracald	是	CRL-CA 标识 HashedId3, 若不使用设置为全零
		crlSeries	是	CRL 序列号, 若不使用设置为全零
		validityPeriod	是	有效期
		region	否	有效地理范围
		assuranceLevel	否	信任级别
		appPermissions	是	应用数据签名权限（例如 OBU/RSU 签名的应用消息类型）
		certIssuePermissions	否	适用于 CA 证书, 描述可签发的证书种类和权限范围
		certRequestPermissions	否	适用于注册证书, 描述可申请的证书

		ns		种类、权限范围
		canRequestRollover	否	是否能够用于请求同等权限的证书。
		encryptionKey	否	加密公钥
		verifyKeyIndicator	是	验证公钥
签名值		signature	否	证书结构类型为显式证书时，此字段为必填字段，用于存储证书的签名值。

6.2.6 CRL 结构

表6描述CRL的结构和使用。CRL（CrlContents）内容的完整性保护，通过SecuredCrl结构体对CrlContent进行签名来实现，SecuredCrl的ASN.1结构见附录C.3.2 错误!未找到引用源。。

表6 CRL 结构

数据域 1	数据域 2	数据域 3	是否必选	说明
版本		version	是	version 是 CRL 的版本号，本标准对应的版本号是 1；
CRL 序列号		crlSeries	是	CRL 序列号
签发者		craCald	是	签发此证书的 CRL CA 的证书的 HashedId8 值
CRL 签发时间		issueDate	是	CRL 的发布时间
预期下次 CRL 签发时间		nextCrl	是	包含预期发出具有相同 crlSeries 和 crlCraca 的下一个 CRL 的时间。
		priorityInfo	是	所包含的信息可协助存储空间有限的设备确定要保留哪些撤销信息以及丢弃哪些撤销信息；
CRL 正文选择结构体	typeSpecific	fullHashCrl		包含一个完整的基于散列值的 CRL，即包含所有已撤销证书的散列列表
		deltaHashCrl		包含基于增量散列的 CRL，即所有已撤销证书的散列列表
		fullLinkedCrl		包含一个完整的基于 Linkage ID 的 CRL，即包含所有已撤销证书的个体和/或群组链接数据的列表
		deltaLinkedCrl		包含基于增量 Linkage ID 的 CRL，即包含所有已撤销证书的个体和/或群组链接数据的列表

6.3 基本元素说明

6.3.1 编码规则

本标准中定义的数据结构应符合GB/T 16262（所有部分）的要求，采用ISO/IEC 8825-7规定的正则八位字节编码规则(COER)对数字证书格式的各项信息进行编码。

6.3.2 数据结构定义

下面的原子类型在数据结构定义中使用：

```

Uint3  ::= INTEGER (0..7)           -- (hex)          07
Uint8  ::= INTEGER (0..255)         -- (hex)          ff
Uint16 ::= INTEGER (0..65535)       -- (hex)         ff ff
Uint32 ::= INTEGER (0..4294967295)  -- (hex)       ff ff ff ff
Uint64 ::= INTEGER (0..18446744073709551615)
                                           -- (hex) ff ff ff ff ff ff ff ff

IValue ::= Uint16

```

下面的八位字节字符串在数据结构定义中使用：

```
Opaque ::= OCTET STRING
```

下列结构用来阐明定义：

```

SequenceOfOctetString ::= SEQUENCE (SIZE (0..MAX)) OF
                           OCTET STRING (SIZE(0..MAX))
                           SequenceOfUint8 ::= SEQUENCE OF Uint8
                           SequenceOfUint16 ::= SEQUENCE OF Uint16

```

6.4 安全协议数据单元

6.4.1 V2XSecData

```

错误!未找到引用源。 ::= SEQUENCE {
    protocolVersion    Uint8(3),
    content             错误!未找到引用源。
}

```

该数据类型用于包含本章节中定义的其他数据类型。 V2XSecData中的字段具有以下含义：

- protocolVersion 指示协议的当前版本。 本标准定义的是版本 3，用整数‘3’表示。
- content 中包含 V2XSecContent 形式的内容。

6.4.2 V2XSecContent

```

错误!未找到引用源。 ::= CHOICE {
    unsecuredData      Opaque,
    signedData         SignedData,
    encryptedData      EncryptedData,
    signedCertificateRequest Opaque,
    ...
}

```

该数据结构中包含：

- unsecuredData 指示用八位字节串表征的无安全处理数据；
- signedData 指示按本标准进行了签名处理的数据；
- encryptedData 指示按本标准进行了加密处理的数据；

——signedCertificateRequest 指示所承载的内容为证书请求；

6.4.3 SignedData

```
SignedData ::= SEQUENCE {
    hashId          错误!书签自引用无效。 ,
    tbsData         ToBeSignedData,
    signer          SignerIdentifier,
    signature       Signature
}
```

该数据结构中包含：

——hashId 指示签名和验签时、对消息进行散列处理所使用的哈希算法；

——tbsData 包含散列后作为签名输入的数据；

——signer 确定对数据进行签名时所使用的密钥材料和哈希算法；

——signature 为数字签名

- 如果 signer 指示选择 self，则签名计算参数化如下：
 - ◆ Data（数据）输入等于 tbsData 字段的 COER 规范化编码；
 - ◆ Verification type（验签类型）等于 self；
 - ◆ Signer identifier（签名者标识符）输入等于空字符串。
- 如果 signer 指示 certificate 或 digest，则签名计算参数化如下：
 - ◆ Data（数据）输入等于 tbsData 字段的 COER 规范化编码；
 - ◆ Verification type（验签类型）输入 等于 certificate；
 - ◆ Signer identifier（签名者标识符）输入等于用于验证 SPDU 的证书的 COER 编码。

6.4.4 HashAlgorithm

```
SignedData ::= SEQUENCE {
    hashId          错误!书签自引用无效。 ,
    tbsData         ToBeSignedData,
    signer          SignerIdentifier,
    signature       Signature
}
```

该数据结构中包含：

——hashId 指示签名和验签时、对消息进行散列处理所使用的哈希算法；

——tbsData 包含散列后作为签名输入的数据；

——signer 确定对数据进行签名时所使用的密钥材料和哈希算法；

——signature 为数字签名

- 如果 signer 指示选择 self，则签名计算参数化如下：
 - ◆ Data（数据）输入等于 tbsData 字段的 COER 规范化编码；
 - ◆ Verification type（验签类型）等于 self；
 - ◆ Signer identifier（签名者标识符）输入等于空字符串。
- 如果 signer 指示 certificate 或 digest，则签名计算参数化如下：
 - ◆ Data（数据）输入等于 tbsData 字段的 COER 规范化编码；

- ◆ Verification type (验签类型)输入 等于 certificate;
- ◆ Signer identifier (签名者标识符)输入等于用于验证 SPDU 的证书的 COER 编码。

```
HashAlgorithm ::= ENUMERATED {
    sha256,
    ...,
    sha384,
    sm3
}
```

该数据结构指示哈希算法。

6.4.5 ToBeSignedData

```
ToBeSignedData ::= SEQUENCE {
    payload          SignedDataPayload,
    headerInfo       HeaderInfo
}
```

该数据结构包含生成或验证签名时要进行哈希处理的数据：

- payload 包含调用实体所提供的数据；
- headerInfo 包含插入的附加数据；

6.4.6 SignedDataPayload

```
SignedDataPayload ::= SEQUENCE {
    data              V2XSecData OPTIONAL,
    extDataHash       HashedData OPTIONAL,
    ...
}
(WITH COMPONENTS {..., data PRESENT} |
 WITH COMPONENTS {..., extDataHash PRESENT})
```

该数据结构包含ToBeSignedData的数据载荷。此结构至少包含data和extDataHash中的一个，有可能两者都包含：

- Data（数据）承载在数据结构中明确传输的数据；
- extDataHash（扩展数据哈希）包含未在数据结构中显式传输的数据的哈希值，并且数据结构的创建者希望以安全方式将它绑定到签名。例如，如果创建者想要指示某个大消息仍然有效，则他们可以使用 extDataHash 字段发送包含该大消息散列值的 SignedData，而不必重新发送消息本身。

6.4.7 HashedData

```
HashedData ::= CHOICE {
    sha256HashedData OCTET STRING (SIZE(32)),
    ...,
    sha384HashedData OCTET STRING (SIZE(48)),
    sm3HashedData OCTET STRING (SIZE(32))
}
```

}

此数据结构中包含使用指定哈希算法的某些数据的哈希值。

6.4.8 Header Info

```
HeaderInfo ::= SEQUENCE {
    aid                      Aid,
    generationTime          本标准中采用十进制方式对AID取值进行编码。
Time64 OPTIONAL,
    expiryTime              本标准中采用十进制方式对AID取值进行编码。
Time64 OPTIONAL,
    generationLocation      ThreeDLocation OPTIONAL,
    p2pcdLearningRequest    HashedId3 OPTIONAL,
    missingCrlIdentifier     MissingCrlIdentifier OPTIONAL,
    encryptionKey           EncryptionKey OPTIONAL,
    ...
    inlineP2pcdRequest      SequenceOfHashedId3 OPTIONAL,
    requestedCertificate     Certificate OPTIONAL,
    pduFunctionalType        PduFunctionalType OPTIONAL
}
```

此数据结构中包含如下信息：

- aid 指示发送方宣称的与有效负载相关联的应用领域；
- generationTime 指示生成数据结构的时间；
- expiryTime（过期时间），如果存在，包含数据不再被视为有效的时间。如果 generationTime 和 expiryTime 都存在，且 generationTime 不早于 expiryTime，则签名的 SPDU 无效；
- generationLocation，如果存在，包含生成签名的位置信息；
- p2pcdLearningRequest，如果存在，用于请求已获知标识符但不知道整个证书内容的证书。该信息字段仅在 inlineP2pcdRequest 不存在时才出现；
- missingCrlIdentifier，如果存在，被用于请求已知存在但尚未收到的 CRL；
- encryptionKey，如果存在，指示用于加密后续通信的加密密钥。仅当包含此字段的 SignedData 已通过某种方式安全加密时，才应使用对称类型的 encryptionKey 字段；
- inlineP2pcdRequest，如果存在，用于请求未知的证书。该信息字段仅在 p2pcdLearningRequest 不存在时才出现；
- requestedCertificate，如果存在，用于按 P2PCD（P2P 证书分发）机制提供证书；
- pduFunctionalType，如果存在，用于指示 SPDU 将由应用程序进程以外的进程使用；

6.4.9 Aid

Aid ::= INTEGER (0..MAX)

该数据结构指示所应用的领域。

本标准中采用十进制方式对AID取值进行编码。

6.4.10 Time64

本标准中采用十进制方式对AID取值进行编码。

Time64 ::= UInt64

该数据结构是64位整数，给出了自2004年1月1日00:00:00 UTC以来微秒数（TAI）的估计值。

6.4.11 ThreeDLocation

```
ThreeDLocation ::= SEQUENCE {
    latitude      Latitude,
    longitude     Longitude,
    elevation     Elevation
}
```

该数据结构包含一个3D位置的估计值，细节在后续字段的定义中给出。

6.4.12 Latitude

```
Latitude ::= NinetyDegreeInt
NinetyDegreeInt ::= INTEGER {
    min                ..... (-900000000),
    max                ..... (900000000),
    unknown            (900000001)
} (-900000000..900000001)
```

```
KnownLatitude ::= NinetyDegreeInt (min..max)
-- Minus 90deg to +90deg in .1 microdegree intervals
UnknownLatitude ::= NinetyDegreeInt (unknown)
```

latitude（纬度）字段为纬度估计值，它为整数类型，精度为1/10微度。

纬度字段中的整数不超过900 000 000且不小于-900 000 000，一个例外是900 000 001被用于指示发送方无法获得纬度信息。

6.4.13 Longitude

```
Longitude ::= OneEightyDegreeInt
OneEightyDegreeInt ::= INTEGER {
    min                (-1799999999),
    max                (1800000000),
    unknown            (1800000001)
} (-1799999999..1800000001)
```

```
KnownLongitude ::= OneEightyDegreeInt (min..max)
UnknownLongitude ::= OneEightyDegreeInt (unknown)
```

longitude（经度）字段为经度估计值，它为整数类型，精度为1/10微度。

经度字段中的整数不超过1 800 000 000且不小于-1 799 999 999，一个例外是1 800 000 001被用于指示发送方无法获得经度信息。

6.4.14 Elevation

Elevation ::= ElevInt

ElevInt ::= UInt16

Elevation（高程）字段为高度估计值。它为16比特值，指示高于最小高度-409.5米的高度值，单位为分米，最大可表达高度为6143.9米。取值65535被用于指示发送方无法获得高度信息。

6.4.15 MissingCrlIdentifier

```
MissingCrlIdentifier ::= SEQUENCE {
    cracaId      HashedId3,
    crlSeries    CrlSeries,
    ...
}
```

该数据结构可用于请求知道已经发布但尚未接收到的CRL：

——cracaId 是 CRA 的 HashedId3 值；

——crlSeries 是所请求的 CRL 序列值。

6.4.16 CrlSeries

CrlSeries ::= UInt16

这个整形数据结构标识在特定CRACA授权下发布的一系列CRL。

6.4.17 EncryptionKey

```
EncryptionKey ::= CHOICE {
    public          PublicEncryptionKey,
    symmetric       SymmetricEncryptionKey
}
```

该数据结构包含一个加密密钥，它可能是公钥或对称密钥。

6.4.18 SymmetricEncryptionKey

```
SymmetricEncryptionKey ::= CHOICE {
    aes128Ccm      OCTET STRING(SIZE(16)),
    ...,
    sm4Ccm         OCTET STRING(SIZE(16))
}
```

该数据结构提供了与所指示的对称加密算法一起使用的密钥字节。本标准所支持的对称加密算法是CCM模式下的SM4等算法。

6.4.19 PublicKey

```
PublicKey ::= SEQUENCE {
    supportedSymmAlg    SymmAlgorithm,
    publicKey           BasePublicEncryptionKey
}
```

该数据结构指定公钥和关联的对称加密算法，该算法在加密该公钥时对数据进行批量加密。

6.4.20 SymmAlgorithm

```

SymmAlgorithm ::= ENUMERATED {
    aes128Ccm,
    ...,
    sm4Ccm
}

```

该枚举值指示所支持的对称算法及其操作模式。

6.4.21 BasePublicEncryptionKey

```

BasePublicEncryptionKey ::= CHOICE {
    eciesNistP256      EccP256CurvePoint,
    eciesBrainpoolP256r1 EccP256CurvePoint,
    ...,
    ecencSm2          EccP256CurvePoint
}

```

该数据结构指示特定算法的加密公钥。

6.4.22 EccP256CurvePoint

```

EccP256CurvePoint ::= CHOICE {
    x-only          OCTET STRING (SIZE (32)),
    fill            NULL, — consistency w 1363 / X9.62
    compressed-y-0  OCTET STRING (SIZE (32)),
    compressed-y-1  OCTET STRING (SIZE (32)),
    uncompressedP256 SEQUENCE {
        x OCTET STRING (SIZE (32)),
        y OCTET STRING (SIZE (32))
    }
}

```

该数据结构指示在256位素数上定义的Weierstrass形式的椭圆曲线上的点。这包括SM2等算法中定义的椭圆曲线点。该结构中的字段使用IEEE Std 1363-2000的5.5.6节中所定义的椭圆曲线点编码和解码方法来产生的八位字节串。对于CHOICE的所有值，x坐标以网络字节顺序编码成长度为32个八位字节的无符号整数；y坐标的编码取决于该点是仅x，压缩还是未压缩。如果该点是仅x，则省略y；如果该点被压缩，则类型的值取决于y的最低有效位。如果y的最低有效位为0，则类型取值为compressed-y-0，如果y的最低有效位为1，则类型取值compressed-y-1；如果该点未压缩，则y被明确编码为按网络字节顺序的长度为32个八位字节的无符号整数。

6.4.23 EccP384CurvePoint

```

EccP384CurvePoint ::= CHOICE {
    x-only          OCTET STRING (SIZE (48)),
    fill            NULL, — consistency w 1363 / X9.62
    compressed-y-0  OCTET STRING (SIZE (48)),
    compressed-y-1  OCTET STRING (SIZE (48)),
    uncompressedP384 SEQUENCE {

```

```

        x OCTET STRING (SIZE (48)),
        y OCTET STRING (SIZE (48))
    }
}

```

该数据结构指示在384位素数上定义的Weierstrass形式的椭圆曲线上的点。该结构中的字段使用IEEE Std 1363-2000的5.5.6节中所定义的椭圆曲线点编码和解码方法、来产生八位字节串。对于CHOICE的所有值，x坐标以网络字节顺序编码成长度为48个八位字节的无符号整数；y坐标的编码取决于该点是仅x、压缩还是未压缩。如果该点是仅x，则省略y；如果该点被压缩，则类型的值取决于y的最低有效位。如果y的最低有效位为0，则类型取值为compressed-y-0，如果y的最低有效位为1，则类型取值compressed-y-1；如果该点未压缩，则y被明确编码为按网络字节顺序的长度为48个八位字节的无符号整数。

6.4.24 PduFunctionalType

```

PduFunctionalType ::= INTEGER (0..255)
tlsHandshake       PduFunctionalType ::= 1
iso21177ExtendedAuth PduFunctionalType ::= 2

```

该数据结构标识了旨在使用SPDU的功能实体。对于包含有PduFunctionalType的SPDU，它应符合PduFunctionalType值所对应的安全属性，而不是AID的应用SPDU安全属性。该数据结构包含：

- tlsHandshake 指示签名的 SPDU 不能直接作为应用 PDU 使用，而是用于提供有关持有者对传输层安全（TLS）握手过程的权限的信息，该过程用于保护与应用程序进程的通信；
- iso21177ExtendedAuth 指示签名的 SPDU 不能直接作为应用 PDU 使用，而是用于提供有关持有者对应用程序进程中的 ISO 21177 安全子系统权限的附加信息。

6.4.25 SignerIdentifier

```

SignerIdentifier ::= CHOICE {
    digest          HashedId8,
    certificate     SequenceOfCertificate,
    self           NULL,
    ...,
    x509           OCTET STRING
}

```

该数据结构指示数据接收方应使用哪种密钥材料来认证数据。它还指示用于生成验证哈希的验证类型。

- 如果指示的选项是 digest（摘要）
 - 该数据结构包含相关证书的 HashedId8 值。HashedId8 使用证书的全部数据作为哈希算法的输入计算得到；
 - 验证类型是 certificate（证书）时，输入到哈希散列函数的证书数据是授权证书。
- 如果指示的选项是 certificate（证书）：
 - 该数据结构包含一个或多个证书结构，顺序是第一个证书为授权证书，每个后续证书是其前一个证书的签发者的证书。
 - 验证类型是 certificate（证书）时，输入到哈希散列函数的证书数据是授权证书。

——如果指示的选项是 self:

- 该数据结构不包含任何除 self 指示外的数据。
- 验证类型是自签名。

6.4.26 HashedId3

HashedId3 ::= OCTET STRING (SIZE(3))

SequenceOfHashedId3 ::= SEQUENCE OF HashedId3

该数据结构包含另一个数据结构的截断哈希值。给定数据结构的HashedId3通过计算编码数据结构的散列值、并获取散列输出的低位三个字节来获得。当以网络字节顺序表示时，低位三字节是散列值的最后三个字节。

用于计算给定数据结构的HashedId3值的哈希算法取决于上下文。在本标准中，对于包含HashedId3字段的每一个数据结构，均有相应的文字说明如何确定哈希算法。

6.4.27 HashedId8

HashedId8 ::= OCTET STRING (SIZE(8))

该数据结构包含另一个数据结构的截断哈希值。给定数据结构的HashedId8通过计算编码数据结构的散列值、并获取散列输出的低位8个字节来获得。当以网络字节顺序表示时，低位8个字节是散列值的最后8个字节。

用于计算给定数据结构的HashedId8值的哈希算法取决于上下文。在本标准中，对于包含HashedId8字段的每一个数据结构，均有相应的文字说明如何确定哈希算法。

6.4.28 HashedId10

HashedId10 ::= OCTET STRING (SIZE(10))

该数据结构包含另一个数据结构的截断哈希值。给定数据结构的HashedId10通过计算编码数据结构的散列值、并获取散列输出的低位10个字节来获得。当以网络字节顺序表示时，低位10个字节是散列值的最后10个字节。

用于计算给定数据结构的HashedId10值的哈希算法取决于上下文。在本标准中，对于包含HashedId10字段的每一个数据结构，均有相应的文字说明如何确定哈希算法。

6.4.29 Signature

```
Signature ::= CHOICE {
    ecdsaNistP256Signature      EcdsaP256Signature,
    ecdsaBrainpoolP256r1Signature EcdsaP256Signature,
    ...,
    ecdsaBrainpoolP384r1Signature EcdsaP384Signature,
    ecdsaNistP384Signature      EcdsaP384Signature,
    sm2Signature                 EcsigP256Signature
}
```

该数据结构表征对应于一个公钥算法的签名，它可能被包含在SignedData 或 Certificate数据结构中。

6.4.30 EcdsaP256Signature

```
EcdsaP256Signature ::= SEQUENCE {
    rSig      EccP256CurvePoint,
    sSig      OCTET STRING (SIZE (32))
}
```

该数据结构表示基于256bit椭圆曲线的ECDSA签名。

如果对此数据结构编码是为了进行哈希运算，则rSig中的EccP256CurvePoint应采用x-only格式。

6.4.31 EcdsaP384Signature

```
EcdsaP384Signature ::= SEQUENCE {
    rSig      EccP384CurvePoint,
    sSig      OCTET STRING (SIZE (48))
}
```

该数据结构表示基于384bit椭圆曲线的签名。

如果对此数据结构编码是为了进行哈希运算，则rSig中的EccP384CurvePoint应采用x-only格式。

6.4.32 EcsigP256Signature

```
EcsigP256Signature ::= SEQUENCE {
    rSig      OCTET STRING (SIZE (32)),
    sSig      OCTET STRING (SIZE (32))
}
```

该数据结构表示基于256bit椭圆曲线SM2算法的签名。

6.4.33 EncryptedData

```
EncryptedData ::= SEQUENCE {
    recipients      SequenceOfRecipientInfo,
    ciphertext      SymmetricCiphertext
}
```

该数据结构用于向一个或多个接收方发送加密数据，加密使用接收方的加密公钥或对称密钥。它包含以下字段：

- recipients 包含一个或多个 RecipientInfo。这些条目可以是多个 RecipientInfo 和多种类型的 RecipientInfo，只要它们都指示或包含相同的数据加密密钥；
- ciphertext 包含加密数据，它是对错误!未找到引用源。的加密。

6.4.34 RecipientInfo

```
RecipientInfo ::= CHOICE {
    pskRecipInfo      PreSharedKeyRecipientInfo,
    symmRecipInfo      SymmRecipientInfo,
    certRecipInfo      PKRecipientInfo,
}
```

```

signedDataRecipInfo PKRecipientInfo,
rekRecipInfo        PKRecipientInfo
}

```

SequenceOfRecipientInfo ::= SEQUENCE OF RecipientInfo

该数据结构用于将数据加密密钥传输到EncryptedData的单个接收方。如果使用静态密钥方法加密EncryptedData，则选择选项pskRecipInfo；如果使用短暂密钥方法加密EncryptedData，则选择其他选项。这些选项的含义是：

- pskRecipInfo：使用预置的对称密钥直接加密数据；
- symmRecipInfo：数据加密密钥被一个对称密钥加密；
- certRecipInfo：数据加密密钥使用公钥加密方案加密，其中公共加密密钥是从证书获得的。
- signedDataRecipInfo：数据加密密钥使用公钥加密，其中加密密钥是从 SignedData 获得的公共响应加密密钥。
- rekRecipInfo：数据加密密钥使用公钥加密，但公钥不是来自 SignedData。在这种情况下，ECIES 中的 P1 参数使用空字符串对应的散列值。

6.4.35 PreSharedKeyRecipientInfo

PreSharedKeyRecipientInfo ::= HashedId8

该数据结构用于确定可以直接解密 SymmetricCiphertext 的对称密钥。它由 SymmetricEncryptionKey 中所包含的对称密钥的COER编码哈希值的低8位字节组成。对称密钥可使用交互双方同意的任何适当方式提前建立。

6.4.36 SymmRecipientInfo

```

SymmRecipientInfo ::= SEQUENCE {
    recipientId      HashedId8,
    encKey           SymmetricCiphertext
}

```

该数据结构包含以下字段：

- recipientId 包含可用于解密数据加密密钥的对称密钥的散列值。它由包含这个对称密钥的 SymmetricEncryptionKey 的 COER 编码哈希值的低 8 位字节组成。对称密钥可使用交互双方同意的任何适当方式提前建立。
- encKey 包含密文，密文中有被加密的数据加密密钥。

6.4.37 PKRecipientInfo

```

PKRecipientInfo ::= SEQUENCE {
    recipientId      HashedId8,
    encKey           EncryptedDataEncryptionKey
}

```

该数据结构包含以下字段：

- recipientId：它包含 RecipientInfo 定义中所指的加密公钥的哈希值，其具体内容取决于 RecipientInfo 结构中所指示的选项：

- 如果所包含的 RecipientInfo 指示 certRecipInfo, 则此字段包含证书的 HashedId8。HashedId8 使用全证书运用哈希算法计算得到;
- 如果所包含的 RecipientInfo 结构指示 signedDataRecipInfo, 则此字段包含签名类型的、含加密密钥的 V2XSecData 的 HashedId8 值, HashedId8 使用所选定的哈希算法计算得到。
- 如果所包含的 RecipientInfo 结构指示 rekRecipInfo, 则此字段包含响应加密密钥的 PublicEncryptionKey 结构的 COER 编码的 HashedId8 值。HashedId8 使用所选定的哈希算法计算得到。

——encKey 包含加密密钥;

6.4.38 EncryptedDataEncryptionKey

```
EncryptedDataEncryptionKey ::= CHOICE {
    eciesNistP256          EciesP256EncryptedKey,
    eciesBrainpoolP256r1  EciesP256EncryptedKey,
    ...,
    ecencSm2              EcencP256EncryptedKey
}
```

该数据结构包含一个已加密的数据加密密钥。

6.4.39 EciesP256EncryptedKey

```
EciesP256EncryptedKey ::= SEQUENCE {
    v          EccP256CurvePoint,
    c          OCTET STRING (SIZE (16)),
    t          OCTET STRING (SIZE (16))
}
```

该数据结构用于传输已使用 IEEE Std 1363a-2004 中规定的 ECIES 进行加密的 16 字节对称密钥。该类型包含以下字段:

- v 是发送方的短暂公钥;
- c 是已加密的对称密钥, 对称密钥的算法由 SymmetricCiphertext 来指示;
- t 是验证标签。

6.4.40 EcencP256EncryptedKey

```
EcencP256EncryptedKey ::= SEQUENCE {
    v          EccP256CurvePoint,
    t          OCTET STRING (SIZE (32)),
    c          OCTET STRING (SIZE (16))
}
```

该数据结构用于传输使用 SM2 时的 16 字节对称加密密钥, 它包含以下字段:

- v 是发送方的短暂公钥;
- t 是验证标签。
- c 是已加密的对称密钥。对称密钥的算法由 SymmetricCiphertext 来指示。对于 SM2, 应使用 SM4 算法;

6.4.41 SymmetricCiphertext

```
SymmetricCiphertext ::= CHOICE {
    aes128ccm          One28BitCcmCiphertext,
    ...,
    sm4Ccm             One28BitCcmCiphertext
}
```

该数据结构封装使用被核准的对称加密算法生成的密文。

6.4.42 One28BitCcmCiphertext

```
One28BitCcmCiphertext ::= SEQUENCE {
    nonce              OCTET STRING (SIZE (12)),
    ccmCiphertext      Opaque
}
```

该数据结构封装CCM模式下对称算法(SM4或AES-128)的加密密文。由于包含消息认证码,密文比相应的明文长16个字节。

6.5 数字证书和证书管理数据格式

6.5.1 概述

6.4 定义的基本数据结构元素适用于V2X车联网数字证书和管理, **错误!未找到引用源。**定义具体的证书格式。

6.5.2 Certificate

```
Certificate ::=
    CertificateBase (ExplicitCertificate | Reserved)
```

```
SequenceOfCertificate ::= SEQUENCE OF Certificate
```

为CertificateBase结构的数据,它指示了传输隐式证书或显式证书或其他证书的有效字段组合。

6.5.3 CertificateBase

```
CertificateBase ::= SEQUENCE {
    version              Uint8(3),
    type                 CertificateType,
    issuer               IssuerIdentifier,
    toBeSigned           ToBeSignedCertificate,
    signature            Signature OPTIONAL
}
```

该数据结构包含:

- version: 是证书格式的版本号,此字段应设置为“3”;
- type: 指示证书是显式的或隐式的还是其他的。对于显式证书,此字段应设置为 explicit,对于隐式证书或其他,此字段应设置为 reserved;

- issuer**: 证书的签发方;
- toBeSigned**: 证书的内容。对于显式证书, 在生成或校证书签名时, 该字段是哈希函数的输入; 对于隐式证书, 在基于重构值生成或校验公钥时, 该字段是哈希函数的输入。该字段的编码细节见 6.5.7 ToBeSignedCertificate;
- signature**: 包含在 ExplicitCertificate 中。它由 issuer 字段中标识的签名者在 toBeSigned 的散列上计算的签名。哈希计算时使用:
 - Data 输入是对 toBeSigned 字段的 COER 编码;
 - Signer identifier 输入取决于验证类型, 而验证类型又取决于 issuer (发行者) 指示的选择。如果 issuer (发行者) 指示的是 self, 则验证类型是自签名的, signer identifier (签名者标识符) 输入是空字符串; 如果 issuer (发行者) 指示的选择不是 self, 则验证类型是证书, signer identifier (签名者标识符) 输入是发行者指示的证书的规范化 COER 编码。
 - 对于所有椭圆曲线上的点, toBeSigned 采用 LSB 方式的压缩编码, 也即这些点选择 compressed-y-0 或 compressed-y-1。

6.5.4 CertificateType

```
CertificateType ::= ENUMERATED {
    explicit,
    reserved,
    .....
}
```

此枚举类型指示是显式证书或其他类型证书。

6.5.5 ExplicitCertificate

```
ExplicitCertificate ::= CertificateBase (WITH COMPONENTS {...,
    type(explicit),
    toBeSigned(WITH COMPONENTS {...,
        verifyKeyIndicator(WITH COMPONENTS {verificationKey})
    }),
    signature PRESENT
})
```

是CertificateBase结构的数据, 包含显式证书所需的所有字段。

6.5.6 IssuerIdentifier

```
IssuerIdentifier ::= CHOICE {
    sha256AndDigest      HashedId8,
    self                  SignedData ::= SEQUENCE {
    hashId                错误!书签自引用无效。 ,
    tbsData               ToBeSignedData,
    signer                SignerIdentifier,
    signature              Signature
    }
```

该数据结构中包含：

——hashId 指示签名和验签时、对消息进行散列处理所使用的哈希算法；

——tbsData 包含散列后作为签名输入的数据；

——signer 确定对数据进行签名时所使用的密钥材料和哈希算法；

——signature 为数字签名

- 如果 signer 指示选择 self，则签名计算参数化如下：
 - ◆ Data（数据）输入等于 tbsData 字段的 COER 规范化编码；
 - ◆ Verification type（验签类型）等于 self；
 - ◆ Signer identifier（签名者标识符）输入等于空字符串。
- 如果 signer 指示 certificate 或 digest，则签名计算参数化如下：
 - ◆ Data（数据）输入等于 tbsData 字段的 COER 规范化编码；
 - ◆ Verification type（验签类型）输入 等于 certificate；
 - ◆ Signer identifier（签名者标识符）输入等于用于验证 SPDU 的证书的 COER 编码。

HashAlgorithm,

```

    ...,
    sha384AndDigest      HashedId8,
    sm3AndDigest         HashedId8
  }

```

证书的接收方通过这个字段来确定使用哪种密钥材料来验证证书。

如果指示的选项是sha256AndDigest或sha384AndDigest或sm3AndDigest，则：

——该数据结构包含签发证书的 HashedId8，它使用证书的所有字段经哈希散列算法计算得到，具体见附录 D；

——用于生成证书哈希值以进行验证的哈希算法包括 SM3（在 sm3AndDigest 的情况下）等。

——证书应使用所指示的签发证书的公钥进行验证。

如果指示的选项是self，则：

——该数据结构指示要使用哪种哈希算法生成证书的哈希值以进行验证。

——证书将使用 6.5.7 ToBeSignedCertificate 中包含的 verifyKeyIndicator 字段所指示的公钥进行验证。

6.5.7 ToBeSignedCertificate

```

ToBeSignedCertificate ::= SEQUENCE {
    id                      CertificateId,
    cracaId                 HashedId3,
    crlSeries              CrlSeries,
    validityPeriod         ValidityPeriod,
    region                 GeographicRegion OPTIONAL,
    assuranceLevel         SubjectAssurance OPTIONAL,
    appPermissions         SequenceOfAidSsp OPTIONAL,
    certIssuePermissions   SequenceOfAidGroupPermissions OPTIONAL,
    certRequestPermissions SequenceOfAidGroupPermissions OPTIONAL,
    canRequestRollover     NULL OPTIONAL,
    encryptionKey          PublicEncryptionKey OPTIONAL,

```

```

        verifyKeyIndicator      VerificationKeyIndicator,
        ...
    }
    (WITH COMPONENTS { ..., appPermissions PRESENT} |
     WITH COMPONENTS { ..., certIssuePermissions PRESENT} |
     WITH COMPONENTS { ..., certRequestPermissions PRESENT})

```

ToBeSignedCertificate结构中的字段具有以下含义：

- id 包含在必要时识别证书持有者的信息；.
- cracaId 用于识别负责发布证书撤销列表（CRL）的证书撤销授权 CA（CRA），它的证书可能出现在证书撤销列表（CRL）上。HashedId3 使用全证书哈希算法计算，如附录 D 所述；
- crlSeries 表示与证书可能出现的特定证书撤销授权 CA（CRACA）相关的 CRL 系列；
- validityPeriod 包含证书的有效期；
- region（如果存在）表示证书的有效区域。如果省略，则有效区域表示如下：
 - 如果证书是自签名的，即证书结构中的签发者字段指示的选项是 self，则证书在全球范围内有效；.
 - 否则，证书与签发证书的证书具有相同的有效区域；.
- assuranceLevel 表示证书持有者的保证级别；
- appPermissions 表示证书持有者使用此证书签署应用程序数据的权限。appPermissions 的有效实例最多包含一个条目的任何特定 Aid 值。
- certIssuePermissions 表示证书持有者使用此证书签署其他证书的权限。此数组通常不包含 aidsRange 字段指示“all”的多个条目。如果数组有多个条目，并且一个条目的 aidsrange 字段显示为“all”，则指示“all”的那个条目将指定除在其他条目中明确指定以外的所有 aid 的权限。具体请参考 AidGroupPermissions 中的描述；
- certRequestPermissions 表示证书持有者使用此证书签署证书请求的权限。此数组通常不包含 aidsRange 字段指示“all”的多个条目。如果数组有多个条目且一个条目的 aidSspRange 字段指示“all”，则表示“all”的那个条目指定除了在其他条目中明确指定的 AID 之外的所有 AID 的权限。具体请参考 AidGroupPermissions 中的说明；
- canRequestRollover 表示该证书可用于签署请求具有相同权限的另一个证书的请求消息；
- encryptionKey 包含用于加密的公钥，证书持有者持有相应的私钥。
- verifyKeyIndicator 包含可用于恢复公钥的材料，所恢复的公钥被用于验证被证书签名的数据。

下级证书的应用权限appPermissions或证书签发权限certIssuePermissions，与其签发证书的证书签发权限certIssuePermissions一致，即对于下级证书的应用权限或证书签发权限中的每条AID+SSP组合（以下称为“下级条目”），签发证书的证书签发权限中都有一个条目（以下称为“签发条目”）与其对应，具体包括：

- a) 下级条目中的 AID 与签发条目中的 AID 相同；
- b) 下级条目中的 SSP 的范围，等于签发条目中的 SSP 范围，或等于签发条目中 SSP 范围的子集；
- c) 如果下级条目是证书签发权限 certIssuePermissions 时，下级条目中允许的证书链长度应和签发条目中的证书链长度保持一致，具体见 6.5.30 AidGroupPermissions；
- d) 签发条目的 eeType 字段许可了下级条目的 eeType 字段，具体见 6.5.30 AidGroupPermissions。

6.5.8 CertificateId

```
CertificateId ::= CHOICE {
    linkageData      LinkageData,
    name             Hostname,
    binaryId         OCTET STRING (SIZE(1..64)),
    none            NULL,
    ...
}
```

此数据结构包含在必要时用于识别证书持有者的信息。

- linkageData 用于识别证书之注销目的，如果该证书出现在链接证书 CRL 中；
- name（名称）用于在非匿名证书的情况下标识证书持有者，它应该是人类可读的；
- binaryId 支持非人类可读的标识符；
- none 表示证书不包含标识符；。

6.5.9 LinkageData

```
LinkageData ::= SEQUENCE {
    iCert          IValue,
    linkage-value   LinkageValue,
    group-linkage-value GroupLinkageValue OPTIONAL
}
```

此数据结构包含与从基于链接ID的CRL获得的信息相匹配的信息，以确定所包含的证书是否已被撤销。

6.5.10 LinkageValue

```
LinkageValue ::= OCTET STRING (SIZE(9))
```

这是各自的链接值。

6.5.11 GroupLinkageValue

```
GroupLinkageValue ::= SEQUENCE {
    jValue OCTET STRING (SIZE(4)),
    value  OCTET STRING (SIZE(9))
}
```

这是群组链接值。

6.5.12 Hostname

```
Hostname ::= UTF8String (SIZE(0..255))
```

按IETF RFC 3629定义的UTF-8进行编码的字符串。

6.5.13 ValidityPeriod

```
ValidityPeriod ::= SEQUENCE {
    start      Time32,
    duration   Duration
}
```

}

该数据类型给出证书的有效期。有效期的开始时间由start指定，结束时间由start + duration 决定。

6.5.14 Time32

Time32 ::= Uint32

该数据类型给出自2004年1月1日00:00:00 UTC以来的秒数（TAI）。。

6.5.15 Duration

```
Duration ::= CHOICE {
    microseconds    Uint16,
    milliseconds    Uint16,
    seconds          Uint16,
    minutes          Uint16,
    hours            Uint16,
    sixtyHours       Uint16,
    years            Uint16
}
```

此数据类型表示证书的有效期。 Uint16值是持续时间，单位由CHOICE选项指示，其中单位“一年”等于31556952秒，这是一年中的平均秒数。

6.5.16 GeographicRegion

```
GeographicRegion ::= CHOICE {
    circularRegion    CircularRegion,
    rectangularRegion SequenceOfRectangularRegion,
    polygonalRegion   PolygonalRegion,
    identifiedRegion  SequenceOfIdentifiedRegion,
    ...
}
```

此数据类型表征特定形式的地理区域：

- rectangularRegion 是包含至少一个 RectangularRegion 数据结构条目的数组。该字段表征一系列矩形，它们可以重叠或不相交。被允许的区域是任意这些矩形内的任何点；
- circularRegion 或 polygonRegion 包含其各自类型的单个实例；
- identifiedRegion 是包含至少一个 IdentifiedRegion 数据结构条目的数组。被允许区域是任何这些已识别区域内的任何点。

如果证书范围字段中指明的区域的任何部分位于其签发者范围内指示的区域之外，则证书无效。

6.5.17 CircularRegion

```
CircularRegion ::= SEQUENCE {
    center    TwoDLocation,
    radius    Uint16
}
```

此数据结构指定一个圆心在Center、半径以米为单位、与参考椭球相切的圆。所示区域为参考椭球表面上的所有点，其与参考椭球中心点的距离小于或等于半径。如果一个包含高程分量的点的水平投影落到参考椭球面上，则该点被视为在圆形区域内。

6.5.18 TwoDLocation

```
TwoDLocation ::= SEQUENCE {
    latitude      Latitude,
    longitude     Longitude
}
```

此数据结构用于定义证书的有效使用区域。

注：该数据结构与SAE J2735中使用的位置编码方法一致，除了纬度值900,000 001（用于表示纬度不可用）和经度值1 800 000 001（用于表示经度不可用）无效以外。

6.5.19 RectangularRegion

```
RectangularRegion ::= SEQUENCE {
    northWest      TwoDLocation,
    southEast      TwoDLocation
}
```

```
SequenceOfRectangularRegion ::= SEQUENCE OF RectangularRegion
```

此数据结构指示按顺序连接四个点而形成的矩形：（northWest.latitude，northWest.longitude）、（southEast.latitude，northWest.longitude）、（southEast.latitude，southEast.longitude）和（northWest.latitude，southEast.longitude）。这些点通过恒定纬度或经度的线连接。包含高程分量的点，只要它对参考椭球面的水平投影位于矩形内，则认为该点在矩形区域内。只有当northWest所指示的点位于southEast点的西北方向时，RectangularRegion才有效，也即这两个点不能具有相等的纬度或相等的经度值。

6.5.20 PolygonalRegion

```
PolygonalRegion ::= SEQUENCE SIZE(3..MAX) OF TwoDLocation
```

该数据结构使用在参考椭球的表面上定义的一系列不同的地理点来定义区域。通过按照它们出现的顺序连接点来指定区域，每对点由参考椭球上的测地线连接。通过将最后一个点连接到第一个点来完成多边形。允许的区域是多边形的内部及其边界。

如果包含高程分量的点在其参考椭球上的水平投影位于该区域内，则认为该点位于多边形区域中。

有效的PolygonalRegion包含至少三个点。在有效的PolygonalRegion中，构成多边形边的隐含线不相交。

6.5.21 IdentifiedRegion

```
IdentifiedRegion ::= CHOICE {
    countryOnly      CountryOnly,
    countryAndRegions CountryAndRegions,
    countryAndSubregions CountryAndSubregions,
    ...
}
```

```
}
```

SequenceOfIdentifiedRegion ::= SEQUENCE OF IdentifiedRegion

此数据类型使用区域标识符来指示证书的有效区域。

6.5.22 CountryOnly

CountryOnly ::= UInt16

这是联合国统计司于2013年10月定义的国家或地区标识符的整数表示。

6.5.23 CountryAndRegions

```
CountryAndRegions ::= SEQUENCE {
    countryOnly    CountryOnly,
    regions        SequenceOfUInt8
}
```

在这种数据类型中，：

- CountryOnly 如上面定义的 CountryOnly；
- Region 标识国家内的一个或多个区域；

6.5.24 CountryAndSubregions

```
CountryAndSubregions ::= SEQUENCE {
    country          CountryOnly,
    regionAndSubregions SequenceOfRegionAndSubregions
}
```

在这种数据类型中，：

- Country 如上面定义的 CountryOnly；
- regionAndSubregions 标识国家内的一个或多个子区域。

6.5.25 RegionAndSubregions

```
RegionAndSubregions ::= SEQUENCE {
    region          UInt8,
    subregions      SequenceOfUInt16
}
```

SequenceOfRegionAndSubregions ::= SEQUENCE OF RegionAndSubregions

在这种数据类型中：

- region 标识 CountryAndRegions 中指定的国家/地区内的区域。
- 子区域标识 CountryAndRegions 下指定的一个或多个子区域。

6.5.26 SubjectAssurance

SubjectAssurance ::= OCTET STRING (SIZE(1))

该字段包含证书持有者的保证级别，该级别指示平台和密钥存储的安全性以及对此评估的置信度。

该字段按表1中的定义进行编码，其中“A”表示指定保证等级的位字段，“R”为保留位字段，“C”为置信度字段。

表7 主题保证的按位编码

比特位号	7	6	5	4	3	2	1	0
解释	A	A	A	R	R	R	C	C

在表7中，比特位号 (Bit number) 0表示最低有效位； 比特位7到位5表示设备的保证级别； 比特位4到位2保留供将来使用； 比特位1和位0表示置信度。

注：该字段最初在ETSI TS 103 097中规定，预计该字段的未来用途将与该文件的未来版本一致。

6.5.27 AidSsp

```
AidSsp ::= SEQUENCE {
    aid                Aid,
    ssp                ServiceSpecificPermissions OPTIONAL
}
```

```
SequenceOfAidSsp ::= SEQUENCE OF AidSsp
```

该数据结构表示证书持有者对于用Aid标识的单个应用程序领域的的数据所具有的权限。 如果省略ServiceSpecificPermissions字段，则表示证书持有者具有与该Aid关联的默认权限。

如果证书具有省略ssp字段的appPermissions条目A，则如果签发证书包含AidSspRange P 且以下条件成立，则A与发布证书一致：

——P 中的 aid 字段等于 A 中的 aid 字段，且以下之一为真：

- P 中的 sspRange 字段指示 all；
 - P 中的 sspRange 字段指示 opaque，而 opaque 中的一个条目是长度为 0 的八进制字符串；
- 其他形式的ssp字段的一致性规则，请参阅后续条款；

6.5.28 ServiceSpecificPermissions

```
ServiceSpecificPermissions ::= CHOICE {
    opaque                OCTET STRING (SIZE(0..MAX)),
    ...,
    bitmapSsp            BitmapSsp
}
```

该数据结构表示与AidSsp中的给定条目相关的服务特定权限（SSP）。 SSP的含义取决于相关联的Aid， 它可以是AID特定的八位字节串或比特位图。

如果证书具有appPermissions条目A、且其中的ssp字段是opaque，则如果签发证书包含以下之一，A与签发证书一致：

- （选项 1）一个 SubjectPermissions 字段，它指示选择 all，且没有 AidSspRange 字段包含 A 中的 aid 值；
- （选项 2）一个 AidSspRange P，其中包含以下内容：
 - P 中的 aid 字段等于 A 中的 aid 字段，且以下之一为真：
 - ◆ P 中的 sspRange 字段表示 all.

- ◆ P 中的 sspRange 字段指示 opaque, P 中 opaque 字段中的一个条目是与 A 中 opaque 字段相同的八位字节串。

其他类型的ServiceSpecificPermissions的一致性规则, 请参阅后续条款。

6.5.29 BitmapSsp

BitmapSsp ::= OCTET STRING (SIZE(0..31))

该数据结构表示SSP的比特位图。比特位图映射到签名的SPDU上的约束取决于AID。

如果证书具有appPermissions条目A, 其ssp字段为bitmapSsp, 则如果签发证书包含以下之一, 则A与签发证书一致:

- (选项1) 一个 SubjectPermissions 字段, 它指示选择 all, 且没有 AidSspRange 字段包含 A 中的 aid 值;
- (选项2) 一个 AidSspRange P, 其中包含以下内容:
 - P 中的 aid 字段等于 A 中的 aid 字段, 且以下之一为真:
 - ◆ P 中的 sspRange 字段表示 all.;
 - ◆ R 中的 sspRange 字段指示 bitmapSspRange, 且 R 的 sspBitmask 中每一个被置为 1 的比特, P 中 sspValue 和 R 中 sspValue 在该比特位上的取值相同;

注: 对于R中sspBitmask被设置为1的每一个位元, 如果B和R的sspValue在该比特位置的值都相同, 则称B的 BitmapSsp 和BitmapSspRange R相一致。对于R中sspBitmask被设置为0的位元, B可以自由设置为0或1, B和R是否一致与这些比特位的取值无关。

6.5.30 AidGroupPermissions

```
AidGroupPermissions ::= SEQUENCE {
    subjectPermissions SubjectPermissions,
    minChainLength      INTEGER DEFAULT 1,
    chainLengthRange    INTEGER DEFAULT 0,
    eeType              EndEntityType DEFAULT {app}
}
```

SequenceOfAidGroupPermissions ::= SEQUENCE OF AidGroupPermissions

此数据结构指示证书持有人在发布和请求对应特定AID集合的证书时所拥有的权限, 它包含如下内容:

- subjectPermissions 表示此字段涵盖的 AID 和 SSP 范围;
- minChainLength 和 chainLengthRange 指示允许从此证书到最终实体证书的证书链的长度。证书链的长度是指链中低于此证书的、直至并包括最终实体证书的证书数量。允许的证书链长度为大于或等于 minChainLength, 同时小于或等于 minChainLength + chainLengthRange。当此类型出现在 ToBeSignedCertificate 的 certIssuePermissions 字段中时, 不允许 minChainLength 使用为 0 的值, 此字段值为 0 的证书无效。chainLengthRange 的值-1 是一种特殊情况, 表示证书链可以是等于或大于 minChainLength 的任何长度。对于下级证书和签发其的上一级证书中, certIssuePermissions 字段 AID 相同的 minChainLength 和 chainLengthRange, 其值还应满足下列关系: (设下级证书的 minChainLength 为 mcls, 下级证书的 chainLengthRange 为 clrs, 上级签发证书的 minChainLength 为 mcli, 上级签发证

书的 chainLengthRange 为 clri,)

- $mcli \leq mcls + 1$
- $(mcli + clri) \geq (mcls + clrs + 1)$

注：在下级证书是V2X设备的情况下，上述公式中的mcls和clrs设置为零（0）。

——eeType 取一个或多个“app”和“enroll”值，它指示证书的类型或请求证书中的AidGroupPermissions实例有权用于授权检查。如果此字段指示app，则允许证书链以授权证书结束，换句话说，如果该字段不指示app，但证书链用授权证书结束，则证书链无效。如果此字段指示enroll，则允许证书链以注册证书结束。ToBeSignedCertificate中的不同AidGroupPermissions实例可能具有不同的eeType值；对于相同AID，如果下级证书的eeType位于证书的应用权限appPermissions字段中，则签发该证书的上级证书其eeType字段包含app；如果下级证书的eeType位于证书的证书签发权限certIssuePermissions字段中，则下级证书中eeType的所有取值，都应当出现在上级证书的证书签发权限certIssuePermissions字段中。

6.5.31 SubjectPermissions

```
SubjectPermissions ::= CHOICE {
    explicit      SequenceOfAidSspRange,
    all           NULL,
    ...
}
```

它指示AidGroupPermissions数据结构向哪些AID和关联的SSP授予证书签发或请求权限。如果它的取值为explicit（显式），则AidGroupPermissions数据结构为指明的特定AID和SSP Ranges授予证书签发或请求权限；如果此值为all，则AidGroupPermissions数据结构为同一certIssuePermissions或certRequestPermissions字段中未由其他AidGroupPermissions指示的所有AID授予证书签发或请求权限。

6.5.32 EndEntityType

```
EndEntityType ::= BIT STRING {app (0), enroll (1)} (SIZE (8)) (ALL EXCEPT {})
```

此数据类型指示哪些类型的权限可能出现在最终实体证书中，其证书链通过包含此值的AidGroupPermissions字段传递权限。如果它指示的是app，则最终实体证书可能包含appPermissions字段；如果它指示的是enroll，则最终实体证书可能包含certRequestPermissions字段。

6.5.33 AidSspRange

```
AidSspRange ::= SEQUENCE {
    aid          Aid,
    sspRange     SspRange OPTIONAL
}
SequenceOfAidSspRange ::= SEQUENCE OF AidSspRange
```

此数据结构表示证书持有者针对一组特定应用程序权限发出证书或请求证书的权限。这个数据结构中包含：

- aid 标识应用区域；
- sspRange 标识特定的 SSP，它们与证书持有者可以签发或请求证书的 AID 相关联。如果

sspRange 被省略，则持有者可以为该 AID 的任何 SSP 签发或请求证书。

6.5.34 SspRange

```
SspRange ::= CHOICE {
    opaque          SequenceOfOctetString,
    all             NULL,
    ... ,
    bitmapSspRange  BitmapSspRange
}
```

该数据结构标识与持有者可以发布或请求证书的AID相关联的SSP。

当一个证书具有ssp字段是Opaque形的AidSspRange A时，如果签发证书包含以下之一，则A与签发证书一致：

- （选项1）一个 SubjectPermissions 字段，它指示选择“all”，且没有 AidSspRange 包含 A 中的 aid 字段；
- （选项2）一个 AidSspRange P，满足以下条件：
 - P 中的 aid 字段等于 A 中的 aid 字段，并且以下之一为真：
 - ◆ P 中的 sspRange 字段表示 all.
 - ◆ P 中的 sspRange 字段表示 Opaque，A 中的 sspRange 字段表示 Opaque，A 中 Opaque 内的每个八位字节串都是 P 中 Opaque 内的八位字节串的副本。

当一个证书具有ssp字段为all的AidSspRange A时，如果签发证书包含满足以下条件的 AidSspRange P，则A与签发证书一致：

- （选项1）一个 SubjectPermissions 字段，它指示选择“all”，且没有 AidSspRange 包含 A 中的 aid 字段；
- （选项2）一个 AidSspRange P，P 中的 aid 字段等于 A 中的 aid 字段，且 P 中的 sspRange 字段为“all”。

有关其他类型SspRange的一致性规则，请参阅后续子条款。

6.5.35 BitmapSspRange

```
BitmapSspRange ::= SEQUENCE {
    sspValue          OCTET STRING (SIZE(1..32)),
    sspBitmask        OCTET STRING (SIZE(1..32)),
}
```

该数据结构为SSP的位图表示，其中sspValue表示权限；sspBitmask包含一个八位字节字符串，用于允许或约束被签发证书中的sspValue字段。sspValue和sspBitmask字段的长度应相同。。

与签发证书的一致性。

当一个证书具有ssRange字段为bitmapSspRange的AidSspRange值P时，如果签发证书包含以下内容之一，则P与签发证书一致：

- （选项1）一个 SubjectPermissions 字段，它指示选择“all”，且没有 AidSspRange 包含 P 中的 aid 字段；
- （选项2）一个 AidSspRange R，满足以下条件：
 - R 中的 aid 字段等于 P 中的 aid 字段，并且以下之一为真：

- ◆ R 中的 sspRange 字段表示 all;
- ◆ R 中的 sspRange 字段指示 bitmapSspRange, 且对于 R 的 sspBitmask 中的每个被设置为 1 的比特位, 有
 - P 中 sspBitmask 的相同比特位也被设置为 1, 且
 - P 的 sspValue 中相同位置的值与 R 的 sspValue 中对应位置的信息相同;

有关 SSP 比特位图的更多信息, 请参考 ETSI TS 103.097。

6.5.36 VerificationKeyIndicator

```
VerificationKeyIndicator ::= CHOICE {
    verificationKey      PublicVerificationKey,
    reconstructionValue  EccP256CurvePoint,
    ...
}
```

此字段的内容取决于证书是隐式证书还是显式证书:

- verificationKey 出现在显式证书中, 它包含用于验证 Certificate (证书) 持有者所生成数字签名的公钥;
- reconstructionValue 出现在隐式证书中, 它包含重构值, 用于恢复国密局隐式证书中指定的公钥。

6.5.37 PublicVerificationKey

```
PublicVerificationKey ::= CHOICE {
    ecdsaNistP256      EccP256CurvePoint,
    ecdsaBrainpoolP256r1 EccP256CurvePoint,
    ... ,
    ecdsaBrainpoolP384r1 EccP384CurvePoint,
    ecdsaNistP384      EccP384CurvePoint,
    ecsigSm2           EccP256CurvePoint
}
```

此数据结构承载公钥, 并指示使用该公钥的算法。

如果 PublicVerificationKey 结构中含 EccP256CurvePoint 或 EccP384CurvePoint, 但指示选择的是 x-only, 则为无效。

7 LTE-V2X 通信安全认证交互流程及接口技术要求

7.1 注册证书管理流程

7.1.1 概述

注册证书管理主要包括 EC 证书的申请和更新操作, 实现 V2X 设备向 EC 证书机构申请、更新 EC 证书的业务处理。

根据 AAA 系统具体实现方式的不同, 本标准支持通过 DCM 和 GBA 两种方式实现注册证书的管理。为了确保数据交互的安全性, 注册证书机构 ECA 与认证授权机构 AAA 之间应预先建立安全连接, 为 V2X 设备提

供统一的安全通信通道。V2X设备与认证授权机构AAA之间应根据系统实现方式的不同，采用安全环境、应用层加密、TLS安全通道等措施建立安全连接。

7.1.2 注册证书申请流程

7.1.2.1 DCM 架构下的注册证书申请流程

基于DCM的PKI系统架构中，V2X设备与DCM服务系统之间、DCM服务系统与ECA之间连接应确保安全性，可基于物理环境安全、TLS安全协议或专用的端到端安全连接完成相关操作。初始状态下，EC证书具体申请流程如图6所示。

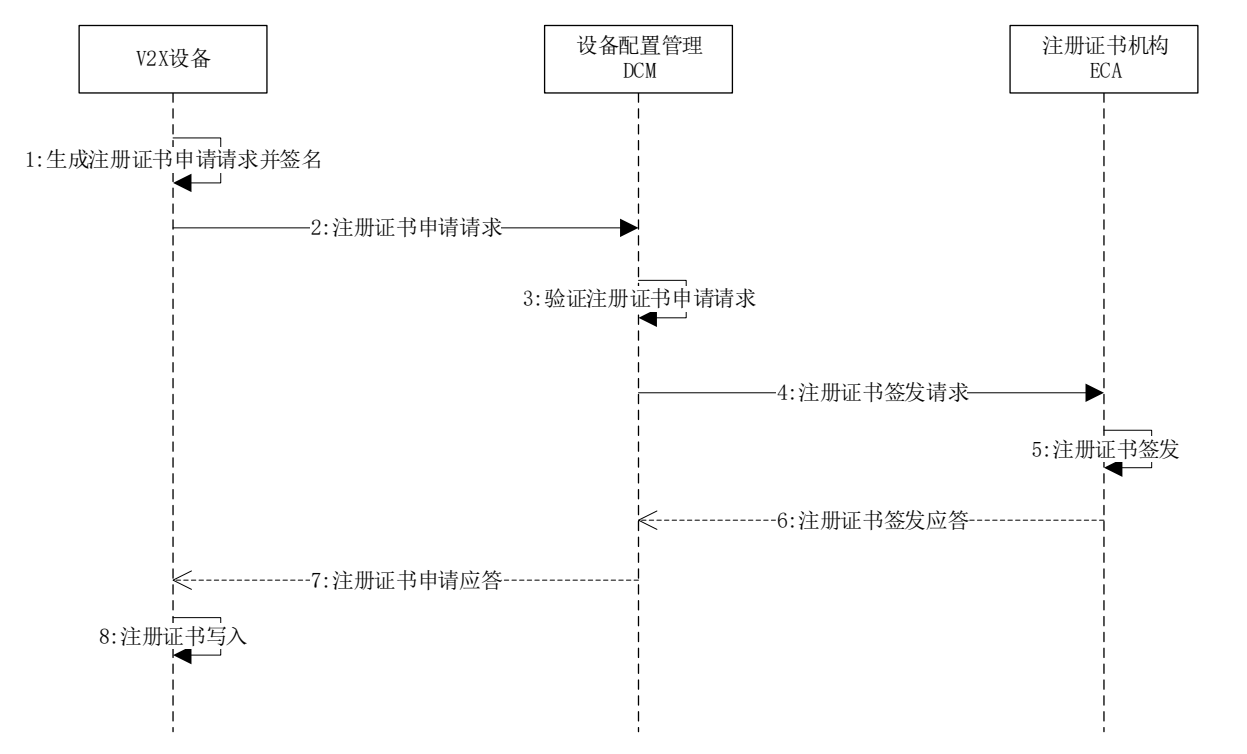


图6 DCM 架构下的注册证书申请流程

DCM架构下的注册证书申请流程如下：

- 1) V2X 设备生成签名公私密钥对，生成注册证书申请请求，并使用签名私钥对注册证书申请请求进行签名，（见附录 E. 4. 8，附录 E. 3. 11，附录 E. 3. 12）。基于系统的具体实现方式，系统可采用自定义的安全机制对该注册证书申请请求提供额外的保护，以便 DCM 或 ECA 能够认证 V2X 设备的证书申请请求。
- 2) V2X 设备将证书申请请求发送给 DCM 服务系统。
- 3) DCM 接收 V2X 设备的证书申请请求。若 V2X 设备的证书申请请求由 DCM 完成认证，则 DCM 验证该请求，并获得 V2X 注册证书申请请求。
- 4) 若由 DCM 执行证书申请请求的验证，则在验证通过后，DCM 向注册证书机构 ECA 发送 V2X 设备的注册证书申请请求。若由 ECA 执行证书申请请求的验证，则 DCM 转发其收到的证书申请请求。
- 5) ECA 验证 V2X 设备的证书申请请求并签发 EC。

- 6) ECA 通过注册证书申请应答将 EC 返回至 DCM。（见附录 E. 4. 7，附录 E. 3. 11，附录 E. 3. 13）
- 7) DCM 将注册证书申请应答转发至 V2X 设备。
- 8) V2X 设备验证接收到的注册证书申请应答并将 EC 写入设备存储空间中。

DCM架构下注册证书申请接口及请求和响应传递的参数如下：

- 注册证书申请请求类型：HTTP POST 或 HTTPS POST
- HTTP Content-Type: application/octet-stream
- HTTP request body 为 EeEcaCertRequestSpdu 结构体，见附录错误!未找到引用源。，包含 V2X 设备生成的公钥，签名所使用的哈希算法，时间，地区，SSP 等参数。
- HTTP response body 为 EcaEeCertResponseSpdu 结构体，见附录错误!未找到引用源。，包含对应请求的哈希值，EC，ECA 证书等参数。

7.1.2.2 GBA 架构下的注册证书申请流程

基于GBA的PKI系统架构中，V2X设备以USIM为安全根并通过LTE网络GBA机制与ECA注册证书机构建立安全关联，在初始状态下申请EC证书。该业务处理流程如图7所示。

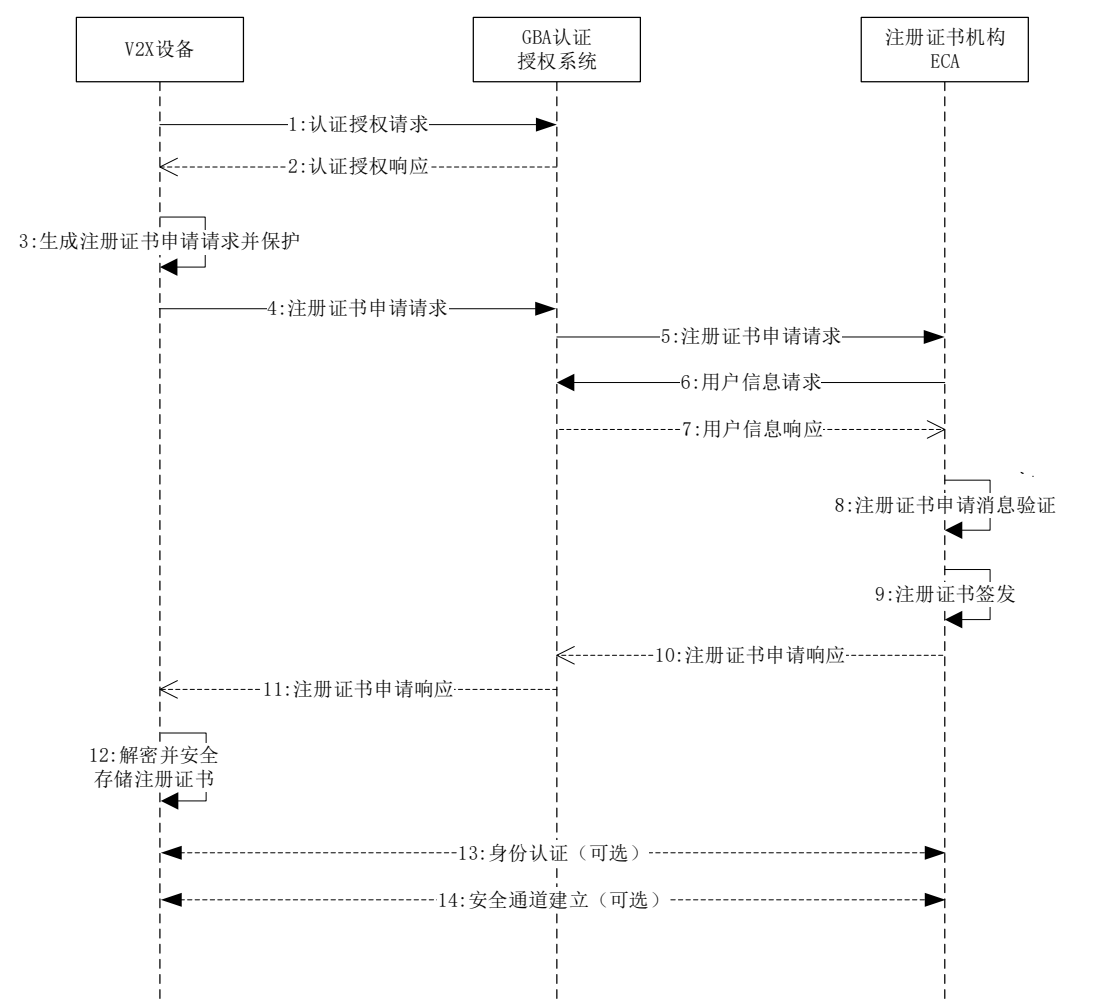


图7 GBA 架构下的注册证书申请流程

GBA架构下的注册证书申请流程如下：

- 1) 申请注册证书时，若 V2X 设备没有有效的 GBA 共享会话密钥，V2X 设备通过蜂窝网接入 GBA 认证授权系统，发起认证授权请求。
- 2) GBA 机制遵从 3GPP TS 33.220 实现，优先采用 GBA_U 方式，成功后向 V2X 设备返回认证授权响应。GBA 认证授权系统中的 BSF 负责对 V2X 设备进行身份认证并向 NAF 提供 GBA 密钥，NAF 负责产生多个 GBA 共享会话密钥供 ECA 使用，密钥的具体生成方式请见附录 F。
- 3) V2X 设备请求 USIM 为注册证书产生密码公私钥对，组建 EC 注册证书申请请求，并使用 USIM 生成的 GBA 共享会话密钥 K1 和 K2 对 EC 申请请求消息进行加密和完整性保护。
- 4) V2X 设备通过 GBA 认证授权系统向注册证书机构 ECA 发送经过保护的注册证书申请请求消息。注册证书申请请求消息由 HTTP 消息承载，HTTP 消息中携带 B-TID，ECA 服务器域名信息。
- 5) GBA 认证授权系统根据 ECA 服务器域名信息向 ECA 转发注册证书申请请求，其中携带 B-TID。
- 6) ECA 向 GBA 认证授权系统申请获取 GBA 共享会话密钥及用户信息。该请求采用 HTTP 消息形式，其中携带 B-TID，ECA 服务器域名信息。
- 7) 操作授权后，GBA 认证授权系统生成 GBA 共享会话密钥 K1、K2、K3、K4，并向 ECA 返回用户信息，GBA 共享会话密钥 K1、K2、K3、K4 及密钥生存期。ECA 使用获得的 GBA 共享会话密钥 K1 和 K2 对注册证书申请请求消息进行完整性保护校验和解密。
- 8) ECA 审核注册证书申请请求，通过后签发 EC 注册证书。
- 9) ECA 使用 GBA 共享会话密钥 K1 和 K2 对 EC 注册证书进行加密和完整性保护。
- 10) ECA 通过 GBA 认证授权系统向 V2X 设备发送注册证书申请响应消息，应答成功。消息中包含签发并经保护的 EC 注册证书。
- 11) GBA 认证授权系统向 V2X 设备转发注册证书申请响应消息，应答成功。
- 12) V2X 设备请求 USIM 使用 K1 和 K2 对消息进行完整性保护校验和解密，并将 EC 注册证书安全存储。
- 13) 基于 GBA 共享会话密钥 K3，ECA 与 V2X 设备可选进行双向身份认证。
- 14) 基于 GBA 共享会话密钥 K4，V2X 设备与 ECA 之间可选建立安全通信通道，如 TLS，应用层加密等，用于数据端到端安全传输。

7.1.3 注册证书更新流程

7.1.3.1 概述

注册证书更新流程用于在 EC 证书未到期的情况下为 V2X 设备设备申请新的 EC 证书，若 V2X 设备的现有 EC 证书已到期，且未申请新的 EC 证书的情况，按照 7.1.2 注册证书申请流程重新申请 EC 证书。申请时，V2X 设备应在注册更新请求消息中携带已有 EC 证书，以便 EC 证书机构核验信息和签发新的 EC 证书。各种 V2X 证书的生命周期可以参考附录 G 的内容。

7.1.3.2 DCM 架构下的注册证书更新流程

基于 DCM 的 PKI 系统架构中，注册证书更新流程指的是 V2X 设备使用已有的、尚未过期的 EC 证书对 EC 更新请求进行签名的情况。V2X 设备与 ECA 之间连接应确保安全性，可基于物理安全、TLS 安全协议或专用的端到端安全连接完成相关操作。具体更新流程如图 8 所示。

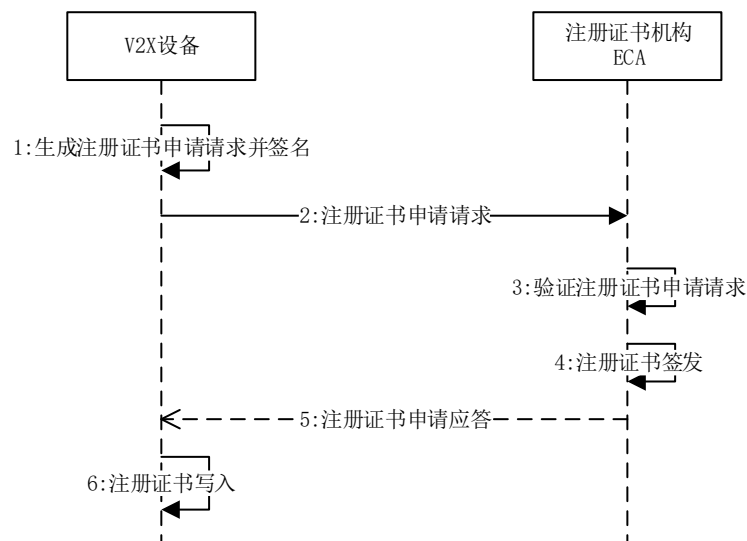


图8 DCM 架构下的注册证书更新流程

DCM架构下的注册证书更新流程如下：

- 1) V2X 设备生成签名公私密钥对，生成注册证书 EC 更新请求，其中 EeRaInterfacePdu 选择 eeRaSuccessorEnrollmentCertRequest，使用新生成的私钥自签名（见附录 E. 3. 15 ）；再使用已有的、尚未过期的 EC 证书对应的私钥对 EC 更新请求进行签名得到 EeRaSuccessorEnrollmentCertRequestSpdu（见附录 E. 4. 13 ）；
- 2) V2X 设备将 EC 更新请求发送给 ECA 服务系统（附录 E. 4. 8，附录 E. 3. 11，附录 E. 3. 12）；
- 3) ECA 接受 EC 更新请求，验证该请求的签名和原有 EC 证书等的有效性；
- 4) ECA 签发 EC；
- 5) ECA 向 V2X 设备发送注册证书更新响应消息，其中包含签发的新的 EC 注册证书（见附录 E. 4. 7，附录 E. 3. 11，附录 E. 3. 13）；
- 6) V2X 设备安全存储并使用新的 EC 注册证书，并将旧的 EC 注册证书作废。

7. 1. 3. 3 GBA 架构下的注册证书更新流程

基于GBA的PKI系统架构中，V2X设备以USIM为安全根并通过LTE网络GBA机制与LTE-V2X证书机构建立安全关联，更新EC证书。该业务处理流程如图9所示。

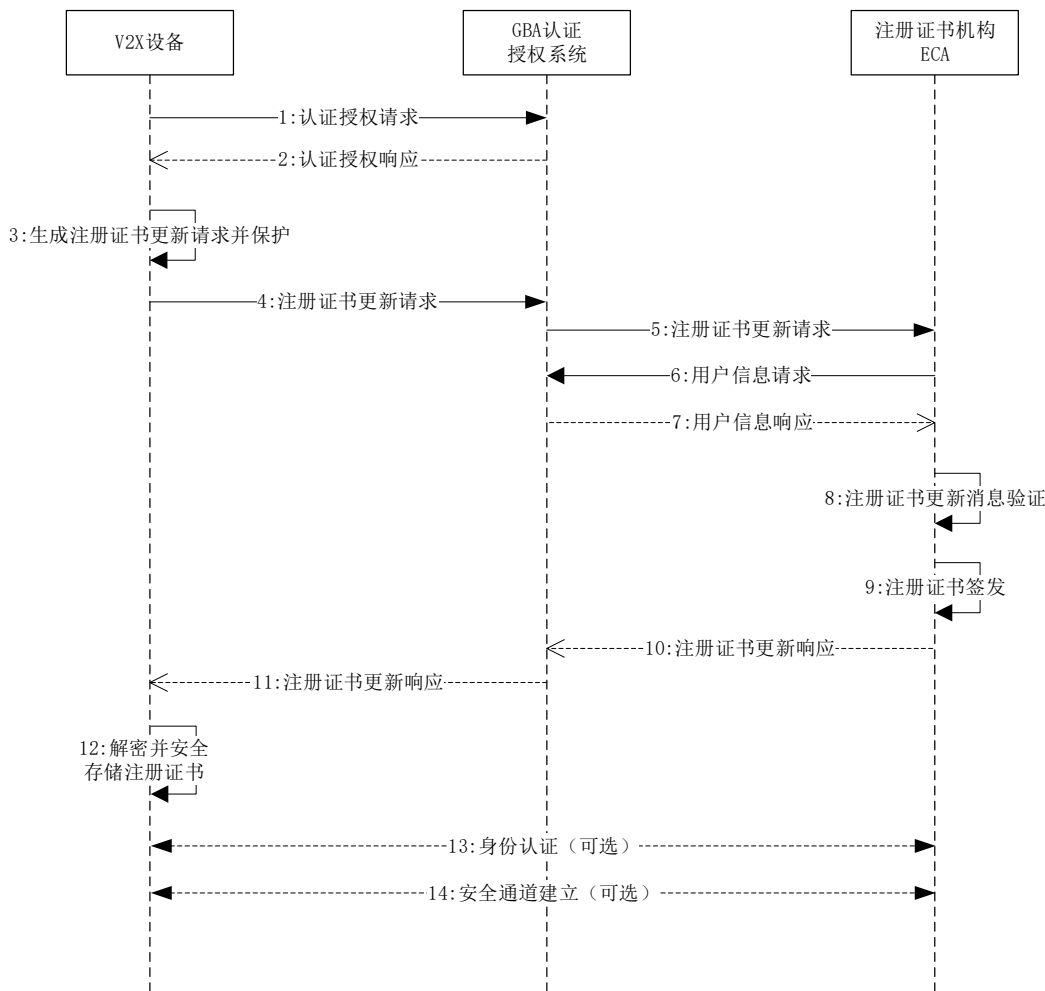


图9 GBA 架构下的注册证书更新流程

GBA架构下的注册证书更新流程如下：

- 1) 更新注册证书时，若 V2X 设备没有有效的 GBA 共享会话密钥，V2X 设备通过蜂窝网接入 GBA 认证授权系统，发起认证授权请求。
- 2) GBA 机制遵从 3GPP TS 33.220 实现，优先采用 GBA_U 方式，成功后向 V2X 设备返回认证授权响应。GBA 认证授权系统中的 BSF 负责对 V2X 设备进行身份认证并向 NAF 提供 GBA 密钥，NAF 负责产生多个 GBA 共享会话密钥供 ECA 使用，密钥的具体生成方式请见附录 F。
- 3) V2X 设备请求 USIM 为新的注册证书产生密码公私钥对，组建 EC 注册证书更新请求，并使用 USIM 生成的 GBA 共享会话密钥 K1 和 K2 对 EC 更新请求消息进行加密和完整性保护。
- 4) V2X 设备通过 GBA 认证授权系统向注册证书机构 ECA 发送经过保护的注册证书更新请求消息。注册证书更新请求消息由 HTTP 消息承载，HTTP 消息中携带 B-TID，ECA 服务器域名信息。
- 5) GBA 认证授权系统根据 ECA 服务器域名信息向 ECA 转发注册证书更新请求，其中携带 B-TID。
- 6) ECA 向 GBA 认证授权系统申请获取 GBA 共享会话密钥及用户信息。该请求采用 HTTP 消息形式，其中携带 B-TID，ECA 服务器域名信息。

- 7) 操作授权后, GBA 认证授权系统生成 GBA 共享会话密钥 K1、K2、K3、K4, 并向 ECA 返回用户信息, GBA 共享会话密钥 K1、K2、K3、K4 及密钥生存期。ECA 使用获得的 GBA 共享会话密钥 K1 和 K2 对注册证书更新请求消息进行完整性保护校验和解密。
- 8) ECA 审核注册证书更新请求, 通过后签发新的 EC 注册证书。
- 9) ECA 使用 GBA 共享会话密钥 K1 和 K2 对 EC 注册证书进行加密和完整性保护。
- 10) ECA 通过 GBA 认证授权系统向 V2X 设备发送注册证书更新响应消息, 应答成功。消息中包含签发并经保护的新的 EC 注册证书。
- 11) GBA 认证授权系统向 V2X 设备转发注册证书更新响应消息, 应答成功。
- 12) V2X 设备请求 USIM 使用 K1 和 K2 对消息进行完整性保护校验和解密, 并将新的 EC 注册证书安全存储, 旧的 EC 注册证书作废。
- 13) 基于 GBA 共享会话密钥 K3, ECA 与 V2X 设备可选进行双向身份认证。
- 14) 基于 GBA 共享会话密钥 K4, V2X 设备与 ECA 之间可选建立安全通信通道, 如 TLS, 应用层加密等, 用于数据端到端安全传输。

7.1.3.4 注册证书更新接口及请求和响应传递的参数说明

该接口用于V2X设备向注册证书CA申请注册证书。

注册证书更新请求类型: HTTP POST或HTTPS POST

HTTP Content-Type: application/octet-stream

HTTP request body为EeRaSuccessorEnrollmentCertRequestSpdu (见附录E.4.13), 包含V2X设备生成的公钥, 签名所使用的哈希算法, 时间, 地区, SSP等参数。

HTTP response body为EcaEeCertResponseSpdu结构体, 见附录E.4.7, 包含对应请求的哈希值, EC证书等参数。

7.2 假名证书申请流程

7.2.1 概述

假名证书申请流程实现V2X设备向PC证书机构申请PC证书的业务处理。假名证书到期后直接作废, 无需更新, V2X设备另向PC证书机构申请新的PC证书。假名证书的撤销处理见7.4.2节。

假名证书申请流程适于安全初始化成功的V2X设备向假名证书注册机构申请、下载PC证书。根据申请时所持安全凭证的不同, 本标准支持采用EC注册证书或令牌(Token)两种方式申请假名证书。为了确保假名证书注册机构无法关联用户身份信息, 泄露用户隐私, V2X设备在申请PC证书时可对用户身份相关的隐私信息(例如与用户身份相关的设备标识号等)进行安全保护, 并由AAA完成相应的认证与授权操作。

假名证书的申请包括申请和下载两个阶段。根据认证授权系统具体实现方式的不同, 本标准支持DCM、GBA和OAuth三种方式实现假名证书的申请。

7.2.2 DCM 架构下的假名证书申请流程

假名证书申请基本流程如图10所示。在申请PC之前, V2X设备和PRA之间、PRA和PCA之间的连接应确保安全性, 可采用基于TLS安全协议或专用的端到端安全连接完成相关操作。

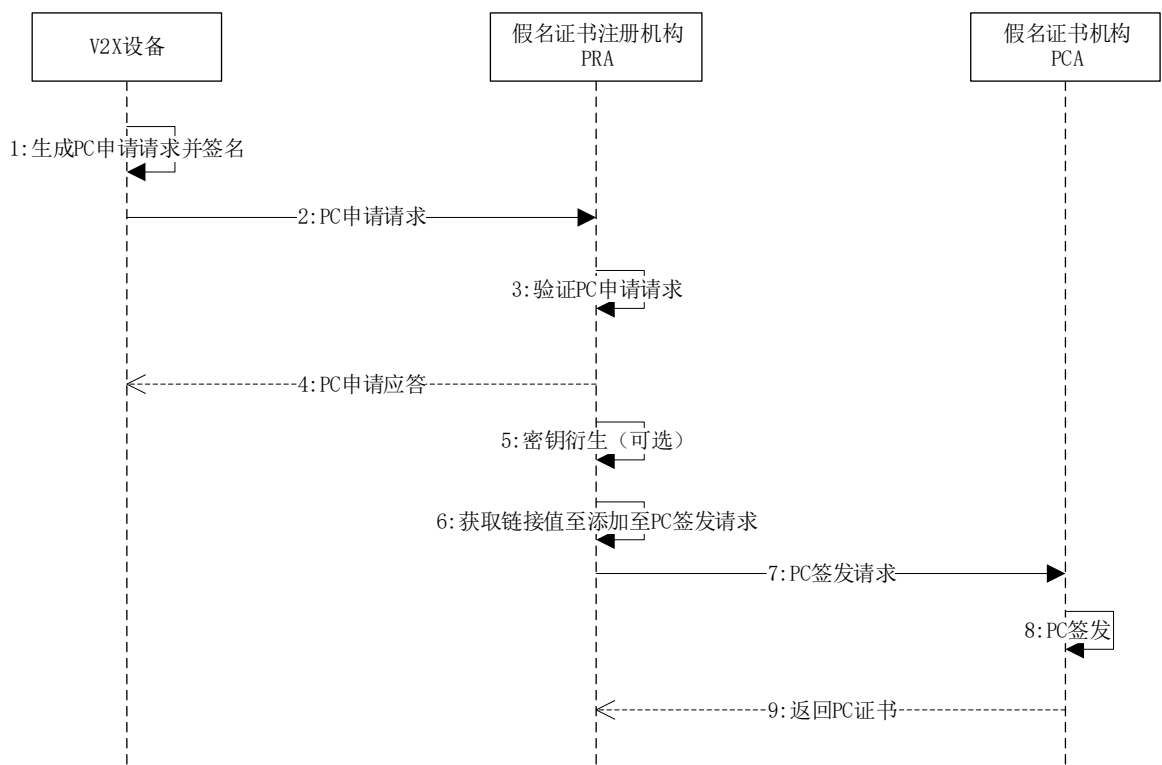


图10 假名证书申请基本流程

假名证书申请流程如下：

- 1) V2X 设备生成假名证书 PC 申请请求，并使用注册证书 EC 对应的私钥对该请求进行签名；
- 2) V2X 设备向假名证书注册机构 PRA 发送 PC 申请请求，其中应包含 EC（见附录 E. 4. 10，附录 E. 3. 15 和附录 E. 3. 16）；
- 3) PRA 验证 PC 申请请求及 EC，如果 V2X 设备对 PC 申请请求中的内容进行了加密，PRA 应向相应系统申请解密并验证 EC 证书及 PC 申请请求的有效性；
- 4) 验证通过后，向 V2X 设备返回 PC 申请应答，应答中包含 PC 下载时间（见附录 E. 4. 14，附录 E. 3. 15，附录 E. 3. 26）；
- 5) （可选步骤）PRA 对 PC 申请请求中的公钥进行扩展（见 7. 2. 6 ）；
- 6) 获取相应链接值密文并添加至 PC 签发请求（见 7. 7. 2）；
- 7) PRA 向假名证书机构 PCA 发送 PC 签发请求（见附录 E. 6. 3 和附录 E. 6. 6）；
- 8) PCA 签发 PC；
- 9) PCA 将生成的 PC 发送至 PRA，PRA 对其进行压缩并储存（见附录 E. 6. 5，附录 E. 6. 7 和附录 E. 4. 15）

7. 2. 3 GBA 架构下的假名证书申请流程

基于GBA的PKI系统架构中，ECA注册证书机构是可选实体。在采用ECA的系统中，V2X设备使用EC注册证书申请PC假名证书。在省略ECA的系统中，V2X设备使用GBA认证授权系统签发的业务令牌申请PC证书。因此，这里分别给出不同情况下的假名证书申请流程。

7. 2. 3. 1 基于 EC 注册证书申请假名证书

在采用ECA的GBA系统架构下，V2X设备使用EC注册证书申请假名证书，处理流程如图11所示。

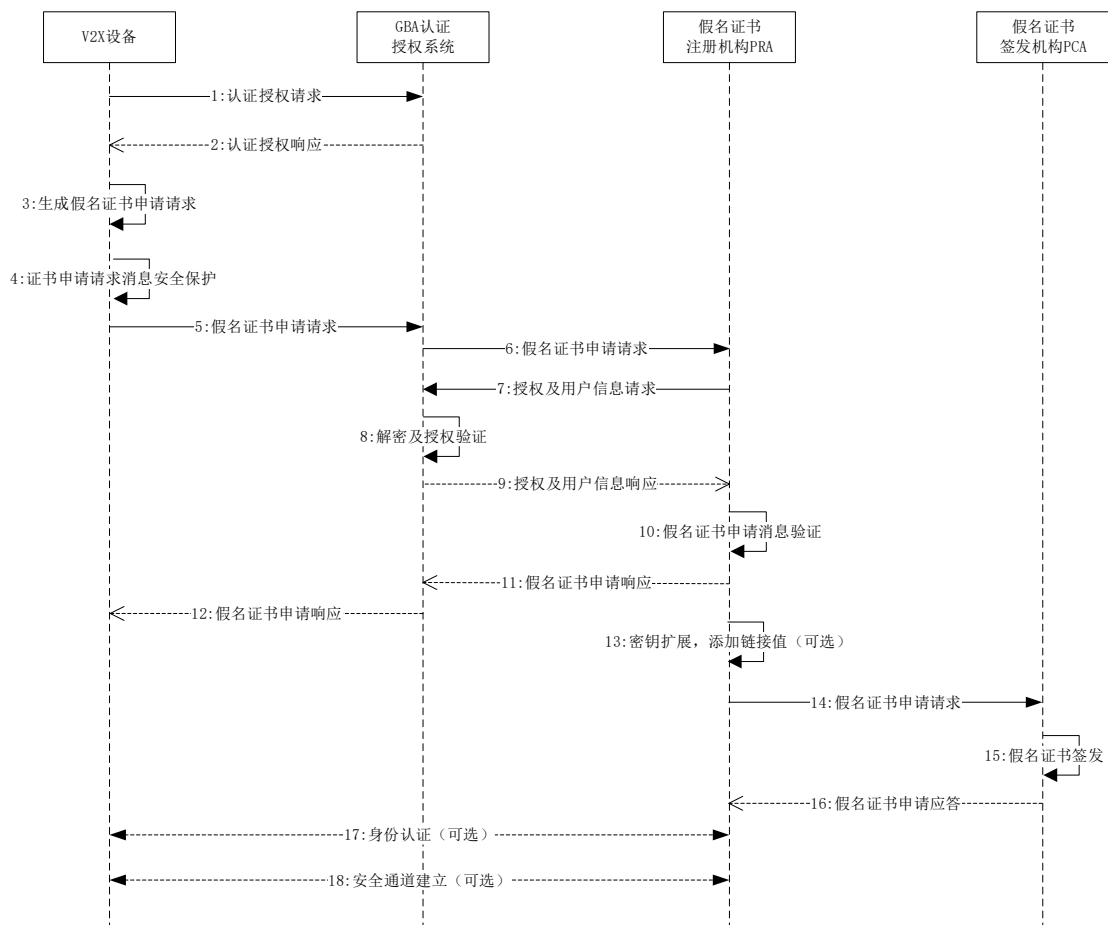


图11 GBA 架构下假名证书申请流程——基于 EC 证书

基于EC证书的GBA架构下假名证书申请流程如下：

- 1) 申请假名证书时，若 V2X 设备没有有效的 GBA 共享会话密钥，V2X 设备通过蜂窝网接入 GBA 认证授权系统，发起认证授权请求。
- 2) GBA 机制遵从 3GPP TS 33.220 实现，优先采用 GBA_U 方式，成功后向 V2X 设备返回认证授权响应。GBA 认证授权系统中的 BSF 负责对 V2X 设备进行身份认证并向 NAF 提供 GBA 密钥，NAF 负责产生多个 GBA 共享会话密钥供 PRA 使用，密钥的具体生成方式请见附录 F。
- 3) V2X 设备请求安全模块为假名证书产生密码公私钥对，组建 PC 假名证书申请请求，并请求 USIM 使用 EC 证书私钥对其签名。
- 4) V2X 设备使用 USIM 生成的 GBA 共享会话密钥 K1 和 K2 对 PC 申请请求消息进行加密和完整性保护。V2X 设备可使用 GBA 共享会话密钥 K5 对涉及用户身份的隐私信息（如，与用户身份相关的设备标识号等）进行加密保护。
- 5) V2X 设备通过 GBA 认证授权系统向假名证书注册机构 PRA 发送经过保护的假名证书申请请求消息，其中包含加密后的用户身份隐私信息。假名证书申请请求消息由 HTTP 消息承载，HTTP 消息中携带 B-TID，PRA 服务器域名信息。

- 6) GBA 认证授权系统根据 PRA 服务器域名信息向 PRA 转发假名证书申请请求，其中携带 B-TID。
- 7) PRA 就 V2X 设备的请求向 GBA 认证授权系统发送消息，提出授权申请并申请获取 GBA 共享会话密钥及用户信息。消息中可包含有加密后的用户身份隐私信息。该请求采用 HTTP 消息形式，其中携带 B-TID，PRA 服务器域名信息。
- 8) GBA 认证授权系统生成 GBA 共享会话密钥 K5，并使用 GBA 共享会话密钥 K5 解密用户身份信息，并授权检验。
- 9) 授权成功后，GBA 认证授权系统生成 GBA 共享会话密钥 K1、K2、K3、K4，并向 PRA 确认授权并返回用户信息、GBA 共享会话密钥 K1、K2、K3、K4 及密钥生存期。如果未成功，GBA 认证授权系统反馈授权失败响应。
- 10) PRA 使用获得的 GBA 共享会话密钥 K1 和 K2 对假名证书申请请求消息进行完整性保护校验和解密，获得 V2X 设备假名证书相关公钥。
- 11) PRA 审核假名证书申请请求，审核通过后生成假名证书申请响应并使用 GBA 共享会话密钥 K1 和 K2 对该消息进行加密和完整性保护。消息中包含 PC 证书下载时间。之后，通过 GBA 认证授权系统向 V2X 设备返回假名证书申请响应消息，应答成功。
- 12) GBA 认证授权系统向 V2X 设备转发假名证书申请响应消息，应答成功。V2X 设备使用 GBA 共享会话密钥 K1 和 K2 对该消息进行完整性保护校验和解密。
- 13) 可选地，PRA 对 PC 申请中的公钥因子进行扩展和/或获取链接值，形成假名证书申请请求。
- 14) PRA 向假名证书签发机构 PCA 发送 PC 假名证书申请请求，请求中包含 V2X 设备假名证书相关公钥。
- 15) PCA 签发 PC 假名证书。
- 16) PCA 将生成的 PC 证书发送至 PRA，PRA 对其进行压缩及储存。
- 17) 基于 GBA 共享会话密钥 K3，PRA 与 V2X 设备可选进行双向身份认证。
- 18) 基于 GBA 共享会话密钥 K4，V2X 设备与 PRA 之间可选建立安全通信通道，如 TLS，应用层加密等，用于数据端到端安全传输。

7.2.3.2 基于业务令牌申请假名证书

在省略ECA的GBA系统架构中，V2X设备在申请假名证书时需要先向GBA认证授权系统申请业务令牌，随后向PRA申请假名证书。

该过程业务处理流程包括以下两个过程：

- a) 业务令牌申请过程，业务令牌申请过程如图 12 所示。

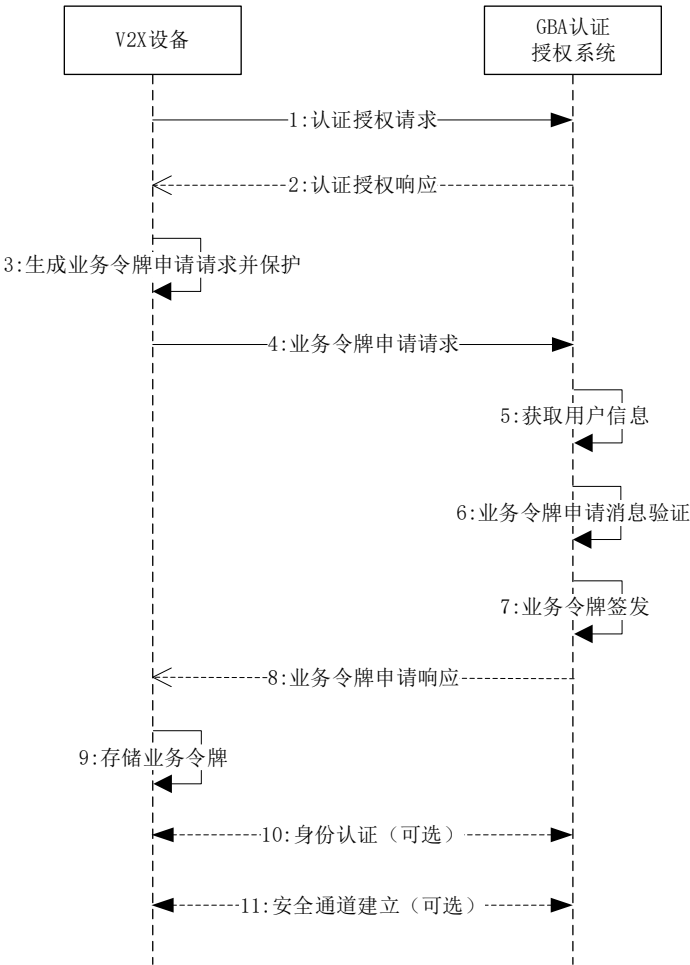


图12 GBA 架构下业务令牌申请流程

GBA架构下业务令牌申请流程如下：

- 1) 申请假名证书时，若 V2X 设备没有有效的业务令牌，V2X 设备通过蜂窝网接入 GBA 认证授权系统，发起认证授权请求。
- 2) GBA 机制遵从 3GPP TS 33.220 实现，优先采用 GBA_U 方式，成功后向 V2X 设备返回认证授权响应。GBA 认证授权系统中的 BSF 负责对 V2X 设备进行身份认证并向 NAF 提供 GBA 密钥，NAF 负责产生多个 GBA 共享会话密钥供 GBA 认证授权系统使用，密钥的具体生成方式请见附录 F。
- 3) V2X 设备请求安全模块为业务令牌产生密码公私钥对，生成业务令牌申请请求，并使用 USIM 生成的 GBA 共享会话密钥 K1 和 K2 对业务令牌申请进行加密和完整性保护。
- 4) V2X 设备向 GBA 业务授权系统发送业务令牌申请请求消息。
- 5) GBA 认证授权系统获取用户信息，其中包括 GBA 安全认证过程中生成的 GBA 共享会话密钥 K1、K2、K3、K4 及密钥生存期。
- 6) GBA 认证授权系统使用 GBA 共享会话密钥 K1 和 K2 对业务令牌申请请求消息进行完整性保护校验和解密。
- 7) GBA 认证授权系统审核业务令牌申请请求，通过后签发业务令牌。

- 8) GBA 认证授权系统向 V2X 设备发送业务令牌申请响应消息，其中包含签发的业务令牌。业务令牌申请响应消息采用 GBA 共享会话密钥 K1 和 K2 进行加密和完整性保护。
 - 9) V2X 设备使用 K1 和 K2 对收到的响应消息进行完整性保护校验并解密，之后将业务令牌安全存储。
 - 10) 基于 GBA 共享会话密钥 K3，GBA 认证授权系统与 V2X 设备可选进行双向身份认证。
 - 11) 基于 GBA 共享会话密钥 K4，V2X 设备与 GBA 认证授权系统之间可选建立安全通信通道，如 TLS，应用层加密等，用于数据端到端安全传输。
- b) 假名证书申请过程。基于业务令牌，V2X 设备申请假名证书的处理流程如图 13。

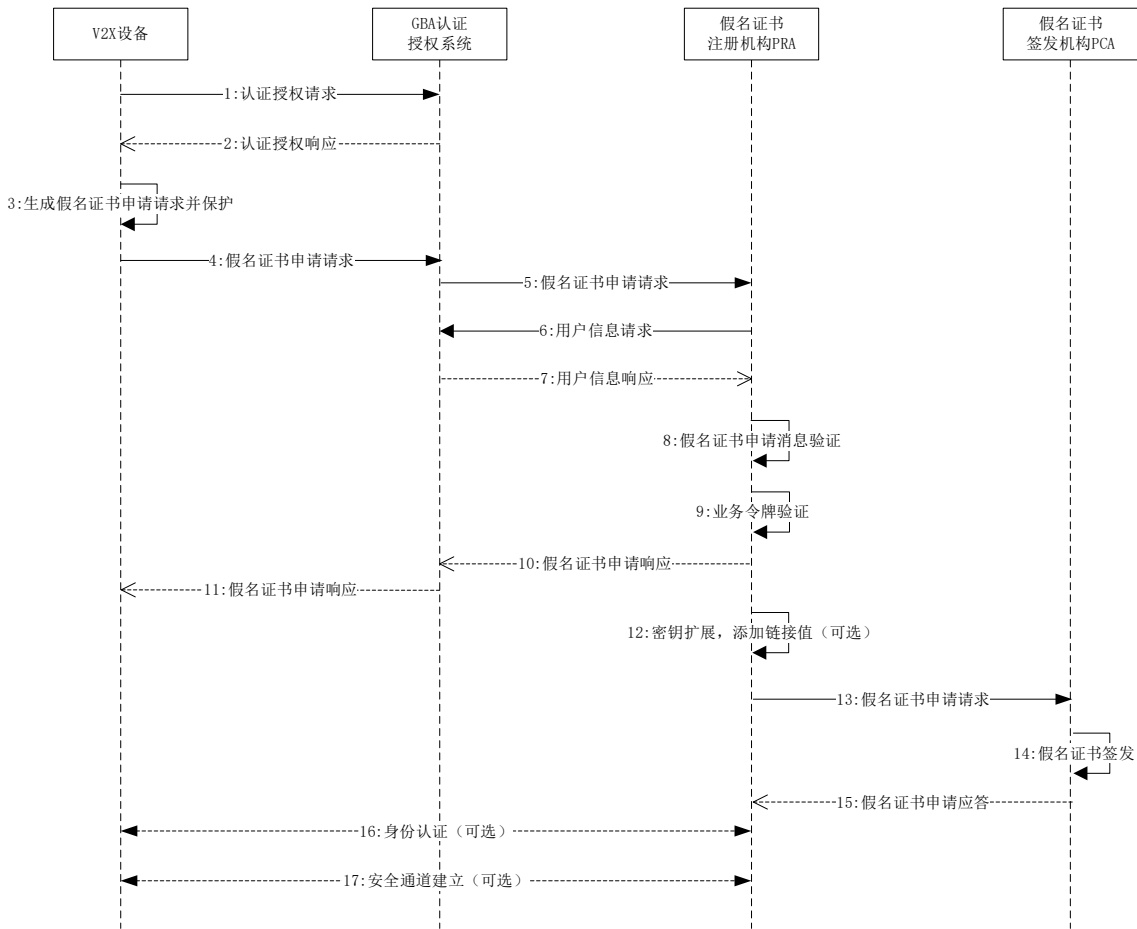


图13 GBA 架构下假名证书申请流程——基于业务令牌

上述流程与基于EC证书的假名证书申请流程大致相同，主要差别点在于假名证书注册机构PRA根据申请消息中的业务令牌来确定是否为V2X设备申请签发PC假名证书。

7.2.4 OAuth 架构下的假名证书申请流程

当认证授权机构AAA为6.1.4所述的OAuth机制时，在7.2.2描述的流程基础上，基于OAuth的假名证书申请流程补充如下：

在步骤错误!未找到引用源。之前，V2X设备：

- a) 基于 OAuth 2.0 机制，向 OAuth 授权服务器请求访问令牌（Access Token）用于访问 PRA；
- b) 授权服务器鉴别 V2X 设备的身份并根据验证策略向 V2X 设备发放访问令牌；

步骤**错误!未找到引用源。**中，V2X设备生成假名证书PC申请请求（参考附录E.4.9）；

步骤**错误!未找到引用源。**中，V2X设备向PRA发送PC申请请求及访问令牌（而非原步骤中的注册证书EC），V2X设备与PRA之间建立TLS安全通道；

步骤**错误!未找到引用源。**中，PRA验证访问令牌（而非原步骤中的注册证书EC），验证成功后执行7.2.2中描述的申请流程中的后续步骤。验证过程中，如果令牌已过期，则终止假名证书申请流程。

7.2.5 假名证书下载流程

V2X设备根据PRA提供的PC证书下载时间决定何时启动假名证书下载流程，具体如图14所示。

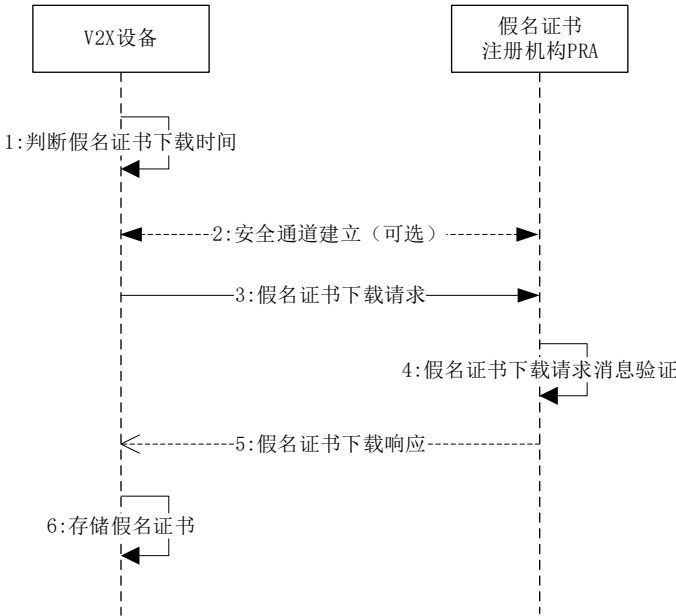


图14 假名证书下载流程

假名证书下载流程如下：

- 1) V2X 设备判断 PC 假名证书下载时间是否已到。
- 2) 如果下载时间已到，可选地，V2X 设备与 PRA 建立安全连接。
- 3) V2X 设备向 PRA 发送假名证书下载请求消息，应对所需传输的参数进行加密和完整性保护（如果有安全通道的情况下，见附录 E.4.11 或 E.4.12；如果没有安全通道的情况下见附录 E.4.12）。
- 4) PRA 验证 PC 证书下载请求消息。
- 5) 验证成功后，PRA 向 V2X 设备发送假名证书下载响应消息，其中包含 PC 假名证书压缩包（见附录 E.4.15 和附录 E.3.20）。
- 6) V2X 设备将 PC 假名证书安全存储。

7.2.6 密钥衍生流程

附录H为本标准对密钥衍生流程中所使用的算法提出的一种建议。在假名证书申请流程和下载流程的基础上，密钥衍生的补充流程如下：

- 1) V2X 设备在生成 PC 申请请求时, 使用非对称密钥算法生成两对种子公私密钥对: 签名密钥对 (a, A) , 和加密密钥对 (p, P) ; 并使用对称密钥算法生成两个对称密钥: 签名对称密钥 k_s 和加密对称密钥 k_e ;
- 2) V2X 设备发送给 PRA 的 PC 申请请求中, 包括了 A 、 P 、 k_s 、 k_e ;
- 3) PRA 对 PC 申请请求、EC 证书或令牌等信息校验通过后, 使用 A 、 P 、 k_s 、 k_e 进行第一轮扩展运算, 公钥因子 A 与 k_s 进行扩展计算, 得到签名扩展公钥 B , P 与 k_e 进行扩展计算, 得到加密扩展公钥 Q ; 如此进行若干轮扩展运算, 得到 $(B_{i,j}, Q_{i,j})$;
- 4) PRA 向 PCA 发送 PC 签发请求时, 将 $(B_{i,j}, Q_{i,j})$ 逐一发送给 PCA, 用于签发批量的 PC;
- 5) PCA 签发 PC 证书前, 随机生成 1 对公私钥对 (c, C) , 其中公钥分量 C 与若干个 $B_{i,j}$ 逐一进行运算, 得到完整公钥 $S_{i,j}$;
- 6) PCA 基于 $S_{i,j}$ 构造的 ToBeSignedCertificate 进行签发得到假名证书 $PC_{i,j}$, 再用对应的 $Q_{i,j}$ 对 $PC_{i,j}$ 及 c 进行加密, 对该密文使用 PCA 的私钥进行签名;
- 7) PCA 返回批量 PC 证书至 PRA 时, 将若干个密文发送至 PRA, PRA 检查并筛选出该 V2X 设备对应的若干个密文消息, 将其打包为 PC 证书下载应答;
- 8) V2X 设备收到 PRA 返回的 PC 证书下载应答后, 使用步骤 1) 中生成的 a 和 p , 分别与 k_s 和 k_e 进行若干轮计算, 得到 $b_{i,j}$ 和 $q_{i,j}$;
- 9) 验证步骤 5) 中 PCA 签名的有效性, 验证通过后, 再使用 $q_{i,j}$ 解密对应的密文, 得到 $PC_{i,j}$ 及 c , 最终使用 $b_{i,j}$ 与 c 计算得到 $PC_{i,j}$ 所对应的完整私钥 $s_{i,j}$ 。

注: 下标 (i, j) 表示单个的假名证书所对应的 i 和 j 的组合。其中 i 为该假名证书所对应的周期, j 为该假名证书在当前周期中对应的序号。

注: 以上所述“签名”的种子密钥对(公钥因子、私钥因子)、对称密钥、扩展密钥, 用于衍生 V2X 设备假名证书 PC 内的完整公钥和对应的完整私钥。

注: 以上所述“加密”的种子密钥对(公钥因子、私钥因子)、对称密钥、扩展密钥, 用于衍生一对公私钥, 对 PCA 返回至 V2X 设备的 PC 证书等消息进行加密、解密。

7.2.7 假名证书申请接口及请求和响应传递的参数说明

该接口用于 V2X 设备向假名证书 CA 申请假名证书。

假名证书申请请求类型: HTTP POST 或 HTTPS POST

HTTP Content-Type: application/octet-stream

HTTP request body 为 EeRaCertRequestSpdu 或 EeRaCertRequestPlainSpdu, 详见附录 E.4.10 或 E.4.9, 请求中包含版本信息、时间, 公钥因子, 生成公私钥对的算法等参数;

HTTP response body 为 RaEeCertAckSpdu, 详见附录 E.4.14, 响应中包含请求哈希值等参数。

7.2.8 假名证书下载接口及请求和响应传递的参数说明

该接口用于 V2X 设备从 RA 下载假名证书。

假名证书申请请求类型: HTTP GET 或 HTTPS GET

HTTP response body 为 RaEeCertInfoSpdu, 见附录 E.4.15, 响应中包含假名证书等参数。

注: 使用上述命令时, EeRaDownloadRequestSpdu (附录 E.4.12) 或 EeRaDownloadRequestPlainSpdu (附录 E.4.11) 可携带于 HTTP request headers 中。

7.3 应用证书和身份证书管理流程

7.3.1 概述

应用证书管理流程与身份证书管理流程相同。应用证书管理流程不支持证书更新操作。为了确保数据交互的安全性，V2X设备与应用RA（ARA）之间应根据系统实现方式的不同，采用安全环境、应用层加密、TLS安全通道等措施建立安全连接。

7.3.2 应用证书和身份证书申请流程

7.3.2.1 基于注册证书的应用证书和身份证书申请流程

以应用证书为例，V2X设备应用证书申请流程如图15所示。V2X应用证书申请流程的具体过程为：

- 1) V2X 设备为所申请证书生成用于签名功能的公私密钥对，并依据具体的证书申请协议生成应用证书申请请求消息。该证书申请消息应用使用与 V2X 设备注册证书对应的私钥进行数字签名。若所申请证书中应包含有专门用于数据加密的加密公钥，则还应生成一个用于数据加密的公私密钥对。若此证书申请过程中还需要具有证书签发授权能力的 AAA 对证书签发进行授权，并且需要 V2X 设备提供 AAA 执行授权所需要的与设备和/或车辆相关的设备描述信息和/或其他相关信息，则 V2X 设备应提供相应数据。该数据对于 ARA/ACA 是透明的，因此 V2X 设备可以使用 V2X 设备和相应 AAA 之间的私有安全协议对数据提供安全保护，例如机密性和完整性等保护。V2X 可提供 AAA 的地址信息。基于具体的安全需求，可对应用证书申请消息提供机密性保护。机密性保护应使用接收方的公钥证书中的公钥对应用证书申请请求消息进行公钥加密保护。若接收方公钥证书中包含有加密公钥，则应优先使用该加密公钥。（见附录 E. 4. 9 或附录 E. 4. 10，附录 E. 3. 15，附录 E. 3. 16）
- 2) V2X 设备将应用证书申请请求消息发送给 ARA。
- 3) ARA 应验证应用证书申请请求消息中的注册证书是否有效，例如有效期和是否已被撤销等，然后使用已验证的 V2X 注册证书验证消息签名是否正确。若该消息为加密消息，则在验证消息签名之前应使用与加密公钥对应的私钥对加密消息进行解密。
- 4) ARA 向 V2X 设备返回应用证书申请响应。响应中包含所申请证书的下载时间和与所下载证书相关的标识等信息（见附录 E. 4. 14，附录 E. 3. 15，附录 E. 3. 26）。
- 5) 若此证书签发过程需要具有证书签发授权能力的 AAA 的授权，则 ARA 应向该 AAA 发送证书签发授权请求。授权请求中应包含有该 V2X 设备的标识信息和描述证书应用领域的应用标识信息，并可包含有描述证书具体适用范围的权限描述信息和描述该应用对 V2X 设备本身的设备需求描述信息。若 V2X 设备证书请求消息中包含有 V2X 设备提供给 AAA 的数据，则应在该授权请求中包含有该数据。AAA 的地址可由 V2X 设备通过其证书申请消息提供，或由 ARA 本地配置的数据确定。本标准不规定此步骤的具体实现方法。
- 6) AAA 检查 ARA 发送的授权请求，包括包含在授权请求中的由 V2X 设备生成的车辆和/或设备描述信息，确定是否允许向该 V2X 设备签发所申请的应用证书，并将授权结果通知给 ARA。本标准不规定此步骤的具体实现方法。
- 7) ARA 基于 AAA 的授权结果和本地证书签发策略确定是否向该 V2X 设备签发证书。若允许，则为该 V2X 设备生成一个应用证书生成请求，并将该请求发送给 ACA。（见附录 E. 3. 15，附录 E. 3. 16；附录 E. 6. 6，附录 E. 6. 2，附录 E. 6. 3）
- 8) ACA 基于 ARA 发送的证书生成请求为该 V2X 设备生成一个应用证书。
- 9) ACA 将生成的应用证书通过证书生成响应发送给 ARA（见附录 E. 4. 3 或附录 E. 4. 4，附录 E. 3. 3，附录 E. 3. 4；附录 E. 6. 7，附录 E. 6. 2，附录 E. 6. 5）。
- 10) ARA 使用接收到的 V2X 设备应用证书数据生成证书下载包，并存储证书下载包以供 V2X 设备下载。（见附录 E. 4. 15，附录 E. 3. 15，附录 E. 3. 20）

- 11) V2X 设备向 ARA 发送应用证书下载请求，其中包含有与下载证书相关的标识信息。（见附录 E. 4. 11 或附录 E. 4. 12，附录 E. 3. 15，附录 E. 3. 21）
- 12) ARA 依据下载请求中的标识信息，获取存储在本地的 V2X 应用证书下载包，然后通过应用证书下载响应发送给 V2X 设备。
- 13) V2X 设备验证应用证书下载响应，并存储签发的应用证书。

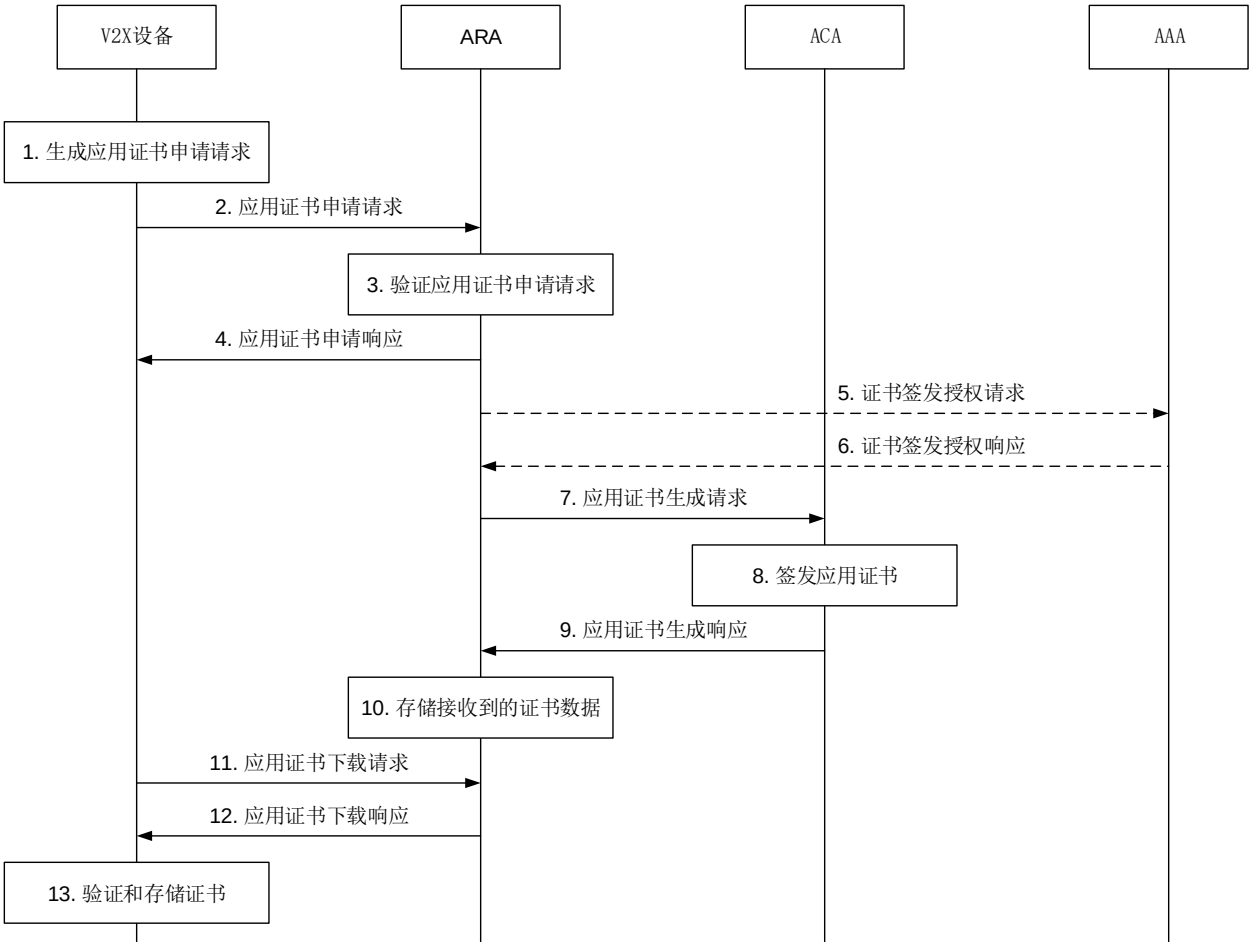


图15 基于注册证书的应用证书申请流程

7. 3. 2. 2 基于 GBA 的应用证书和身份证书申请流程

基于GBA的应用证书和身份证书申请流程如图16所示，具体流程如下：

- 1) 以应用证书为例，申请应用证书时，若 V2X 设备没有有效的 GBA 共享会话密钥，V2X 设备通过蜂窝网接入 GBA 认证授权系统，发起认证授权请求。
- 2) GBA 机制遵从 3GPP TS 33.220 实现，优先采用 GBA_U 方式，成功后向 V2X 设备返回认证授权响应。GBA 认证授权系统中的 BSF 负责对 V2X 设备进行身份认证并向 NAF 提供 GBA 密钥，NAF 负责产生多个 GBA 共享会话密钥供 ARA 使用，密钥的具体生成方式见附录 F。
- 3) V2X 设备请求安全模块为应用证书产生密码公私钥对，组建 AC 应用证书申请请求，并请求 USIM 使用 EC 证书私钥对其签名。
- 4) V2X 设备使用 USIM 生成的 GBA 共享会话密钥 K1 和 K2 对 AC 申请请求消息进行加密和完整性保护。在需要传输设备和/或车辆相关的设备描述或用户身份信息以供授权检验时，V2X 设备可使用 USIM 生成的 GBA 共享会话密钥 K5 对相关信息进行加密保护。

- 5) V2X 设备通过 GBA 认证授权系统向应用证书注册机构 ARA 发送经过保护的应用证书申请请求消息。其中包含加密后的设备描述或用户身份信息。应用证书申请请求消息由 HTTP 消息承载，HTTP 消息中携带 B-TID，ARA 服务器域名信息。
- 6) GBA 认证授权系统根据 ARA 服务器域名信息向 ARA 转发应用证书申请请求，其中携带 B-TID。
- 7) ARA 向 GBA 认证授权系统发送消息，对 V2X 设备的操作请求提出授权申请并申请获取 GBA 共享会话密钥及用户信息。消息中可包含有加密后的设备描述或用户身份信息。该请求采用 HTTP 消息形式，其中携带 B-TID，ARA 服务器域名信息。
- 8) GBA 认证授权系统生成 GBA 共享会话密钥 K5，并使用 GBA 共享会话密钥 K5 解密设备描述或用户身份信息，并授权检验。
- 9) 授权成功后，GBA 认证授权系统生成 GBA 共享会话密钥 K1、K2、K3、K4，并向 ARA 确认授权并返回用户信息、GBA 共享会话密钥 K1、K2、K3、K4 及密钥生存期。如果未成功，GBA 认证授权系统反馈授权失败响应。
- 10) ARA 使用获得的 GBA 共享会话密钥 K1 和 K2 对应用证书申请请求消息进行完整性保护校验和解密，之后审核应用证书申请请求，同时获得 V2X 设备应用证书相关公钥。
- 11) 对于无法即刻签发的 AC 证书，审核通过后 ARA 可生成应用证书申请响应消息，其中包含 AC 应用证书下载时间，以便通知 V2X 设备到时间后下载证书。该消息使用 GBA 共享会话密钥 K1 和 K2 进行加密和完整性保护，通过 GBA 认证授权系统向 V2X 设备转发，应答证书申请成功。
- 12) 审核通过后，ARA 向应用证书签发机构 ACA 发送 AC 应用证书申请请求，请求中包含 V2X 设备应用证书相关公钥。
- 13) ARA 签发 AC 应用证书。
- 14) ACA 将生成的 AC 证书发送至 ARA。
- 15) 对于步骤 11 向 V2X 设备已发送过应用证书响应消息的情况，ARA 存储 AC 应用证书，等待 V2X 设备请求下载。
- 16) 对于步骤 11 向 V2X 设备未发送过应用证书响应消息的情况，ARA 生成应用证书响应消息，其中包括签发的应用证书。该消息使用 GBA 共享会话密钥 K1 和 K2 进行加密和完整性保护，通过 GBA 认证授权系统向 V2X 设备转发，应答证书申请成功。
- 17) V2X 设备使用 GBA 共享会话密钥 K1 和 K2 对应用证书响应消息进行完整性保护校验和解密，安全存储申请获得的应用证书。
- 18) 基于 GBA 共享会话密钥 K3，ARA 与 V2X 设备可选进行双向身份认证。
- 19) 基于 GBA 共享会话密钥 K4，V2X 设备与 ARA 之间可选建立安全通信通道，如 TLS，应用层加密等，用于数据端到端安全传输。

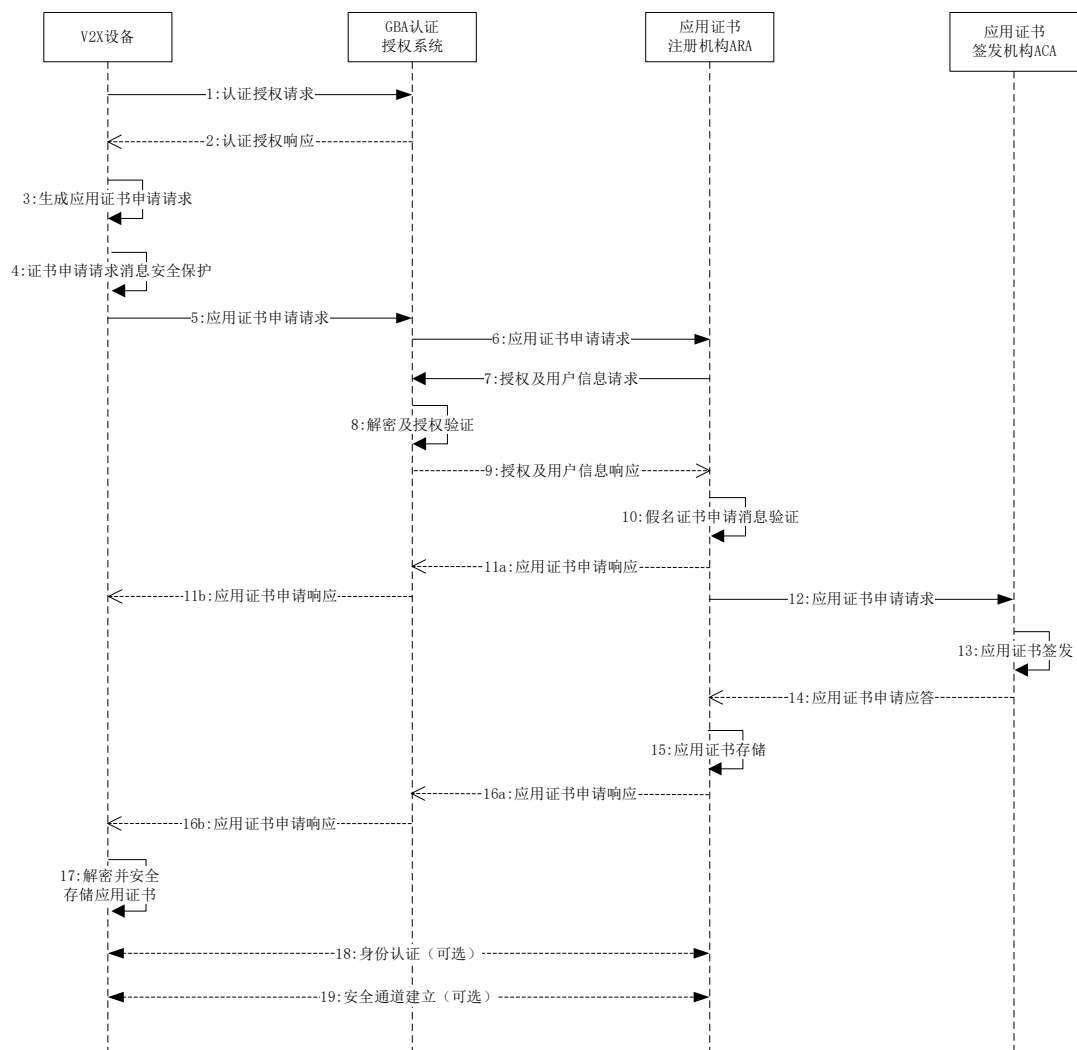


图16 基于 GBA 的应用证书申请流程

7.3.3 应用证书和身份证书申请接口及请求和响应传递的参数说明

该接口用于V2X设备向应用证书CA申请应用证书或身份证书。

应用证书申请请求类型：HTTP POST或HTTPS POST

HTTP Content-Type: application/octet-stream

HTTP request body为EeRaCertRequestSpdu或EeRaCertRequestPlainSpdu，详见附录E.4.10或E.4.9, 请求中包含版本信息、时间、公钥等参数；

HTTP response body为RaEeCertAckSpdu，详见附录E.4.14, 响应中包含应用证书等参数。

7.3.4 应用证书和身份证书下载接口及请求和响应传递的参数说明

该接口用于V2X设备向应用证书CA申请应用证书或身份证书。

身份证书申请请求类型：HTTP GET或HTTPS GET

HTTP response body为RaEeCertInfoSpdu，详见附录E.4.15, 响应中包含身份证书等参数。

注：使用上述命令时，EeRaDownloadRequestSpdu（附录E.4.12）或EeRaDownloadRequestPlainSpdu（附录E.4.11）可携带于HTTP request headers中。

7.4 证书撤销列表管理流程

7.4.1 概述

本节描述V2X通信证书的撤销管理。V2X通信证书包括注册证书、假名证书、应用证书、身份证书。

V2X通信证书的撤销，主要基于异常行为机制实现。当MA收到针对某V2X设备的异常行为上报信息后，根据MA的异常行为决策策略，判断该V2X通信证书是否需要撤销。当判断该V2X通信证书需要撤销时，由MA负责该设备假名、应用、身份证书CRL的签发。同时，PRA/ARA也可将注册证书加入黑名单。当收到假名、应用、身份证书证书申请时，PRA/ARA可基于黑名单拒绝相应的证书签发申请。

当V2X通信证书的撤销列表过大时，可根据实际需求，针对不同的V2X设备类型、证书类型、地理区域的证书，签发不同的CRL，以减小单个CRL文件的大小。

本标准支持全量和增量CRL，CRL签发机构可根据实际情况，签发全量和增量CRL。

证书撤销列表使用的AID应为十进制3628。

注：本节仅描述各证书撤销的主体流程，不涉及其具体实现定义，例如：如何判断某个lv所在LA的主机地址等。

7.4.2 假名证书撤销列表管理

7.4.2.1 概述

假名证书的撤销，本标准仅支持基于LinkageValue的撤销机制。基于LinkageValue的证书撤销列表的验证算法，见附录I.2和I.3。

7.4.2.2 基于ls的假名证书撤销

当某张假名证书被判断为需要撤销时，需要撤销同属于该证书所在链接链（属于同一个ls的所有lv值组成的链）上的一批假名证书。具体流程如图17所示：

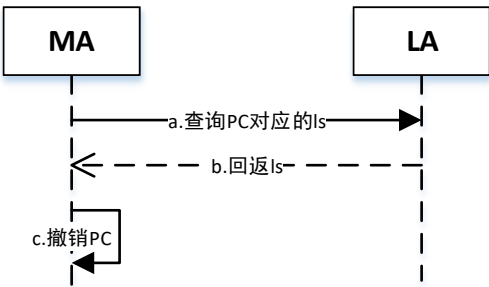


图17 假名证书撤销

- 具体步骤如下：
- 1) MA 基于 PC 及其签发者，识别其对应的 LA，向相应 LA 查询待撤销的 PC 对应的 ls；
 - 2) LA 查询 lv 对应的 ls，并返回给 MA；
 - 3) MA 基于 ls 参数等，根据具体场景需要，将待撤销的证书信息放入 CRL 内，定期签名发布。

7.4.3 黑名单管理

7.4.3.1 概述

当假名证书、应用证书、身份证书撤销时，并需要将其对应的EC放入黑名单时，可通过本节流程实现。

7.4.3.2 基于 Is 的 EC 黑名单管理

当基于Is的假名证书被撤销后，可通过本流程将Is对应的V2X设备EC证书放入黑名单。本流程仅适用于基于EC申请的假名证书的架构。具体流程如图18所示：

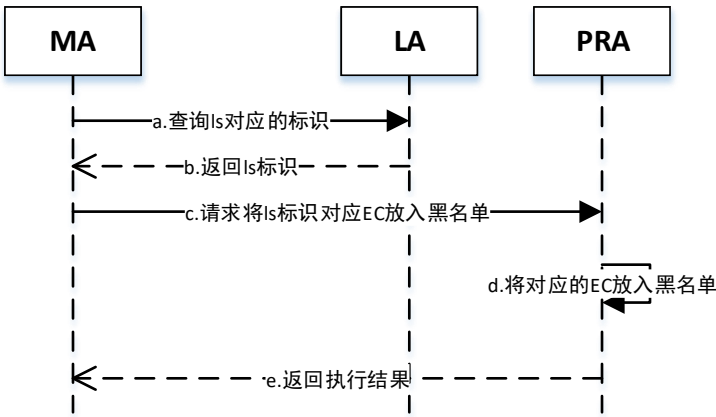


图18 基于 Is 的 EC 黑名单管理

具体步骤如下：

- 1) MA 基于 Is 向 LA 查询 PRA 申请该 Is 对应的标识；
- 2) LA 返回 Is 标识，该标识可为 PRA 发送给 LA 的 1v 申请请求的标识；
- 3) MA 通知 PRA 将 Is 标识对应的的 EC 放入黑名单；
- 4) PRA 将相应的 EC 放入黑名单；
- 5) PRA 返回执行结果。

7.4.3.3 基于 Is 的 OAuth 黑名单管理

当基于Is的假名证书被撤销后，可通过本流程将Is对应的车辆身份标识放入黑名单。本流程仅适用于基于OAuth申请的假名证书的架构。具体流程如图19所示：

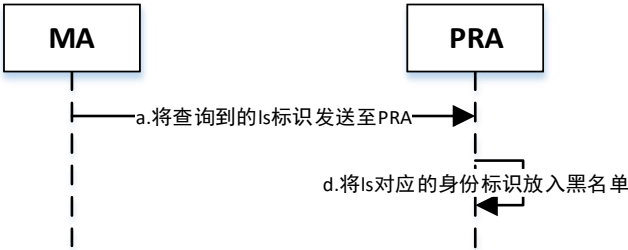


图19 基于 Is 的 OAuth 黑名单管理

具体步骤如下：

- 1) MA 将查询到的 Is 标识发送至 PRA；

2) PRA 将 1s 对应的车辆身份标识放入黑名单。

7.4.3.4 基于应用/身份证书的 EC 黑名单管理

当某张应用/身份证书被撤销后，可通过本流程将其对应的V2X设备EC证书放入黑名单。本流程仅适用于基于EC申请的应用/身份证书的架构。具体流程如图20所示：

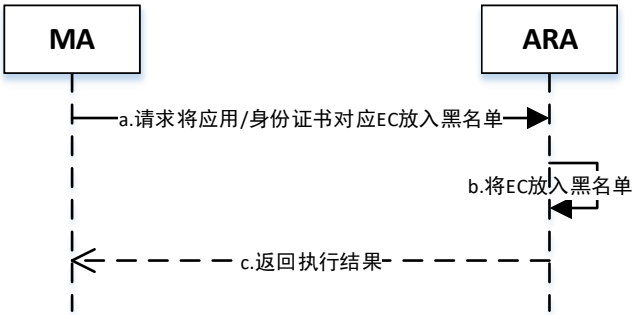


图20 基于应用/身份证书对应的 EC 黑名单管理

具体步骤如下：

- 1) MA 通知 ARA 将应用或身份证书对应的 EC 放入黑名单；
- 2) ARA 将相应的 EC 放入黑名单；
- 3) 返回执行结果

7.4.4 证书撤销列表下载流程

V2X设备从RA下载由证书撤销机构CRA签发的证书撤销列表CRL，其流程如图21 所示，在下载CRL前，V2X设备应和RA建立TLS安全通道，步骤如下：

- 1) 证书撤销机构 CRA 根据 CRA 的证书撤销列表更新策略，定期更新 CrlContents（见 7.4.6.2 ），并使用 CRA 的私钥对 CrlContents 签名得到 SecuredCrl（见附录 C.3.2）；
- 2) CRA 将生成的 CRL 下发至 RA（CRL 服务）；
- 3) V2X 设备根据设备的 CRL 更新策略，向 RA 发送 CRL 下载请求，请求中包含了 cracaId 和 crlSeries；
- 4) RA 将与请求中 cracaId 和 crlSeries 相对应的 CRL 放入下载应答中，以 SecuredCrl 的形式发送给 V2X 设备；
- 5) V2X 设备先验证 SecuredCrl 的签名有效性和内容完整性，然后判断是否更新本地 CRL（根据 cracaId、crlSeries 和 crlSerial 进行判断），若需要更新，则将 CrlContents 写入本地安全存储中。

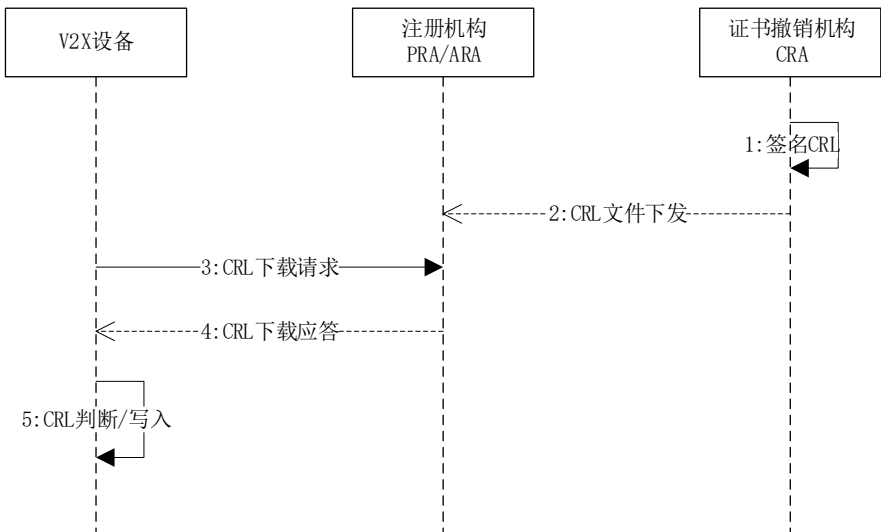


图21 证书撤销列表下载流程

7.4.5 证书撤销列表申请接口及请求和响应传递的参数说明

该接口用于V2X设备向证书撤销CA申请证书撤销列表。
证书撤销列表申请请求类型：HTTP GET或HTTPS GET
响应中包含证书撤销列表等参数。

7.4.6 CRL 数据结构

7.4.6.1 概述

本节使用ASN.1编码定义CRL内容，附录C.3为CRL提供了完整的ASN.1模版。
CRL不为每个条目单独列出信息字段，而是嵌套字段以提供更紧凑的编码，这是因为嵌套在外层的字段预期有最少的不同值。特别地，CRL设计时假设CA按特定方式组织证书发放，从而使得可能在同一CRL中出现的所有证书（即，具有相同的crlSeries和cracaId值的证书）同时使用相同的iCert值。

7.4.6.2 CrlContents

```
CrlContents ::= SEQUENCE {
    version          Uint8 (1),
    crlSeries        CrlSeries,
    crlCraca         HashedId8,
    issueDate        Time32,
    nextCrl          Time32,
    priorityInfo     CrlPriorityInfo,
    typeSpecific     CHOICE {
        fullHashCrl      ToBeSignedHashIdCrl,
        deltaHashCrl     ToBeSignedHashIdCrl,
        fullLinkedCrl    ToBeSignedLinkageValueCrl,
```

```

        deltaLinkedCrl      ToBeSignedLinkageValueCrl,
        ...
    }
}

```

此结构中的字段具有如下含义：

- version 是 CRL 的版本号，本标准对应的版本号是 1；
- crlSeries 代表 CRL 所属的 CRL 系列。这用于确定 CRL 中的撤销信息是否与特定证书相关；
- crlCraca 包含最终授权发布此 CRL 的证书撤销授权 CA（CRACA）证书的哈希低八位字节，它用于确定 CRL 中的撤销信息是否与特定证书相关。带有效签名的 CRL，其 crlCraca 应与服务特定权限中的 associatedCraca 字段一致。HashedId8 使用证书的所有字段经哈希算法计算得到，如附录 D 中所述；
- issueDate 指示 CRL 的发布时间；
- nextCrl 包含预期发出具有相同 crlSeries 和 crlCraca 的下一个 CRL 的时间。除非 nextCrl 严格位于 issueDate 之后，否则 CRL 无效。此字段用于设置与（crlCraca, crlSeries）相关联的撤销信息的预期更新时间；
- priorityInfo 所包含的信息可协助存储空间有限的设备确定要保留哪些撤销信息以及丢弃哪些撤销信息；
- typeSpecific 包含 CRL 正文：
 - fullHashCrl 包含一个完整的基于散列值的 CRL，即包含所有以下证书的散列列表：
 - ◆ 包含指定的 cracaId 和 crlSeries 值，以及
 - ◆ 被哈希撤销的，和
 - ◆ 已被撤销的，和
 - ◆ 尚未过期的；
- deltaHashCrl 包含基于增量散列的 CRL，即所有以下证书的散列列表：
 - 包含指定的 cracaId 和 crlSeries 值，以及
 - 被哈希撤销的，和
 - 自上一个包含指定 cracaId 和 crlSeries 值的 CRL 以来已被撤销的；
- fullLinkedCrl 包含一个完整的基于链接 ID 的 CRL，即包含所有如下证书的个体和/或群组链接数据的列表：
 - 包含指定的 cracaId 和 crlSeries 值，以及
 - 被链接 ID 数据撤销的，和
 - 已被撤销的，和
 - 尚未过期的；
- deltaLinkedCrl 包含基于增量链接 ID 的 CRL，即包含所有如下证书的个体和/或群组链接数据的列表：
 - 包含指定的 cracaId 和 crlSeries 值，以及
 - 被链接 ID 数据撤销的，和
 - 自上一个包含指定 cracaId 和 crlSeries 值的 CRL 以来已被撤销的；

7.4.6.3 CrlPriorityInfo

```

CrlPriorityInfo ::= SEQUENCE {
    priority          Uint8 OPTIONAL,

```

```
...
}
```

该数据结构承载优先级信息，它能协助具有有限存储空间的设备确定要保留哪些撤销信息以及丢弃哪些撤销信息。

——Priority 指示相对于为具有相同 cracaId 和 crlSeries 值的证书发布的其他 CRL、本撤销信息的优先级。此字段的值越高，表示该撤销信息的重要性越高。

7.4.6.4 ToBeSignedHashIdCrl

```
ToBeSignedHashIdCrl ::= SEQUENCE {
    crlSerial      Uint32,
    entries        SequenceOfHashBasedRevocationInfo,
    ...
}
```

此数据结构包含如下信息：

- crlSerial 是一个计数器，每次为指定的 crlCraca 和 crlSeries 值发出新的完整或增量 CRL 时，该计数器递增 1；
- entries（条目）包含各个撤销信息项。

7.4.6.5 HashBasedRevocationInfo

```
HashBasedRevocationInfo ::= SEQUENCE {
    id      HashedId10,
    expiry   Time32,
    ...
}
```

```
SequenceOfHashBasedRevocationInfo ::=
    SEQUENCE OF HashBasedRevocationInfo
```

此数据结构包含如下信息：

- id 是 CertId10，用于标识已撤销的证书。HashedId10 使用证书的所有字段经哈希算法计算得到，如附录 D 中所述；
- expiry 为到期时间，它根据该证书中有效期的开始和持续时间值计算得到。

7.4.6.6 ToBeSignedLinkageValueCrl

```
ToBeSignedLinkageValueCrl ::= SEQUENCE {
    iRev          IValue,
    indexWithinI   Uint8,
    individual     SequenceOfJMaxGroup OPTIONAL,
    groups         SequenceOfGroupCrlEntry OPTIONAL,
    ...,
    groupsSingleSeed SequenceOfGroupSingleSeedCrlEntry OPTIONAL
}
(WITH COMPONENTS {..., individual PRESENT} |
```

WITH COMPONENTS {..., groups PRESENT})

此数据结构包含如下信息：

- iRev 是基于链接值的撤销算法的一个输入，它适用于 individual 或 groups 的链接值撤销；
- indexWithinI 是一个计数器。为特定的 crlCraca、crlSeries 和 iRev 值组合发出第一个 CRL 时，该计数器被置为 0，以后每次为同一 crlCraca 和 crlSeries 组合值发出新的完整或增量 CRL 且不更改 iRev 时，计数器都增加 1；
- individual 包含个体关联数据；
- groups 包含双种子群组链接数据，也即这些链接值都使用双种子算法推导得到。
- groupsSingleSeed 包含单种子群组链接数据，也即这些链接值都使用单种子算法推导得到。

当使用群组链接值方式对证书进行批量撤销时，如果所有被撤销证书的链接值都是通过双种子算法计算得到的，则出现 groups 字段，而 groupsSingleSeed 字段不呈现；如果所有被撤销证书的链接值都是通过单种子算法计算得到的，则 groups 字段依然呈现，但其 SEQUENCE 的长度应该为 '0'。

7.4.6.7 JMaxGroup

```
JMaxGroup ::= SEQUENCE {
    jmax          Uint8,
    contents      SequenceOfLAGroup,
    ...
}
```

SequenceOfJMaxGroup ::= SEQUENCE OF JMaxGroup

此数据结构包含如下信息：

- jMax 用于基于链接值的撤销算法，此值适用于 contents 中包含的所有撤销信息；
- contents 包含个体链接数据。

7.4.6.8 LAGroup

```
LAGroup ::= SEQUENCE {
    la1Id          GroupSingleSeedCrlEntry
    GroupSingleSeedCrlEntry ::= SEQUENCE {
        iMax          Uint16,
        laId          错误!书签自引用无效。，
        linkageSeed    LinkageSeed
    }
}
```

SequenceOfGroupSingleSeedCrlEntry ::= SEQUENCE OF GroupSingleSeedCrlEntry

此数据结构包含如下信息：

- iMax 指示对于这些证书、一旦 iCert > iMax，就不再需要计算撤销信息，这是因为能知道持有者没有更多对应于 (crlCraca, crlSeries) 的有效证书；
- laId 作为 LinkageAuthorityIdentifier 参数用于基于单种子链接值的撤销算法；
- linkedSeed 作为 LinkageSeed 参数用于基于单种子链接值的撤销算法；

```
LaId,
la2Id          GroupSingleSeedCrlEntry
```

```

GroupSingleSeedCrlEntry ::= SEQUENCE {
    iMax          Uint16,
    laId          错误!书签自引用无效。 ,
    linkageSeed    LinkageSeed
}

```

SequenceOfGroupSingleSeedCrlEntry ::= SEQUENCE OF GroupSingleSeedCrlEntry

此数据结构包含如下信息：

- iMax 指示对于这些证书、一旦 $iCert > iMax$ ，就不再需要计算撤销信息，这是因为能知道持有者没有更多对应于 (crlCraca, crlSeries) 的有效证书；
- laId 作为 LinkageAuthorityIdentifier 参数用于基于单种子链接值的撤销算法；
- linkedSeed 作为 LinkageSeed 参数用于基于单种子链接值的撤销算法；

```

LaId,
    contents          SequenceOfIMaxGroup,
    ...
}

```

SequenceOfLAGroup ::= SEQUENCE OF LAGroup

此数据结构包含如下信息：

- 对于双种子链接值计算算法，la1Id 作为 LinkageAuthorityIdentifier1 参数用于基于链接值的撤销算法；对于单种子链接值计算算法，la1Id 作为 LinkageAuthorityIdentifier 参数用于基于链接值的撤销算法。la1Id 适用于 contents 中包含的所有撤销信息；
- 对于双种子链接值计算算法，la2Id 作为 LinkageAuthorityIdentifier2 参数用于基于链接值的撤销算法，此值适用于 contents 中包含的所有撤销信息；对于单种子链接值计算算法，la2Id 无意义，应被接收方忽略；
- contents 包含个体链接数据。

7.4.6.9 IMaxGroup

```

IMaxGroup ::= SEQUENCE {
    iMax          Uint16,
    contents          SequenceOfIndividualRevocation,
    ...,
    singleSeeds      SequenceOfLinkageSeed
}

```

SequenceOfIMaxGroup ::= SEQUENCE OF IMaxGroup

此数据结构包含如下信息：

- iMax 指示 Contents 或 singleSeeds 中的条目数目。一旦 $iCert > iMax$ ，则能知道持有者没有更多有效证书，故不再需要计算撤销信息。iMax 不直接用于计算链接值，但用于确定何时可以安全删除撤销信息；
- contents 包含双种子个体链接数据，这些个体链接值通过双种子算法推导得到；
- singleSeeds 包含单种子个体链接数据，这些个体链接值通过单种子算法推导得到；

7.4.6.10 IndividualRevocation

```
IndividualRevocation ::= SEQUENCE {
    linkageSeed1    LinkageSeed,
    linkageSeed2    LinkageSeed,
    ...
}
```

SequenceOfIndividualRevocation ::= SEQUENCE OF IndividualRevocation

此数据结构包含如下信息：

- linkedSeed1 作为 LinkageSeed1 参数用于基于双种子链接值的撤销算法；
- linkedSeed2 作为 LinkageSeed2 参数用于基于双种子链接值的撤销算法；

7.4.6.11 GroupCrlEntry

```
GroupCrlEntry ::= SEQUENCE {
    iMax          Uint16,
    laId          GroupSingleSeedCrlEntry
GroupSingleSeedCrlEntry ::= SEQUENCE {
    iMax          Uint16,
    laId          错误!书签自引用无效。 ,
    linkageSeed    LinkageSeed
}
```

SequenceOfGroupSingleSeedCrlEntry ::= SEQUENCE OF GroupSingleSeedCrlEntry

此数据结构包含如下信息：

- iMax 指示对于这些证书、一旦 $i_{\text{Cert}} > i_{\text{Max}}$ ，就不再需要计算撤销信息，这是因为能知道持有者没有更多对应于 (crlCraca, crlSeries) 的有效证书；
- laId 作为 LinkageAuthorityIdentifier 参数用于基于单种子链接值的撤销算法；
- linkedSeed 作为 LinkageSeed 参数用于基于单种子链接值的撤销算法；

```
LaId,
    linkageSeed1    LinkageSeed,
    la2Id          GroupSingleSeedCrlEntry
GroupSingleSeedCrlEntry ::= SEQUENCE {
    iMax          Uint16,
    laId          错误!书签自引用无效。 ,
    linkageSeed    LinkageSeed
}
```

SequenceOfGroupSingleSeedCrlEntry ::= SEQUENCE OF GroupSingleSeedCrlEntry

此数据结构包含如下信息：

- iMax 指示对于这些证书、一旦 $i_{\text{Cert}} > i_{\text{Max}}$ ，就不再需要计算撤销信息，这是因为能知道持有者没有更多对应于 (crlCraca, crlSeries) 的有效证书；
- laId 作为 LinkageAuthorityIdentifier 参数用于基于单种子链接值的撤销算法；

——linkedSeed 作为 LinkageSeed 参数用于基于单种子链接值的撤销算法；

```

LaId,
    linkageSeed2      LinkageSeed,
    ...
}

```

SequenceOfGroupCrlEntry ::= SEQUENCE OF GroupCrlEntry

此数据结构包含如下信息：

- iMax 指示对于这些证书、一旦 $iCert > iMax$ ，就不再需要计算撤销信息，这是因为能知道持有者没有更多对应于（crlCraca, crlSeries）的有效证书；
- la1Id 作为 LinkageAuthorityIdentifier1 参数用于基于双种子链接值的撤销算法；
- linkedSeed1 作为 LinkageSeed1 参数用于基于双种子链接值的撤销算法；
- la2Id 作为 LinkageAuthorityIdentifier2 参数用于基于双种子链接值的撤销算法；
- linkedSeed2 作为 LinkageSeed2 参数用于基于双种子链接值的撤销算法；

7.4.6.12 GroupSingleSeedCrlEntry

```

GroupSingleSeedCrlEntry ::= SEQUENCE {
    iMax          Uint16,
    laId          错误!书签自引用无效。 ,
    linkageSeed    LinkageSeed
}

```

SequenceOfGroupSingleSeedCrlEntry ::= SEQUENCE OF GroupSingleSeedCrlEntry

此数据结构包含如下信息：

- iMax 指示对于这些证书、一旦 $iCert > iMax$ ，就不再需要计算撤销信息，这是因为能知道持有者没有更多对应于（crlCraca, crlSeries）的有效证书；
- laId 作为 LinkageAuthorityIdentifier 参数用于基于单种子链接值的撤销算法；
- linkedSeed 作为 LinkageSeed 参数用于基于单种子链接值的撤销算法；

7.4.6.13 LaId

```

GroupSingleSeedCrlEntry
GroupSingleSeedCrlEntry ::= SEQUENCE {
    iMax          Uint16,
    laId          错误!书签自引用无效。 ,
    linkageSeed    LinkageSeed
}

```

SequenceOfGroupSingleSeedCrlEntry ::= SEQUENCE OF GroupSingleSeedCrlEntry

此数据结构包含如下信息：

- iMax 指示对于这些证书、一旦 $iCert > iMax$ ，就不再需要计算撤销信息，这是因为能知道持有者没有更多对应于（crlCraca, crlSeries）的有效证书；
- laId 作为 LinkageAuthorityIdentifier 参数用于基于单种子链接值的撤销算法；

——linkedSeed 作为 LinkageSeed 参数用于基于单种子链接值的撤销算法；

LaId ::= OCTET STRING (SIZE(2))

此数据结构包含 LA Identifier 参数，它用于基于链接值的撤销算法。

7.4.6.14 LinkageSeed

LinkageSeed ::= OCTET STRING (SIZE(16))

SequenceOfLinkageSeed ::= SEQUENCE OF LinkageSeed

此数据结构包含链接种子值，它适用于单种子链接值的撤销算法；

7.4.6.15 CrlSsp

```
CrlSsp ::= SEQUENCE {
    version Uint8(1),
    associatedCraca CracaType,
    crls PermissibleCrls,
    ...
}
```

此数据结构包含如下信息：

- version 是 SSP 的版本号，对于此版本的 SSP 为 1。
- associatedCraca 标识此证书与 CRACA 之间的关系。如果 associatedCraca = isCraca，则此证书为 CRACA 证书，并对链接回该证书的证书签署 CRL。如果 associatedCraca = issuerIsCraca，则此证书的颁发者为 CRACA，并且此证书可以为链接回其颁发者的证书签署 CRL。
- crls 标识证书持有者可以签发哪种类型的 CRL。

7.4.6.16 CracaType

CracaType ::= ENUMERATED {isCraca, issuerIsCraca}

CracaType 数据结构用于确定 CRL 的 CrlContents 结构中 crlCraca 字段的有效性：

- 如果 CracaType 为 isCraca，则 CRL 的 CrlContents 结构中 crlCraca 字段与签名 CRL 的证书一致，该 CRL 才有效。
- 如果 CracaType 为 issuerIsCraca，则 CRL 的 CrlContents 结构中 crlCraca 字段与签名 CRL 的证书的签发者证书一致，该 CRL 才有效。

7.4.6.17 PermissibleCrls

PermissibleCrls ::= SEQUENCE OF CrlSeries

PermissibleCrls 数据结构用于确定 CRL 的 CrlContents 结构中 crlSeries 字段的有效性。当 CrlContents 结构中 crlSeries 的值出现在 PermissibleCrls 中时，该 CRL 才有效。

7.5 机构证书管理流程

7.5.1 概述

本节描述机构证书的管理。机构证书包括 RootCA、MA、ICA、ECA、PCA、ACA、PRA、ARA、LA 证书。

7.5.2 机构证书发布与更新

一个PKI系统的机构证书存储在由该PKI系统的RootCA签发的证书可信列表（Certificate Trust List，CTL）中。（CTL的数据结构参见附录E3.8）

当机构证书有更新时，RootCA应发布新的CTL，同时旧的CTL自动作废。

机构证书CTL申请接口及请求和响应传递的参数说明：

机构证书CTL申请请求类型：HTTP GET或HTTPS GET

响应中包含证书信任列表等参数。

7.5.3 机构证书撤销管理

当机构证书需要撤销时，由RootCA负责为其签发CRL。当机构证书被撤销时，RootCA应据此发布新的CTL。

当ICA、ECA、PCA、ACA的机构证书被撤销时，不需要额外为其签发的下级证书签发CRL，这些证书将自动失效。

机构证书CRL申请接口及请求和响应传递的参数说明：

证书撤销列表申请请求类型：HTTP GET或HTTPS GET

响应中包含证书撤销列表等参数。

7.6 异常行为检测上报流程

V2X异常行为是指在V2X直连通信中，部分节点由于有意或故障的原因，对外发送包含虚假或错误内容的信息，从而对于V2X直连通信自组网络中其他的通信节点造成影响其进行业务判断的行为。其中恶意的V2X行为攻击者可能对于整体V2X直连通信自组网络的运转效率和安全性带来严重的影响。因此，需要考虑在V2X直连通信自组网中实现V2X异常行为检测的能力。

V2X异常行为检测，通常包括如下过程：

- 1) 本地异常行为检测：终端设备根据接收到的周边V2X消息，以及本地的检测算法，判断V2X消息是否异常；
- 2) 上报：对于确定的V2X异常行为，需要上报给专门的异常行为管理机构（Misbehavior Authority, MA）进行进一步的检测。
- 3) 全局异常行为检测：MA根据收到的多方上报的本地异常行为检测报告信息，汇总并决策是否确实是异常行为；
- 4) 证书撤销：对于确认为异常行为的终端设备，需要对其证书进行撤销。

异常行为管理使用的AID应为十进制3629。

7.7 LA管理架构和流程

7.7.1 概述

LA的管理主要包括lv获取和ls查询。其中，链接值的算法，见附录I.1。

7.7.2 获取链接值流程

基于lv签发个体PC证书时，PRA可通过此流程获取设备生命周期内的链接值lvs，流程如图22所示。

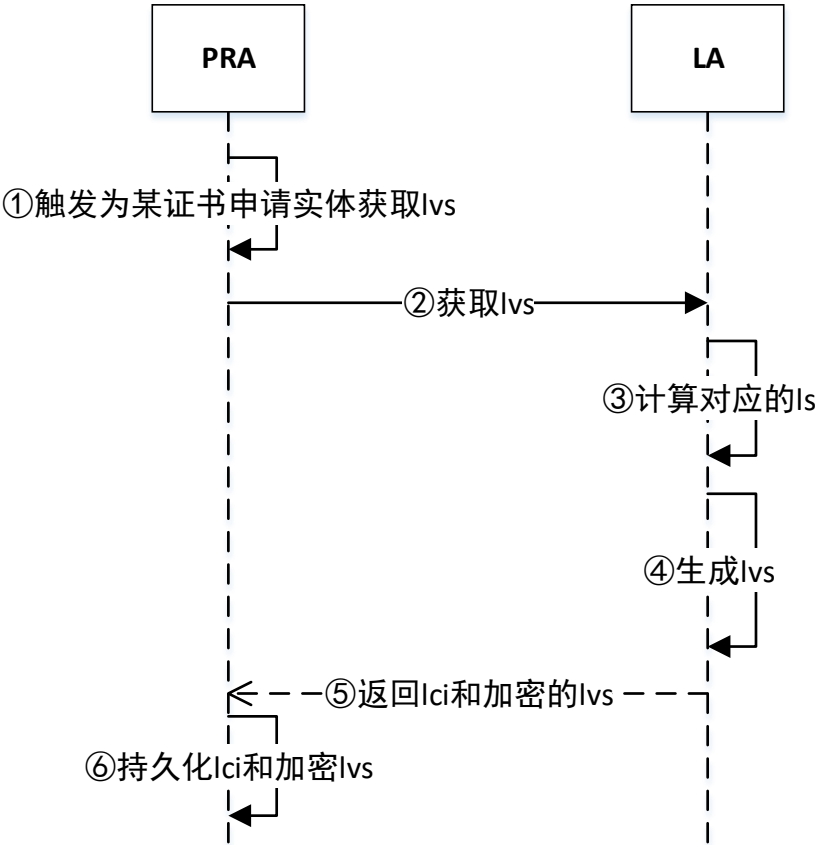


图22 获取链接值流程

链接值获取过程如下：

- 1) PRA 触发为某证书申请实体获取链接值 lvs。
- 2) PRA 构建相应结构体向 LA 获取链接值 lvs。
- 3) 若是为某个设备首次申请链接值，LA 会随机生成 128bit 的随机数作为该设备的初始链接种子 $ls(0)$ ，并根据 PRA 发送的 $iMin$ 计算链接种子 $ls(iMin)$ 。若不是首次申请，则申请时会携带上次的 lci（链接链标识符，用于标识一条链接链），链接机构会通过 lci 查找到对应的链接种子 $ls(i)$ ，并将 $ls(i)$ 作为 $ls(iMin-1)$ ，计算本次的链接种子 $ls(iMin)$ 。
- 4) LA 通过链接种子生成一批 lvs 和 lci，并将 lvs 与 ls 进行关联（撤销时使用），ls 与 lci 进行关联（PRA 下次为该设备申请链接值时使用）。
- 5) LA 将 lci 和加密的 lvs 返回给 PRA。
- 6) PRA 持久化 lci 和加密的 lvs。

7.7.3 获取组链接值流程

基于 glv 签发支持基于组链接值撤销的 PC 证书时，PRA 可通过此流程获取设备生命周期内的 glvs，流程如图 23 所示。

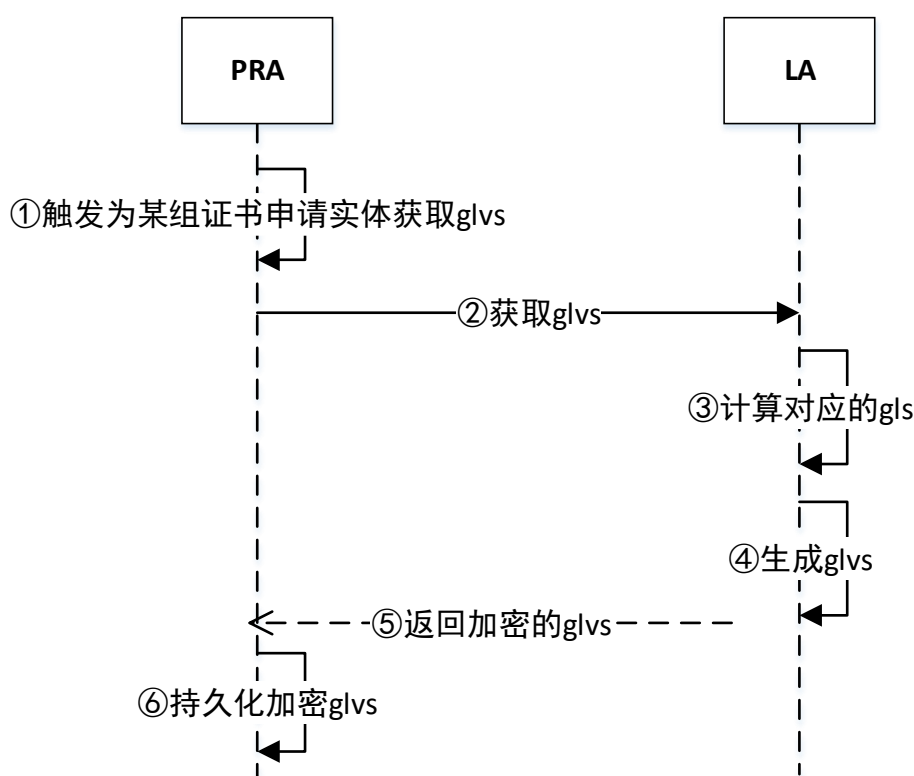


图23 获取组链接值流程

获取组链接值流程如下：

- 1) PRA 触发为某组证书申请实体获取组链接值 glvs。
- 2) PRA 通过 groupIdentifier 构建相应结构体向 LA 获取组链接值 glvs
- 3) LA 会通过收到的 groupIdentifier 查找是否之前有接受过该 groupIdentifier。如果没有接收过，LA 会随机生成 128bit 的对称密钥作为该组设备的初始组链接种子 gls(0)，并根据 PRA 发送的 iMin 计算对应的组链接种子 gls(iMin)；如果 LA 通过该 groupIdentifier 查找到对应的组链接种子 gls(i)，则使用 gls(i)作为 gls(iMin-1)对应的组链接种子，并计算本次所需的组链接种子 gls(iMin)。
- 4) LA 通过组链接种子 gls 生成一批组链接值 glvs，并将 glvs 与 gls 进行关联(撤销时使用)，将 gls 与 groupIdentifier 进行关联(PRA 下次为该组设备请求时使用)。
- 5) LA 将加密的组链接值 glvs 返回给 PRA
- 6) PRA 持久化 glvs。

7.7.4 查询链接种子流程

如果在异常行为调查期间，假名证书被识别为行为异常，MA可通过此流程，获取假名证书对应的链接种子，用于批量撤销证书，流程如图24所示。

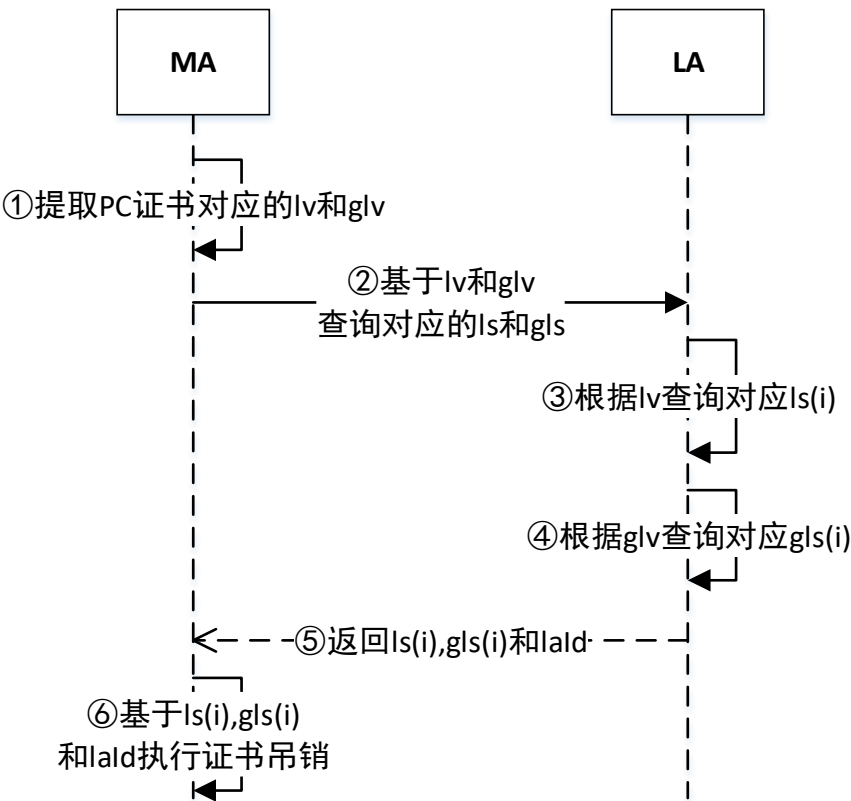


图24 查询链接种子流程

查询链接种子的流程如下：

- 1) MA 取出出现异常行为的假名证书中的链接值 lv 和组链接值 glv，并将其发送给 LA。
- 2) MA 通过 lv 和 glv 构建响应结构体请求 LA，以查询对应的 ls 和 gls。
- 3) LA 通过映射关系由链接值 lv 找到对应的链接种子 ls(i)，
- 4) LA 通过映射关系由组链接值 glv 找到对应的组链接种子 gls(i)。
- 5) LA 将链接种子 ls(i)，组链接种子 gls(i) 和 LA 唯一标识符 laId 一起返回给 MA。
- 6) MA 将 laId，gls(i) 和 ls(i) 一起添加到 CRL 进行撤销。

7.7.5 请求个体链接值接口及请求和响应传递的参数说明

该接口用于PRA向PCA提交PC申请前，PRA向LA请求特定证书申请实体生命周期的链接值。

请求类型：HTTP POST。

HTTP Content-Type: application/octet-stream

HTTP request body为PraLaIndividualLinkageValueRequestSpdu，见附录E.8.13，请求中包含 praId、pcaId、iMin、iMax、jMax、新链的个数、可选的链接标识符等参数；

HTTP response body为LaPraLinkageValueResponseSpdu，见附录E.8.15，响应中包含链接标识符、由PCA加密公钥保护的的一批lv值等参数。

7.7.6 请求组链接值接口及请求和响应传递的参数说明

该接口用于PRA向PCA提交PC申请前，PRA向LA请求一组证书生命周期的组链接值。

请求类型：HTTP POST。

HTTP Content-Type: application/octet-stream

HTTP request body为PraLaGroupLinkageValueRequestSpdu，见附录E.8.14，请求中包含praId、pcaId、iMin、iMax、jMax、groupIdentifier等参数；

HTTP response body为LaPraLinkageValueResponseSpdu，见附录E.8.15，响应中包含由PCA加密公钥保护的一批glv值等参数。

7.7.7 基于链接值查询链接种子接口及请求和响应传递的参数说明

该接口用于证书撤销时，为MA提供需要撤销的假名证书对应的链接种子。

请求类型：HTTP POST。

HTTP Content-Type: application/octet-stream

HTTP request body为MaLaLinkageSeedRequestSpdu，见附录E.7.9，请求中包含maId、lv、glv等参数；

HTTP response body为LaMaLinkageSeedResponseSpdu，见附录E.7.10，响应中包含ls(i)、gls(i)、laId等参数。

8 LTE-V2X 通信安全认证 PKI 互信技术要求

8.1 概述

在车联网系统中，可能会有多个独立的PKI系统为车联网设备提供证书服务，每个PKI的服务范围称为一个认证域。在本标准中，认证域是指由一个PKI系统全部证书构成的可信域。跨认证域认证是指位于一个认证域中的车联网设备能够认证由其他认证域签发给该域车联网设备的证书。本章节描述了在车联网系统里实现跨域认证的技术规范。

8.2 PKI 互信架构

8.2.1 概述

为实现跨域认证，一个认证域中的设备需要获取另一个认证域签发证书的CA证书或证书链。本标准将一个认证域提供给另一个认证域的用于互信操作的顶级CA证书为可信根CA证书（Trusted Root Certificate）。该顶级CA证书可以是该认证域的自签名的根CA证书，也可以是该认证域的非自签名的子CA证书。

PKI互信架构如图25所示。该架构由如下功能实体和数据构成：

- 可信根证书列表管理机构（Trusted Root Certificate List Authority, TRCLA）：负责签发可信根证书列表。
- 可信根证书列表（Trusted Root Certificate List, TRCL）：由可信的PKI系统的根证书、可信的PKI系统的可信域CA证书列表下载地址和保护可信根证书列表的安全机制构成。保护可信根证书列表的安全机制为数字签名技术。
- 可信根证书列表签名证书：用于对可信根证书列表提供数字签名保护的公钥证书。该证书可以是符合本标准定义的车联网证书，也可以是符合X.509证书规范的证书。
- 可信证书管理功能（Trusted Certificate Management Function, TCMF）：在一个PKI系统内负责与可信根证书列表管理机构交互，向可信根证书列表管理机构提供本PKI系统与互信

操作相关的数据，从可信根证书列表管理机构获取实现 PKI 互信所需要的数据和向本 PKI 系统内的车联网设备提供实现 PKI 互信所需要的数据。

——可信域 CA 证书列表（Trusted Domain CA Certificates List，TDCL）：由一个 PKI 系统签发的包含有其他 PKI 系统验证本 PKI 系统所签发证书时需要 CA 证书的列表和保护可信域 CA 证书列表的安全机制构成。列表中的 CA 证书应为从相关 CA 至该 PKI 根的证书链。保护可信域 CA 证书列表的安全机制为数字签名技术。

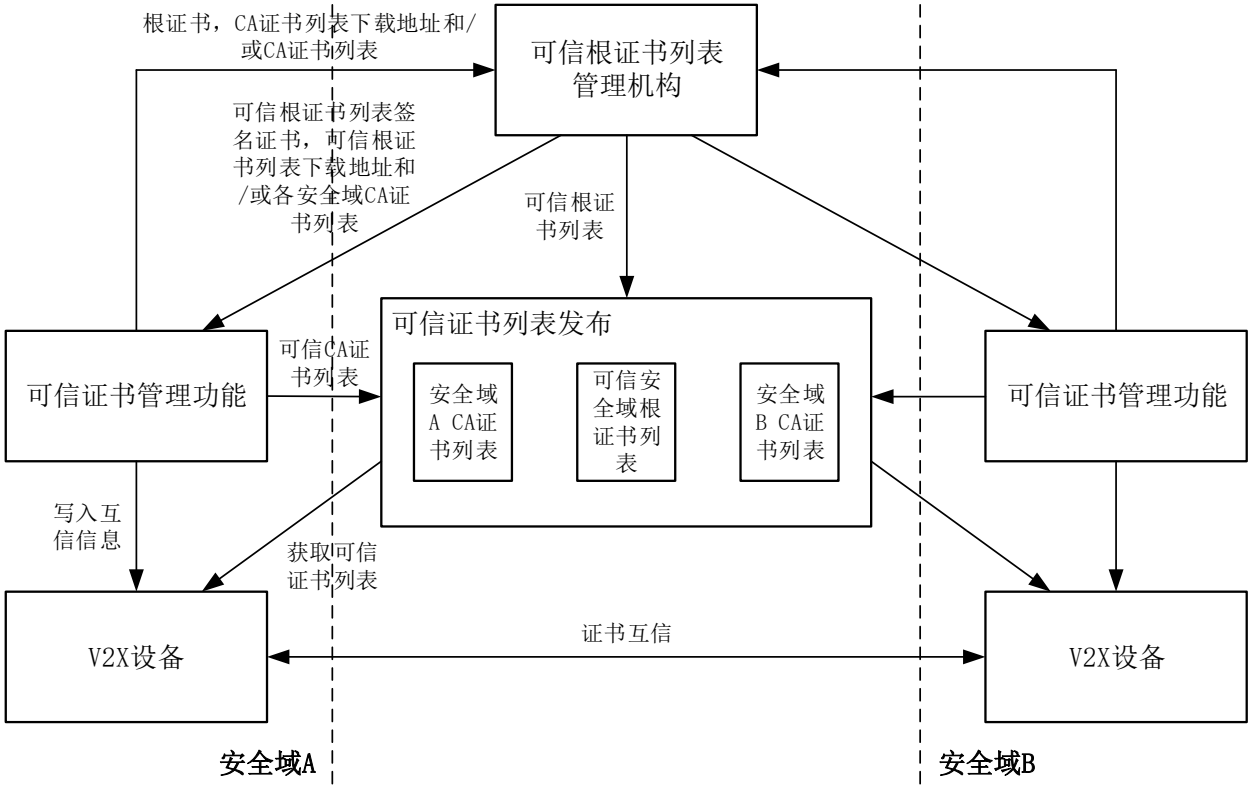


图25 PKI 互信架构

可信根证书列表（TRCL）和可信域CA证书列表（TDCL）的发布可以分为集中式和分布式两种。

集中式发布方式是指可信根证书列表管理机构（TRCLA）将TRCL和各个安全域的TDCL发送给各个安全域，然后由各个安全域确定如何分发给域内的车联网设备。

分布式发布方式是指向车联网设备提供下载TRCL和各个安全域的TDCL的地址，然后由车联网设备自行下载交叉认证时需要的各个认证域的CA证书。

车联网设备可内置TRCLA签名证书，通过对TRCL的验证建立对其它域根证书的信任，通过使用TRCL中的各个根证书验证TDCL，建立对其它域内各个CA的信任，进而具备验证其它域内实体证书的能力，具体过程见8.3.1和8.3.2节。若TRCL和TDCL由各个安全域分发，各安全域分发的内容应与从TRCLA获得的内容一致。

TRCLA的签名证书生命周期不在本标准讨论的范围内。TRCLA证书的更新和到期替换，PKI互信管理机构应通知各安全域进行相应处理。

8.2.2 可信根证书列表结构

可信根证书列表结构及各字段的用途如表8所示。TRCL的ASN.1结构见附录J.3。

表8 可信根证书列表结构

数据域 1	数据域 2	数据域 3	是否必选	说明
版本		version	是	描述列表结构的版本。与本标准对应的版本号为 1。
签发者		issuer	是	签发此列表的证书的 HashedId8 值
序列号		series	是	每次更新列表，序列号应较上一次更新加 1。
签发时间		issueDate	是	签发时间
下次签发时间		nextRootCtl	是	预计下次签发时间
根证书列表	某可信 PKI 系统相关的数据	rootCertificate	是	某 PKI 系统的可信根 CA 证书
		caListUrl	否	某可信 PKI 系统的可信域 CA 证书列表下载地址
				
签名值		signature	是	可信根证书列表的签名值

8.2.3 可信域 CA 证书列表结构

可信域CA证书列表结构及各字段的用途如表9所示。TDCL的ASN.1结构见附录J.4。

表9 可信域 CA 证书列表结构

数据域 1	数据域 2	数据域 3	是否必选	说明
版本		Version	是	描述列表结构的版本。与本标准对应的版本号为 1。
签发者		issuer	是	签发此列表的证书的 HashedId8 值
序列号		series	是	每次更新列表，序列号应较上一次更新加 1。
签发时间		issueDate	是	签发时间
下次签发时间		nextPkiCtl	是	预计下次签发时间
可信域 CA 证书列表	可信 CA 证书	certificateChain	是	可信 CA 证书的证书链
		crlUrl	否	该 CA 所签发证书的 CRL 列表下载地址
		maUrl	否	该 CA 所签发证书异常行为上报地址
				
签名值		signature	是	可信域 CA 证书列表的签名值

8.3 PKI 互信管理过程

8.3.1 分布式 PKI 互信管理过程

分布式PKI互信管理的一般过程如图26所示，具体的流程如下：

- 1) “可信根证书列表管理机构”将“可信根证书列表签名证书”和“可信根证书列表下载地址”提供给一个可信 PKI 系统中的“可信证书管理功能”。
- 2) 一个可信 PKI 系统的“可信证书管理功能”生成并发布“可信 CA 证书列表”。该列表的管理和发布应符合相关规范。

- 3) “可信证书管理功能”向“可信根证书列表管理机构”提供其“根证书”和“可信域 CA 证书列表下载地址”。
- 4) “可信根证书列表管理机构”利用各可信 PKI 系统提供的“根证书”和“可信域 CA 证书列表下载地址”生成一个“可信根证书列表”并将该列表发布至可公开下载的网络上。
- 5) 在一个可信 PKI 系统内，“可信证书管理功能”以安全的方式向其域内的“车联网设备”提供“可信根证书列表签名证书”和“可信根证书列表下载地址”。
- 6) “车联网设备”首先依据“可信根证书列表下载地址”下载“可信根证书列表”，然后使用“可信根证书列表签名证书”验证“可信根证书列表”的数字签名。
- 7) 基于下载的“可信根证书列表”，“车联网设备”执行如下操作：
 - a) 获取“可信根证书列表”中的某个列项；
 - b) 使用该列项中的“可信域 CA 证书列表下载地址”下载“可信域 CA 证书列表”；
 - c) 使用该列项中的“根证书”验证下载的“可信域 CA 证书列表”的数字签名；
 - d) 使用该列项中的“根证书”验证“可信域 CA 证书列表”中证书链的正确性。
 - e) 若“可信域 CA 证书列表”提供了 CRL 下载地址，则还应下载相应的 CRL。

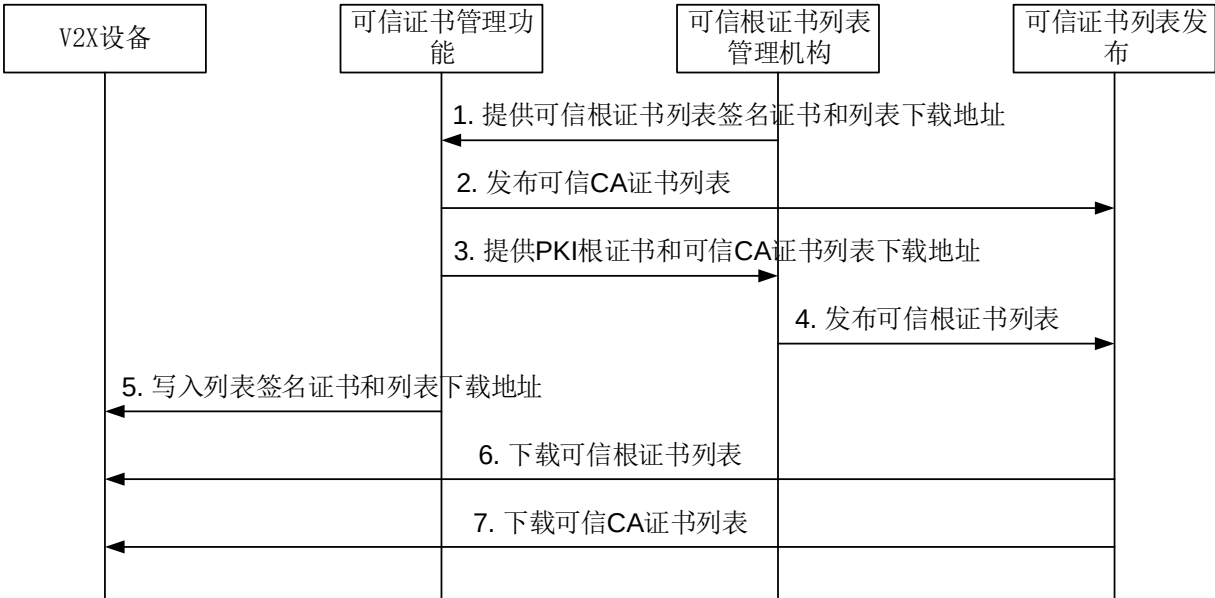


图26 分布式 PKI 互信管理的一般过程

以上互信管理过程的阶段与步骤，在车联网与可信域的具体场景中，可根据实际情况选择执行。

8.3.2 集中式 PKI 互信管理过程

集中式PKI互信管理的一般过程如图27所示，具体的流程如下：

- 1) “可信根证书列表管理机构”将“可信根证书列表签名证书”提供给一个可信 PKI 系统中的“可信证书管理功能”。
- 2) “可信证书管理功能”向“可信根证书列表管理机构”提供其“根证书”和“可信域 CA 证书列表”。
- 3) “可信根证书列表管理机构”生成一个“可信根证书列表”，并将该列表和其他可信域的“可信域 CA 证书列表”提供给可信 PKI 系统的“可信证书管理功能”。

- 4) 可信 PKI 系统的“可信证书管理功能”以安全的方式向其域内的“车联网设备”写入“可信根证书列表签名证书”和各可信列表的下载地址。
- 5) “车联网设备”依据提供的“可信根证书列表下载地址”下载“可信根证书列表”，然后使用“可信根证书列表签名证书”验证“可信根证书列表”的数字签名。
- 6) “车联网设备”依据提供的“可信 CA 证书列表下载地址”下载“可信域 CA 证书列表”，然后利用“可信根证书列表”中相应的可信域根证书验证下载的列表。

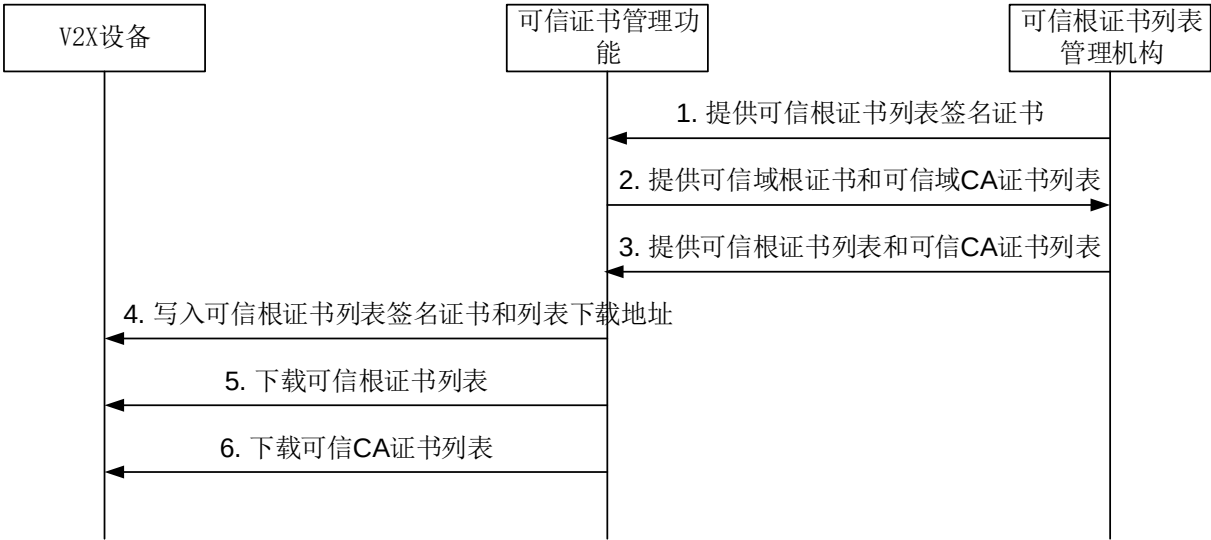


图27 集中式 PKI 互信管理的一般过程

以上互信管理过程的阶段与步骤，在车联网与可信域的具体场景中，可根据实际情况选择性执行。其中，步骤4、5、6也可通过7. 5. 2中的机构证书发布与更新功能来实现。

8.4 PKI 互信认证过程

PKI互信认证的一般过程为：

- a) “车联网设备”接收其他“车联网设备”播发的签名消息。
- b) “车联网设备”获取签名消息中携带的签名证书。
- c) “车联网设备”获取签名证书中证书签发者标识。
- d) “车联网设备”利用获取的证书签发者标识与本 PKI 系统的 CA 证书和其他可信 PKI 系统的 CA 证书的 HashedId8 值相比较，若有匹配，则利用相应的 CA 证书验证消息签名证书，并检查相应的 CRL，以确定该证书是否已被撤销。
- e) “车联网设备”利用通过验证的消息签名证书验证消息签名。

8.5 可信根证书列表管理策略

若需要修改“可信根证书列表”中的信息，例如加入新的根证书或移除已有的根证书，则需要发布新的“可信根证书列表”。“车联网设备”应能够获取这种更新信息，并下载新的“可信根证书列表”。“车联网设备”如何能够获取这种更新信息不属于本标准规定的范围。

8.6 可信域 CA 证书列表管理策略

若需要修改“可信域CA证书列表”中的信息，例如加入新的CA证书或移除已有的CA证书，则需要发布新的“可信域CA证书列表”。“车联网设备”应能够获取这种更新信息，并下载新的“可信域CA证书列表”。“车联网设备”如何能够获取这种更新信息不属于本标准规定的范围。

8.7 可信域的异常行为检查

当车联网设备检查到某可信域设备的异常行为时，若该域提供了异常行为上报地址，则应将该异常行为上报至该地址，否则上报至设备缺省配置的上报地址。

附录 A

（资料性附录）

车联网通信安全基本应用模式

A.1 概述

本附录描述基于LTE的车联网安全通信的基本应用模式，但不限于这些应用模式。

A.2 车辆主动安全系统模式

在车辆主动安全系统中，车辆之间会播发其行驶状态信息。该行驶状态信息需要使用假名CA签发给OBU的假名证书提供数字签名的保护。假名证书利用密码技术隐藏OBU或车辆的身份信息，以保护用户的隐私。OBU首先使用与假名证书对应的私钥对包含有其行驶状态信息的信息进行数字签名；然后OBU将该签名消息连同证书一起广播出去。周围接收到该消息的车辆首先利用签发假名证书的CA的证书验证消息中的签名证书是否有效，然后利用通过验证的证书中的公钥验证签名消息中的签名是否正确，最后接收车辆利用通过验证的签名消息中的内容确定消息发送车辆的行驶状态。

A.3 路侧设备广播消息模式

道路设施管理机构（应用CA）向道路基础设施（RSU），例如交通信号灯和交通信息发布实施等，签发应用证书；应用证书描述了道路基础设施能够播发的内容。路侧设备利用应用证书对应的私钥对其播发的消息进行数字签名，并将其广播出去，该广播消息中还包含有应用证书。周围的OBU接收到该消息。OBU首先利用应用CA的CA证书验证该广播消息中的RSU应用证书；如果通过验证，继续利用该RSU证书中的公钥验证签名消息中的数字签名；若通过验证，则可以使用该广播消息携带的内容，例如当前红绿灯的状态和即将切换的时间。

A.4 特权车辆与路侧设备间通信模式

特权车辆，例如警车，能够控制一些道路基础设施的状态，例如交通信号灯。道路设施管理机构（应用CA）向道路基础设施（RSU）签发应用证书；应用证书描述了道路基础设施能够播发的内容。道路设施管理机构向特权车辆（OBU）签发身份证书；该身份证书中描述了授权车辆拥有的控制道路基础设施的权限。特权车辆上的OBU接收道路基础设施发送的签名消息，其中包含有道路基础设施的RSU应用证书。OBU首先生成控制RSU状态的控制指令，并利用其身份证书对该控制指令进行数字签名；然后利用RSU的应用证书中的公钥对签名消息进行加密，生成签名加密消息，签名消息中还包含有OBU的身份证书；最后将签名加密消息广播出去。RSU接收到该签名加密消息，首先利用与其公钥证书对应的私钥对消息进行解密，然后验证OBU的公钥证书，进而验证消息签名，最终获得控制指令，并执行该控制指令。

由路侧设备发送的广播信息也可以是由应用服务提供商签名的各种应用信息，例如道路交通信息、天气信息、本地商业信息等。

A.5 基于应用的OBU与RSU/VSP通信模式

本场景以使用V2X通信系统实现道路收费为场景描述基于应用的OBU与RSU/VSP通信。道路收费管理机构（应用CA）向负责道路收费的RSU签发应用证书；应用证书中描述了这些RSU具有道路收费的权限，证书中还描述了证书适用的地理区域，确定该RSU适用的范围。道路收费管理机构向OBU签发用于道路缴费的身份证书。OBU道路缴费身份证书中描述了该OBU在道路收费系统中的唯一身份和车辆类型等信息。

在道路收费入口，OBU获得入口RSU的应用证书；OBU使用其身份证书和入口RSU应用证书生成签名加密消息，并发送给入口RSU。入口RSU解密并验证OBU的签名加密消息，确定OBU的起始位置。

在道路收费出口，OBU获得出口RSU的应用证书；OBU使用其身份证书和出口RSU应用证书生成签名加密消息，并发送给出口RSU。出口RSU解密并验证OBU的签名加密消息，确定OBU的结束位置。

附 录 B

（资料性附录）

基于 OAUTH 的 token 授权机制

B.1 概述

在传统的授权访问方法中，如果用户需要第三方应用访问自己存储在某个实体中的信息资源，需要将自己的身份凭证共享给访问者，而OAuth 2.0是为访问授权提供的一组开放标准。它通常被用户用来授权第三方网站或应用访问他们在其他资源服务器上存储的信息，但不需要共享自己的身份凭证，除此之外它还定义了认证和授权机制，称为客户端凭证授权（CCG: Client Credentials Grant）。本附录描述OAuth 2.0标准由一系列RFCs组成，其中 RFC 6749定义了OAuth 2.0的框架，另外一些RFCs基于这个框架定义了一些附加的特性。（具体参考<https://www.oauth.com/oauth2-servers/map-oauth-2-0-specs/>）。

如果在V2X安全体系中使用OAuth 2.0以及利用客户端凭证授权CCG为承载授权，需要在现有SCMS体系架构内添加一个组件OAuth授权服务器(OAS)和两个附带策略：

- 1) OAS 应具备相应验证策略，该策略定义了客户端为了获得授权和获得访问令牌(AT: Access Token)而必须满足的需求。
- 2) RA 应具有 AT 验证策略，该策略定义了如果客户端需要访问相关的服务 API，则客户端携带的 AT 必须满足的需求。

B.2 协议流程

需要访问RA服务的终端实体，应发送授权请求来访问RA服务，例如，使用协议约定的预先存储在终端的‘客户端凭证’向OAS（OAuth Authorization Server）发送授权证书请求。

OAS应根据OAS策略准许或拒绝授权。如果准许授权，OAS应发布针对特定服务的AT。

ATs应该是预先定义的包含JSON 的Web令牌(JWT)。

AT应至少包含以下报头：

- “typ”：令牌(Token)类型-应为“JWT”
- “alg”：在 JWT 中使用的签名算法的标识符

AT应至少包含以下已注册的有效载荷：

- “iss”：“iss”（发布方）是一个区分大小写的字符串，包含一个 StringOrURI 值，该值能够标识发布 JWT 的主体。
- “aud”：“aud”（接收方）定义了 JWT 的目标接收方。每一个意图处理 JWT 的主体都必须在“aud”中识别出自己。如果没有在“aud”中识别出自己，则该 JWT 必须被拒绝。通常情况下，“aud”是一组区分大小写的字符串，每个字符串包含一个 OrURI 值字符串。在特殊情况下（JWT 只有一个接收主体），“aud”值可能是单一一个字符串，该字符串中包含一个 StringOrURI 值。
- “iat”：“iat”（签发于）确定了 JWT 签发的时间，从而识别重放攻击。它的值必须是一组

包含 NumericDate 值的数字。

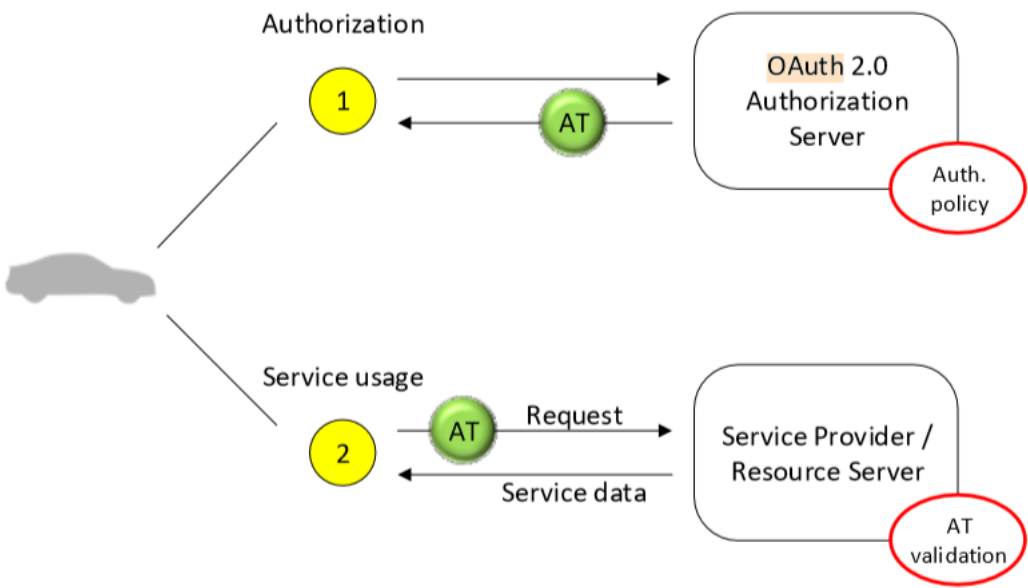
—— “exp”：“exp”（过期时间）标识了过期时间，到达或者超过这个时间之后 JWT 将不被接受。
即要求当前日期/时间必须早于“exp”声明中列出的到期日期/时间。

注：有关StringOrURI和NumericDate的详细信息和定义，请参阅RFC 7519。

AT应包含“cnf”（确认）声明，用来声明JWT的提交人的所有权证明，并保证提交人拥有遵循RFC 7800格式的特定所有权证明密钥。“cnf”的值应是“kid”（密钥标识），包含“<iss>.<kid>”形式的公钥标识符。其中报头“iss”（AT的发行者）是<iss>，<kid>是所使用的签名密钥的唯一标识符。

OAS应根据RFC7517提供JWKS API。资源服务器或服务提供者应该定期获取最新的可用密钥集并缓存这些密钥，以便能够验证JWT的签名，而不必在每次服务请求时请求所需的公钥。在请求JWKS API时，资源服务器应始终验证授权服务器的TLS证书链，然后进行处理。为此，资源服务器需要TLS根证书和中间CA证书。接收到来自OAS的AT后，终端实体应向服务提供者（例如RA）发送服务请求，例如，授权证书请求，使用HTTP报头“Authorization: Bearer {AT}”，其中{AT}是JWT的占位符。服务提供者根据其AT接受策略接受或拒绝请求。

当OAuth 2.0承载授权用于sessAuthenticationType: webAPI时，应使用本协议流程，如图B.1所示。AT在传输和存储时都应受到安全保护。



基于 OAUTH 的 token 授权机制

B.3 OAuth 2.0 AT的验证策略

OAuth 2.0 AT验证策略应基于AT中的报头信息和声明，在验签AT时，至少应验证以下内容：

- “typ” “typ”（AT 类型）报头应为“JWT”值；
- “iss”：“iss”（发行者）应识别 OAS。
- “aud”：“aud”（接收方）对于本标准中规定的每种服务，应该是唯一的
- “iat”：“iat”（签发时间）声明不应早于 AT 接收方的规定值
- “exp”：与 AT 接收方的当前系统时间相比，“exp”（到期时间）不得早于当前时间超过 5 秒。
- “cnf”：“cnf”（确认）应包含有效的“kid”声明，应解析为 OAS 的 JWKS API 提供的有效公钥，且 AT 的签名应使用此公钥进行验证。

附 录 C
(规范性附录)
ASN.1 模板

C.1 概述

本附录介绍本标准正文部分定义的数据结构的ASN.1编码，格式为一系列ASN.1模板。 这些模板已经与商业编译器一起编译检验，并且在没有警告的情况下完成编译。
如果本附录中的ASN.1编码与本标准正文部分的ASN.1发生冲突，则本标准正文中的ASN.1优先。

C.2 V2X安全服务

C.2.1 V2X安全模板

```
--*****
--
--  LTE-V2X Security Data Types
--
--*****

DEFINITIONS AUTOMATIC TAGS ::= BEGIN

EXPORTS ALL;

IMPORTS
    CrlSeries,
    EccP256CurvePoint,
    EciesP256EncryptedKey,
    EcencP256EncryptedKey,
    EncryptionKey,
    GeographicRegion,
    GroupLinkageValue,
    SignedData ::= SEQUENCE {
        hashId          错误!书签自引用无效。 ,
        tbsData          ToBeSignedData,
        signer           SignerIdentifier,
        signature        Signature
    }
```

该数据结构中包含：
——hashId 指示签名和验签时、对消息进行散列处理所使用的哈希算法；

——tbsData 包含散列后作为签名输入的数据；

——signer 确定对数据进行签名时所使用的密钥材料和哈希算法；

——signature 为数字签名

- 如果 signer 指示选择 self，则签名计算参数化如下：
 - ◆ Data（数据）输入等于 tbsData 字段的 COER 规范化编码；
 - ◆ Verification type（验签类型）等于 self；
 - ◆ Signer identifier（签名者标识符）输入等于空字符串。
- 如果 signer 指示 certificate 或 digest，则签名计算参数化如下：
 - ◆ Data（数据）输入等于 tbsData 字段的 COER 规范化编码；
 - ◆ Verification type（验签类型）输入 等于 certificate；
 - ◆ Signer identifier（签名者标识符）输入等于用于验证 SPDU 的证书的 COER 编码。

HashAlgorithm,

HashedId3,

HashedId8,

HashedId32,

Hostname,

IValue,

LinkageValue,

Opaque,

Aid,

AidSsp,

AidSspRange,

PublicEncryptionKey,

PublicVerificationKey,

SequenceOfHashedId3,

SequenceOfAidSsp,

SequenceOfAidSspRange,

ServiceSpecificPermissions,

Signature,

SubjectAssurance,

SymmetricEncryptionKey,

ThreeDLocation,

本标准中采用十进制方式对AID取值进行编码。

Time64,

Uint3,

Uint8,

Uint16,

Uint32,

ValidityPeriod

--

```

--*****
--
-- Structures for describing secured data
--
--*****

-- this structure belongs later in the file but putting it here avoids
-- compiler errors with certain tools
SignedDataPayload ::= SEQUENCE {
    data          V2XSecData OPTIONAL,
    extDataHash    HashedData OPTIONAL,
    ...
}

(WITH COMPONENTS {..., data PRESENT} |
 WITH COMPONENTS {..., extDataHash PRESENT})

V2XSecData ::= SEQUENCE {
    protocolVersion    Uint8(3),
    content             错误!未找到引用源。
}

错误!未找到引用源。 ::= CHOICE {
    unsecuredData      Opaque,
    signedData         SignedData,
    encryptedData      EncryptedData,
    signedCertificateRequest Opaque,
    ...
}

SignedData ::= SEQUENCE {
    hashId          SignedData ::= SEQUENCE {
        hashId      错误!书签自引用无效。 ,
        tbsData      ToBeSignedData,
        signer       SignerIdentifier,
        signature     Signature
    }
}

```

该数据结构中包含：

- hashId 指示签名和验签时、对消息进行散列处理所使用的哈希算法；
- tbsData 包含散列后作为签名输入的数据；
- signer 确定对数据进行签名时所使用的密钥材料和哈希算法；
- signature 为数字签名
 - 如果 signer 指示选择 self，则签名计算参数化如下：

- ◆ Data（数据）输入等于 tbsData 字段的 COER 规范化编码；
- ◆ Verification type（验签类型）等于 self；
- ◆ Signer identifier（签名者标识符）输入等于空字符串。
- 如果 signer 指示 certificate 或 digest，则签名计算参数化如下：
 - ◆ Data（数据）输入等于 tbsData 字段的 COER 规范化编码；
 - ◆ Verification type（验签类型）输入 等于 certificate；
 - ◆ Signer identifier（签名者标识符）输入等于用于验证 SPDU 的证书的 COER 编码。

HashAlgorithm,

```

    tbsData          ToBeSignedData,
    signer           SignerIdentifier,
    signature        Signature
}

```

```

SignerIdentifier ::= CHOICE {
    digest          HashedId8,
    certificate     SequenceOfCertificate,
    self           NULL,
    ...,
    x509           OCTET STRING
}

```

```

ToBeSignedData ::= SEQUENCE {
    payload         SignedDataPayload,
    headerInfo      HeaderInfo
}

```

```

HashedData ::= CHOICE {
    sha256HashedData OCTET STRING (SIZE(32)),
    ...,
    sha384HashedData OCTET STRING (SIZE(48)),
    sm3HashedData   OCTET STRING (SIZE(32))
}

```

```

HeaderInfo ::= SEQUENCE {
    aid              Aid,
    generationTime   本标准中采用十进制方式对AID取值进行编码。
Time64 OPTIONAL,
    expiryTime       本标准中采用十进制方式对AID取值进行编码。
Time64 OPTIONAL,
    generationLocation ThreeDLocation OPTIONAL,
    p2pcdLearningRequest HashedId3 OPTIONAL,

```

```

missingCrIIdentifier  MissingCrIIdentifier OPTIONAL,
encryptionKey        EncryptionKey OPTIONAL,
...,
inlineP2pcdRequest   SequenceOfHashedId3 OPTIONAL,
requestedCertificate  Certificate OPTIONAL,
pduFunctionalType     PduFunctionalType OPTIONAL
}

```

```

MissingCrIIdentifier ::= SEQUENCE {
    cracaId          HashedId3,
    crlSeries        CrlSeries,
    ...
}

```

```

PduFunctionalType ::= INTEGER (0..255)
    tlsHandshake      PduFunctionalType ::= 1
    iso21177ExtendedAuth PduFunctionalType ::= 2

```

```

--*****
--
-- Structures for describing encrypted data
--
--*****

```

```

EncryptedData ::= SEQUENCE {
    recipients      SequenceOfRecipientInfo,
    ciphertext       SymmetricCiphertext
}

RecipientInfo ::= CHOICE {
    pskRecipInfo      PreSharedKeyRecipientInfo,
    symmRecipInfo     SymmRecipientInfo,
    certRecipInfo     PKRecipientInfo,
    signedDataRecipInfo PKRecipientInfo,
    rekRecipInfo       PKRecipientInfo
}

```

```

SequenceOfRecipientInfo ::= SEQUENCE OF RecipientInfo

```

```

PreSharedKeyRecipientInfo ::= HashedId8
SymmRecipientInfo ::= SEQUENCE {
    recipientId       HashedId8,
    encKey             SymmetricCiphertext
}

```

```

}

PKRecipientInfo ::= SEQUENCE {
    recipientId      HashedId8,
    encKey           EncryptedDataEncryptionKey
}

EncryptedDataEncryptionKey ::= CHOICE {
    eciesNistP256      EciesP256EncryptedKey,
    eciesBrainpoolP256r1 EciesP256EncryptedKey,
    ...,
    ecencSm2          EcencP256EncryptedKey
}

SymmetricCiphertext ::= CHOICE {
    aes128ccm          One28BitCcmCiphertext,
    ...,
    sm4Ccm            One28BitCcmCiphertext
}

One28BitCcmCiphertext ::= SEQUENCE {
    nonce             OCTET STRING (SIZE (12)),
    ccmCiphertext     Opaque
}

--*****
--
-- Certificates and other security management data structures
--
--*****

-- Certificates is explicit (type = explicit,
-- toBeSigned includes verification key, signature present) or Reserved (Reserved
for implicit cert).

Certificate ::= CertificateBase (ExplicitCertificate | Reserved)

SequenceOfCertificate ::= SEQUENCE OF Certificate

CertificateBase ::= SEQUENCE {
    version           Uint8(3),
    type              CertificateType,

```

```

        issuer                IssuerIdentifier,
        toBeSigned             ToBeSignedCertificate,
        signature              Signature OPTIONAL
    }

```

```

CertificateType ::= ENUMERATED {
    explicit,
    reserved,
    ...
}

```

```

ExplicitCertificate ::= CertificateBase (WITH COMPONENTS {...,
    type(explicit),
    toBeSigned(WITH COMPONENTS {...,
        verifyKeyIndicator(WITH COMPONENTS {verificationKey})
    })),
    signature PRESENT
})

```

```

Reserved ::= CertificateBase (WITH COMPONENTS {...,
    type(reserved),
    toBeSigned(WITH COMPONENTS {...,
        verifyKeyIndicator(WITH COMPONENTS {reconstructionValue})
    })),
    signature ABSENT
})

```

```

IssuerIdentifier ::= CHOICE {
    sha256AndDigest          HashedId8,
    self                      SignedData ::= SEQUENCE {
        hashId               错误!书签自引用无效。 ,
        tbsData              ToBeSignedData,
        signer               SignerIdentifier,
        signature            Signature
    }
}

```

该数据结构中包含：

- hashId 指示签名和验签时、对消息进行散列处理所使用的哈希算法；
- tbsData 包含散列后作为签名输入的数据；
- signer 确定对数据进行签名时所使用的密钥材料和哈希算法；
- signature 为数字签名

- 如果 signer 指示选择 self，则签名计算参数化如下：

- ◆ Data（数据）输入等于 tbsData 字段的 COER 规范化编码；
- ◆ Verification type（验签类型）等于 self；
- ◆ Signer identifier（签名者标识符）输入等于空字符串。
- 如果 signer 指示 certificate 或 digest，则签名计算参数化如下：
 - ◆ Data（数据）输入等于 tbsData 字段的 COER 规范化编码；
 - ◆ Verification type（验签类型）输入 等于 certificate；
 - ◆ Signer identifier（签名者标识符）输入等于用于验证 SPDU 的证书的 COER 编码。

HashAlgorithm,

```

    ...,
    sha384AndDigest      HashedId8,
    sm3AndDigest         HashedId8
  }

```

```

ToBeSignedCertificate ::= SEQUENCE {
    id                CertificateId,
    cracaId           HashedId3,
    crlSeries         CrlSeries,
    validityPeriod    ValidityPeriod,
    region            GeographicRegion OPTIONAL,
    assuranceLevel     SubjectAssurance OPTIONAL,
    appPermissions    SequenceOfAidSsp OPTIONAL,
    certIssuePermissions SequenceOfAidGroupPermissions OPTIONAL,
    certRequestPermissions SequenceOfAidGroupPermissions OPTIONAL,
    canRequestRollover NULL OPTIONAL,
    encryptionKey     PublicEncryptionKey OPTIONAL,
    verifyKeyIndicator VerificationKeyIndicator,
    ...
}

```

```

(WITH COMPONENTS { ..., appPermissions PRESENT} |
 WITH COMPONENTS { ..., certIssuePermissions PRESENT} |
 WITH COMPONENTS { ..., certRequestPermissions PRESENT})

```

```

CertificateId ::= CHOICE {
    linkageData      LinkageData,
    name             Hostname,
    binaryId         OCTET STRING(SIZE(1..64)),
    none            NULL,
    ...
}

```

```

LinkageData ::= SEQUENCE {

```

```

        iCert                IValue,
        linkage-value        LinkageValue,
        group-linkage-value   GroupLinkageValue OPTIONAL
    }

    EndEntityType ::= BIT STRING {app (0), enroll (1) } (SIZE (8)) (ALL EXCEPT {})

    AidGroupPermissions ::= SEQUENCE {
        subjectPermissions SubjectPermissions,
        minChainLength      INTEGER DEFAULT 1,
        chainLengthRange    INTEGER DEFAULT 0,
        eeType               EndEntityType DEFAULT {app}
    }

    SequenceOfAidGroupPermissions ::= SEQUENCE OF AidGroupPermissions
    SubjectPermissions ::= CHOICE {
        explicit             SequenceOfAidSspRange,
        all                  NULL,
        ...
    }

    VerificationKeyIndicator ::= CHOICE {
        verificationKey      PublicVerificationKey,
        reconstructionValue   EccP256CurvePoint,
        ...
    }

    END

```

C.2.2 V2X安全基本类型

```

--*****
--  LTE-V2X Security Base Data Types
--
--*****
DEFINITIONS AUTOMATIC TAGS ::= BEGIN

EXPORTS ALL;

-- -----
--
--  Integers

```

-- -----

```

Uint3  ::= INTEGER (0..7)                -- (hex)                07
Uint8  ::= INTEGER (0..255)              -- (hex)                ff
Uint16 ::= INTEGER (0..65535)            -- (hex)                ff ff
Uint32 ::= INTEGER (0..4294967295) --<LONGLONG>-- -- (hex)        ff ff ff ff
Uint64 ::= INTEGER (0..18446744073709551615) -- (hex) ff ff ff ff ff ff ff ff

```

SequenceOfUint8 ::= SEQUENCE OF Uint8

SequenceOfUint16 ::= SEQUENCE OF Uint16

-- -----

--

-- OCTET STRING types

--

-- -----

Opaque ::= OCTET STRING

HashedId32 ::= OCTET STRING (SIZE(32))

HashedId10 ::= OCTET STRING (SIZE(10))

HashedId8 ::= OCTET STRING (SIZE(8))

HashedId4 ::= OCTET STRING (SIZE(4))

HashedId3 ::= OCTET STRING (SIZE(3))

SequenceOfHashedId3 ::= SEQUENCE OF HashedId3

-- -----

--

-- Time

--

-- -----

Time32 ::= Uint32

本标准中采用十进制方式对AID取值进行编码。

Time64 ::= Uint64

```

ValidityPeriod ::= SEQUENCE {
    start          Time32,

```

```

        duration      Duration
    }

```

```

Duration ::= CHOICE {
    microseconds      Uint16,
    milliseconds      Uint16,
    seconds            Uint16,
    minutes            Uint16,
    hours              Uint16,
    sixtyHours         Uint16,
    years              Uint16
}

```

```

-- -----
--
-- Location
--
-- -----

```

```

GeographicRegion ::= CHOICE {
    circularRegion      CircularRegion,
    rectangularRegion   SequenceOfRectangularRegion,
    polygonalRegion     PolygonalRegion,
    identifiedRegion    SequenceOfIdentifiedRegion,
    ...
}

```

```

CircularRegion ::= SEQUENCE {
    center          TwoDLocation,
    radius           Uint16
}

```

```

RectangularRegion ::= SEQUENCE {
    northWest        TwoDLocation,
    southEast         TwoDLocation
}

```

```

SequenceOfRectangularRegion ::= SEQUENCE OF RectangularRegion

```

```

PolygonalRegion ::= SEQUENCE SIZE(3..MAX) OF TwoDLocation

```

```
TwoDLocation ::= SEQUENCE {
    latitude      Latitude,
    longitude     Longitude
}
```

```
IdentifiedRegion ::= CHOICE {
    countryOnly      CountryOnly,
    countryAndRegions CountryAndRegions,
    countryAndSubregions CountryAndSubregions,
    ...
}
```

```
SequenceOfIdentifiedRegion ::= SEQUENCE OF IdentifiedRegion
```

```
CountryOnly ::= UInt16
```

```
CountryAndRegions ::= SEQUENCE {
    countryOnly      CountryOnly,
    regions          SequenceOfUInt8
}
```

```
CountryAndSubregions ::= SEQUENCE {
    country          CountryOnly,
    regionAndSubregions SequenceOfRegionAndSubregions
}
```

```
RegionAndSubregions ::= SEQUENCE {
    region          UInt8,
    subregions      SequenceOfUInt16
}
```

```
SequenceOfRegionAndSubregions ::= SEQUENCE OF RegionAndSubregions
```

```
ThreeDLocation ::= SEQUENCE {
    latitude      Latitude,
    longitude     Longitude,
    elevation     Elevation
}
```

```
Latitude ::= NinetyDegreeInt
```

```
Longitude ::= OneEightyDegreeInt
```

```
Elevation ::= ElevInt
```

```

NinetyDegreeInt ::= INTEGER {
    min                ..... (-900000000),
    max                ..... (900000000),
    unknown            (900000001)
} (-900000000..900000001)

KnownLatitude ::= NinetyDegreeInt (min..max) -- Minus 90deg to +90deg in microdegree
intervals
UnknownLatitude ::= NinetyDegreeInt (unknown)

OneEightyDegreeInt ::= INTEGER {
    min                (-1799999999),
    max                (1800000000),
    unknown            (1800000001)
} (-1799999999..1800000001)

KnownLongitude ::= OneEightyDegreeInt (min..max)
UnknownLongitude ::= OneEightyDegreeInt (unknown)

ElevInt ::= Uint16 -- Range is from -4096 to 61439 in units of one-tenth of a meter

-- -----
--
-- Crypto
--
-- -----

Signature ::= CHOICE {
    ecdsaNistP256Signature      EcdsaP256Signature,
    ecdsaBrainpoolP256r1Signature EcdsaP256Signature,
    ...,
    ecdsaBrainpoolP384r1Signature EcdsaP384Signature,
    ecdsaNistP384Signature      EcdsaP384Signature,
    sm2Signature                EcsigP256Signature
}

EcdsaP256Signature ::= SEQUENCE {
    rSig    EccP256CurvePoint,
    sSig    OCTET STRING (SIZE (32))
}

```

```

EcsigP256Signature ::= SEQUENCE {
    rSig OCTET STRING (SIZE (32)),
    sSig OCTET STRING (SIZE (32))
}

```

```

EcdsaP384Signature ::= SEQUENCE {
    rSig      EccP384CurvePoint,
    sSig      OCTET STRING (SIZE (48))
}

```

```

EccP256CurvePoint ::= CHOICE {
    x-only          OCTET STRING (SIZE (32)),
    fill            NULL, -- consistency with 1363 / X9.62
    compressed-y-0  OCTET STRING (SIZE (32)),
    compressed-y-1  OCTET STRING (SIZE (32)),
    uncompressedP256 SEQUENCE {
        x OCTET STRING (SIZE (32)),
        y OCTET STRING (SIZE (32))
    }
}

```

```

EccP384CurvePoint ::= CHOICE {
    x-only          OCTET STRING (SIZE (48)),
    fill            NULL, -- consistency w 1363 / X9.62
    compressed-y-0  OCTET STRING (SIZE (48)),
    compressed-y-1  OCTET STRING (SIZE (48)),
    uncompressedP384 SEQUENCE {
        x OCTET STRING (SIZE (48)),
        y OCTET STRING (SIZE (48))
    }
}

```

```

SymmAlgorithm ::= ENUMERATED {
    aes128Ccm,
    ...,
    sm4Ccm
}

```

```

SignedData ::= SEQUENCE {
    hashId      错误!书签自引用无效。

```

```

        tbsData      ToBeSignedData,
        signer       SignerIdentifier,
        signature     Signature
    }

```

该数据结构中包含：

——hashId 指示签名和验签时、对消息进行散列处理所使用的哈希算法；

——tbsData 包含散列后作为签名输入的数据；

——signer 确定对数据进行签名时所使用的密钥材料和哈希算法；

——signature 为数字签名

- 如果 signer 指示选择 self，则签名计算参数化如下：
 - ◆ Data（数据）输入等于 tbsData 字段的 COER 规范化编码；
 - ◆ Verification type（验签类型）等于 self；
 - ◆ Signer identifier（签名者标识符）输入等于空字符串。
- 如果 signer 指示 certificate 或 digest，则签名计算参数化如下：
 - ◆ Data（数据）输入等于 tbsData 字段的 COER 规范化编码；
 - ◆ Verification type（验签类型）输入 等于 certificate；
 - ◆ Signer identifier（签名者标识符）输入等于用于验证 SPDU 的证书的 COER 编码。

```

HashAlgorithm ::= ENUMERATED {
    sha256,
    ...,
    sha384,
    sm3
}

```

```

EciesP256EncryptedKey ::= SEQUENCE {
    v      EccP256CurvePoint,
    c      OCTET STRING (SIZE (16)),
    t      OCTET STRING (SIZE (16))
}

```

```

EcencP256EncryptedKey ::= SEQUENCE {
    v      EccP256CurvePoint,
    t      OCTET STRING (SIZE (32)),
    c      OCTET STRING (SIZE (16))
}

```

```

EncryptionKey ::= CHOICE {
    public      PublicEncryptionKey,
    symmetric   SymmetricEncryptionKey
}

```

```

PublicEncryptionKey ::= SEQUENCE {

```

```

        supportedSymmAlg      SymmAlgorithm,
        publicKey             BasePublicEncryptionKey
    }

```

```

BasePublicEncryptionKey ::= CHOICE {
    eciesNistP256             EccP256CurvePoint,
    eciesBrainpoolP256r1     EccP256CurvePoint,
    ...,
    ecencSm2                  EccP256CurvePoint
}

```

```

PublicVerificationKey ::= CHOICE {
    ecdsaNistP256             EccP256CurvePoint,
    ecdsaBrainpoolP256r1     EccP256CurvePoint,
    ...,
    ecdsaBrainpoolP384r1     EccP384CurvePoint,
    ecdsaNistP384            EccP384CurvePoint,
    ecsigSm2                  EccP256CurvePoint
}

```

```

SymmetricEncryptionKey ::= CHOICE {
    aes128Ccm      OCTET STRING(SIZE(16)),
    ...,
    sm4Ccm         OCTET STRING(SIZE(16))
}

```

```

-- -----
--
-- AID / ITS-AID
--
-- -----

```

```

AidSsp ::= SEQUENCE {
    aid          Aid,
    ssp          ServiceSpecificPermissions OPTIONAL
}

```

```

SequenceOfAidSsp ::= SEQUENCE OF AidSsp

```

```

Aid ::= INTEGER (0..MAX)

```

SequenceOfAid ::= SEQUENCE OF Aid

```
ServiceSpecificPermissions ::= CHOICE {
    opaque          OCTET STRING (SIZE(0..MAX)),
    ...,
    bitmapSsp      BitmapSsp
}
```

BitmapSsp ::= OCTET STRING (SIZE(0..31))

SspValue ::= OCTET STRING (SIZE(0..31))

SspBitmask ::= OCTET STRING (SIZE(0..31))

```
AidSspRange ::= SEQUENCE {
    aid          Aid,
    sspRange     SspRange OPTIONAL
}
```

SequenceOfAidSspRange ::= SEQUENCE OF AidSspRange

```
SspRange ::= CHOICE {
    opaque      SequenceOfOctetString,
    all         NULL,
    ...,
    bitmapSspRange  BitmapSspRange
}
```

```
BitmapSspRange ::= SEQUENCE {
    sspValue      OCTET STRING (SIZE(1..32)),
    sspBitmask    OCTET STRING (SIZE(1..32))
}
```

SequenceOfOctetString ::= SEQUENCE (SIZE (0..MAX)) OF
OCTET STRING (SIZE(0..MAX))

```
-- -----
--
-- Goes in certs
--
-- -----
```

SubjectAssurance ::= OCTET STRING (SIZE(1))

CrlSeries ::= Uint16

```
-- -----
--
-- Pseudonym Linkage
--
-- -----
```

IValue ::= Uint16

Hostname ::= UTF8String (SIZE(0..255))

LinkageValue ::= OCTET STRING (SIZE(9))

```
GroupLinkageValue ::= SEQUENCE {
    jValue  OCTET STRING (SIZE(4)),
    value   OCTET STRING (SIZE(9))
}
```

GroupSingleSeedCrlEntry

```
GroupSingleSeedCrlEntry ::= SEQUENCE {
    iMax          Uint16,
    laId          错误!书签自引用无效。 ,
    linkageSeed   LinkageSeed
}
```

SequenceOfGroupSingleSeedCrlEntry ::= SEQUENCE OF GroupSingleSeedCrlEntry

此数据结构包含如下信息：

- iMax 指示对于这些证书、一旦 iCert > iMax，就不再需要计算撤销信息，这是因为能知道持有者没有更多对应于（crlCraca，crlSeries）的有效证书；
- laId 作为 LinkageAuthorityIdentifier 参数用于基于单种子链接值的撤销算法；
- linkedSeed 作为 LinkageSeed 参数用于基于单种子链接值的撤销算法；

LaId ::= OCTET STRING (SIZE(2))

LinkageSeed ::= OCTET STRING (SIZE(16))

END

C.3 证书撤销列表（CRL）

C.3.1 证书撤销列表：基本类型

DEFINITIONS AUTOMATIC TAGS ::= BEGIN

EXPORTS ALL;

IMPORTS

 CrlSeries,
 GeographicRegion,
 HashedId8,
 HashedId10,
 IValue,
 GroupSingleSeedCrlEntry

GroupSingleSeedCrlEntry ::= SEQUENCE {
 iMax Uint16,
 laId **错误!书签自引用无效。,**
 linkageSeed LinkageSeed
}

SequenceOfGroupSingleSeedCrlEntry ::= SEQUENCE OF GroupSingleSeedCrlEntry

此数据结构包含如下信息：

- iMax 指示对于这些证书、一旦 iCert > iMax，就不再需要计算撤销信息，这是因为能知道持有者没有更多对应于（crlCraca，crlSeries）的有效证书；
- laId 作为 LinkageAuthorityIdentifier 参数用于基于单种子链接值的撤销算法；
- linkedSeed 作为 LinkageSeed 参数用于基于单种子链接值的撤销算法；

LaId,
 LinkageSeed,
 Opaque,
 Aid,
 Signature,
 Time32,
 Uint3,
 Uint8,
 Uint16,
 Uint32,
 ValidityPeriod
;

--
--
-- CRL contents
--
--

```

CrlContents ::= SEQUENCE {
    version          Uint8 (1),
    crlSeries        CrlSeries,
    crlCraca         HashedId8,
    issueDate        Time32,
    nextCrl          Time32,
    priorityInfo     CrlPriorityInfo,
    typeSpecific     CHOICE {
        fullHashCrl      ToBeSignedHashIdCrl,
        deltaHashCrl     ToBeSignedHashIdCrl,
        fullLinkedCrl    ToBeSignedLinkageValueCrl,
        deltaLinkedCrl   ToBeSignedLinkageValueCrl,
        ...
    }
}

```

```

CrlPriorityInfo ::= SEQUENCE {
    priority          Uint8 OPTIONAL,
    ...
}

```

```

ToBeSignedHashIdCrl ::= SEQUENCE {
    crlSerial         Uint32,
    entries           SequenceOfHashBasedRevocationInfo,
    ...
}

```

```

HashBasedRevocationInfo ::= SEQUENCE {
    id               HashedId10,
    expiry           Time32,
    ...
}

```

```

SequenceOfHashBasedRevocationInfo ::=
    SEQUENCE OF HashBasedRevocationInfo

```

```

ToBeSignedLinkageValueCrl ::= SEQUENCE {
    iRev             IValue,
    indexWithinI     Uint8,
    individual       SequenceOfJMaxGroup OPTIONAL,
}

```

```

        groups                SequenceOfGroupCrlEntry OPTIONAL,
        ...,
        groupsSingleSeed      SequenceOfGroupSingleSeedCrlEntry OPTIONAL,
    }
(WITH COMPONENTS {..., individual PRESENT} |
 WITH COMPONENTS {..., groups PRESENT})

```

```

JMaxGroup ::= SEQUENCE {
    jmax                Uint8,
    contents            SequenceOfLAGroup,
    ...
}

```

```
SequenceOfJMaxGroup ::= SEQUENCE OF JMaxGroup
```

```

LAGroup ::= SEQUENCE {
    la1Id              GroupSingleSeedCrlEntry
    GroupSingleSeedCrlEntry ::= SEQUENCE {
        iMax            Uint16,
        laId            错误!书签自引用无效。 ,
        linkageSeed     LinkageSeed
    }
}

```

```
SequenceOfGroupSingleSeedCrlEntry ::= SEQUENCE OF GroupSingleSeedCrlEntry
```

此数据结构包含如下信息：

- iMax 指示对于这些证书、一旦 iCert > iMax，就不再需要计算撤销信息，这是因为能知道持有者没有更多对应于 (crlCraca, crlSeries) 的有效证书；
- laId 作为 LinkageAuthorityIdentifier 参数用于基于单种子链接值的撤销算法；
- linkedSeed 作为 LinkageSeed 参数用于基于单种子链接值的撤销算法；

```

LaId,
la2Id              GroupSingleSeedCrlEntry
GroupSingleSeedCrlEntry ::= SEQUENCE {
    iMax            Uint16,
    laId            错误!书签自引用无效。 ,
    linkageSeed     LinkageSeed
}

```

```
SequenceOfGroupSingleSeedCrlEntry ::= SEQUENCE OF GroupSingleSeedCrlEntry
```

此数据结构包含如下信息：

- iMax 指示对于这些证书、一旦 iCert > iMax，就不再需要计算撤销信息，这是因为能知道持

有者没有更多对应于 (crlCraca, crlSeries) 的有效证书;
 ——laId 作为 LinkageAuthorityIdentifier 参数用于基于单种子链接值的撤销算法;
 ——linkedSeed 作为 LinkageSeed 参数用于基于单种子链接值的撤销算法;

```
LaId,
    contents      SequenceOfIMaxGroup,
    ...
}
```

SequenceOfLAGroup ::= SEQUENCE OF LAGroup

```
IMaxGroup ::= SEQUENCE {
    iMax      Uint16,
    contents  SequenceOfIndividualRevocation,
    ...,
    singleSeeds  SequenceOfLinkageSeed
}
```

SequenceOfIMaxGroup ::= SEQUENCE OF IMaxGroup

```
IndividualRevocation ::= SEQUENCE {
    linkageSeed1  LinkageSeed,
    linkageSeed2  LinkageSeed,
    ...
}
```

SequenceOfIndividualRevocation ::= SEQUENCE OF IndividualRevocation

LinkageSeed ::= OCTET STRING (SIZE(16))

SequenceOfLinkageSeed ::= SEQUENCE OF LinkageSeed

```
GroupCrlEntry ::= SEQUENCE {
    iMax      Uint16,
    laId      GroupSingleSeedCrlEntry
GroupSingleSeedCrlEntry ::= SEQUENCE {
    iMax      Uint16,
    laId      错误!书签自引用无效。 ,
    linkageSeed  LinkageSeed
}
```

SequenceOfGroupSingleSeedCrlEntry ::= SEQUENCE OF GroupSingleSeedCrlEntry

此数据结构包含如下信息:

——iMax 指示对于这些证书、一旦 iCert > iMax, 就不再需要计算撤销信息, 这是因为能知道持

有者没有更多对应于 (crlCraca, crlSeries) 的有效证书;

——laId 作为 LinkageAuthorityIdentifier 参数用于基于单种子链接值的撤销算法;

——linkedSeed 作为 LinkageSeed 参数用于基于单种子链接值的撤销算法;

```
LaId,
    linkageSeed1      LinkageSeed,
    la2Id             GroupSingleSeedCrlEntry
GroupSingleSeedCrlEntry ::= SEQUENCE {
    iMax              Uint16,
    laId              错误!书签自引用无效。 ,
    linkageSeed       LinkageSeed
}
```

```
SequenceOfGroupSingleSeedCrlEntry ::= SEQUENCE OF GroupSingleSeedCrlEntry
```

此数据结构包含如下信息:

——iMax 指示对于这些证书、一旦 iCert > iMax, 就不再需要计算撤销信息, 这是因为能知道持有者没有更多对应于 (crlCraca, crlSeries) 的有效证书;

——laId 作为 LinkageAuthorityIdentifier 参数用于基于单种子链接值的撤销算法;

——linkedSeed 作为 LinkageSeed 参数用于基于单种子链接值的撤销算法;

```
LaId,
    linkageSeed2      LinkageSeed,
    ...
}
```

```
SequenceOfGroupCrlEntry ::= SEQUENCE OF GroupCrlEntry
```

```
GroupSingleSeedCrlEntry ::= SEQUENCE {
    iMax      Uint16,
    laId      LaId,
    linkageSeed LinkageSeed,
}
```

```
SequenceOfGroupSingleSeedCrlEntry ::= SEQUENCE OF GroupSingleSeedCrlEntry
```

END

C.3.2 CRL: 安全封装

```
DEFINITIONS AUTOMATIC TAGS ::= BEGIN
```

```
EXPORTS ALL;
```

IMPORTS

V2XSecData
 Opaque,
 Aid
 CrlContents
 ;

CrlAid ::= Aid(3628)

SecuredCrl ::= V2XSecData (WITH COMPONENTS {...,
 content (WITH COMPONENTS {
 signedData (WITH COMPONENTS {...,
 tbsData (WITH COMPONENTS {
 payload (WITH COMPONENTS {...,
 data (WITH COMPONENTS {...,
 content (WITH COMPONENTS {
 unsecuredData (CONTAINING CrlContents)
 })
 })
 })),
 headerInfo (WITH COMPONENTS {...,
 aid (CrlAid),
 generationTime ABSENT,
 expiryTime ABSENT,
 generationLocation ABSENT,
 p2pcdLearningRequest ABSENT,
 missingCrlIdentifier ABSENT,
 encryptionKey ABSENT
 })
 })
 })
 })
))

END

C.3.3 CRL：业务特定权限（SSP）

DEFINITIONS AUTOMATIC TAGS ::= BEGIN

EXPORTS ALL;

```

IMPORTS
    CrlSeries,
    Uint8
;

CrlSsp ::= SEQUENCE {
    version          Uint8(1),
    associatedCraca  CracaType,
    crls             错误!未找到引用源。 ,
    ...
}

CracaType ::= ENUMERATED {isCraca, issuerIsCraca}

错误!未找到引用源。 ::= SEQUENCE OF CrlSeries

END

```

附 录 D

（规范性附录）

密码算法的输入与输出

D.1 哈希算法

本标准默认使用GB/T32905所规定的中国商用密码算法SM3哈希算法，本附录的数据结构中还定义了其他哈希算法（例如SHA-256）的标识符，这是为了保证本标准有更好的兼容性和可扩展性。

D.2 签名算法

D.2.1 概述

本标准默认使用GB/T32918.2所规定的中国商用密码算法SM2数字签名算法，本附录的数据结构中还定义了其他数字签名算法的标识符，这是为了保证本标准有更好的兼容性和可扩展性。

签名算法编码示例参见附录K.2。

SM2是一种椭圆曲线密码学算法，SM2只与单个特定的256位椭圆曲线相关联，因此在本标准中与SM2相关联的标识符不需要额外的曲线指示。

本标准中SM2的签名格式 r 用整数表示。SM2通过将身份字符串添加到要散列的消息前来命名散列函数，这个身份字符串是：

$$ZA = \text{HASH}(ENTLA || ID_A || a || b || xG || yG || xA || yA)$$

其中

- a) $ENTLA$ 是由 ID_A 长度转换而成的两个字节字符，值为 ID_A 的有效比特数；
- b) ID_A 是发送方的 ID；
- c) a 和 b 是 SM2 椭圆曲线的方程参数；
- d) xG 和 yG 是 SM2 基点的 x 和 y 坐标；
- e) xA 和 yA 是验签公钥的 x 和 y 坐标；

数字签名用 $H(ZA || \text{Data input})$ 计算，而不是只用 M 计算，此处的 Data input 为待签名的消息数据。

本标准约定 ID_A 为发送方正在使用证书的32字节哈希值，也即 $ID_A = \text{HashedId32}(\text{Certificate})$ 并按 GB/T 32918.1 进行比特串到字节串的转变， $ENTLA$ 是 $0x0100$ 。当签发的证书为自签名证书时， ID_A 为16进制整数31323334353637383132333435363738，并按 GB/T 32918.1 进行整数到字节串的转变，因此 $ENTLA$ 是 $0x0080$ 。

D.2.2 签名和验签操作中的预处理

对数据进行哈希处理以进行签名或验签时，应使用本节定义的规则创建哈希运算。该规则假设签名或验签过程使用身份字符串 ZA 和消息数据作为输入、并在内部创建级联和计算散列值。SM2中使用的哈希值如下所述。

——哈希算法应该是 SM3：

——签名或验签过程中输入的消息值应为：

$$\text{Hash}(\text{ZA} || \text{Data input})$$

其中

- 1) Data input 为待签名的数据；
 - 2) ZA 为按照 D.2.1 中方法计算结果
- 对PDU进行签名时需进行哈希处理，哈希处理时数据输入的编码规则见6.4.3。

D.3 公钥加密算法

D.3.1 概述

本标准默认使用GB/T32918.4所规定的中国商用密码算法SM2非对称公钥加密算法，本章的数据结构中还定义其他非对称公钥加密算法（例如ECIES）的标识符，这是为了保证本标准有更好的兼容性和可扩展性。

D.3.2 SM2加密

如**错误!未找到引用源。**所述，本标准支持使用SM2公钥加密算法来加密对称加密密钥，但不支持直接使用SM2算法去加密数据。

使用SM2公钥加密算法时，应为每个加密操作新生成短暂的公钥，即加密操作不应重用短暂的公钥。SM2加密的输出是一个三元组（V，tag，C），其中：

- V 表示发送方的短暂公钥；
- tag 是认证标签，长度是 256 比特；
- C 是加密后的对称密钥，长度为 128 比特。

D.4 对称加密算法

本标准默认使用GB/T32907所规定的中国商用密码算法SM4分组加密算法作为对称加密算法，本章的数据结构中还定义其他对称加密算法（例如AES）的标识符，这是为了保证本标准有更好的兼容性和可扩展性。

本标准默认的对称加解密操作模式是GB/T 36624中规定的CCM模式。

D.5 通过哈希值对编码后的数据结构进行标识

HashedId3、HashedId8和HashedId10结构（此后，HashedIdX用于指代其中任何一个，具体取决于上下文）在整个标准中被用于标识密钥和证书。特别是：

- 在 SignerIdentifier 中使用它来标识对 SPDU 进行签名的证书；
- 在 RecipientInfo 中使用它来标识加密密钥：
 - PreSharedKeyRecipientInfo：用于加密数据的对称预共享密钥的哈希值；
 - SymmRecipientInfo：用于加密对称加密密钥的对称预共享密钥的哈希值；
 - PKRecipientInfo：用于将接收方的公钥传递给加密器的“公钥容器”（证书、签名后的 SPDU 或“原始”公钥）的哈希值；

——在 IssuerIdentifier 中使用它来标识证书的签发者；

HashedIdX结构不包含哈希算法标识符。在本标准中，哈希算法SM3将用于计算所有HashedIdX。

D.6 加密数据

D.6.1 概述

SSF根据SDPF的请求生成加密数据。在请求中，SDPF可以向SSF提供接收方密钥列表。SDPF还可以请求SSF包括数据加密密钥的直接指示。这使得SDPF能够为多个接收方加密数据，而每个接收方都可以解密该数据。

加密过程如下：

首先，使用对称数据加密密钥k对数据进行加密，以产生6.4.41中规定的SymmetricCiphertext。SSF可以通过多种方式获取数据加密密钥k，包括：

- 预先存储的密钥；
- SDPF 通过在前面发送的 SPDU 中包含数据加密密钥，来向接收方提供密钥，并存储在 SSF 中；
- 接收方通过在前面发送的 SPDU 中包含数据加密密钥，来向 SDPF 提供密钥，并存储在 SSF 中；
- 由另一个 SDPF 通过在前面发送的 SPDU 中包含数据加密密钥，来向 SDPF 提供密钥，并存储在 SSF 中；
- 使用早期 SPDU 为另一个接收方生成的密钥；
- 一个“短暂”密钥，它是为该 SPDU 适当生成的，只使用过一次。

如果密钥不是“短暂的”，即在数据加密时在SSF内生成，则称其为“静态的”。

在数据加密之后，SSF创建RecipientInfos，允许目标接收方访问数据加密密钥。每个不同的密钥加密密钥通常将对应于被加密PDU的不同目标接收方。

接收方将获得有关数据加密密钥的信息，如下所示：

- 如果调用 SDPF 提供了一个或多个接收方密钥，则 SSF 使用这些密钥中的每一个（记为 i）来对密钥 k 进行加密、产生一系列被加密的密钥{eki}。对于每一个密钥加密密钥 i，SSF 创建一个 RecipientInfo 结构，其中包含加密密钥 eki，其类型由接收方加密密钥的类型确定。这使得拥有相应解密密钥的接收方能够恢复数据加密密钥；
- 如果调用 SDPF 请求直接指示数据加密密钥，则 SSF 将创建一个类型为 PreSharedKeyRecipientInfo 的 RecipientInfo 结构，用于标识数据加密密钥。这使得已经拥有数据加密密钥的接收方能够识别出可用于解密被加密数据的密钥；
- 然后，SSF 将所有 RecipientInfos 编码为 SequenceOfRecipientInfo，将 SequenceOfRecipientInfo 和 SymmetricCiphertext 编码为 EncryptedData，并将 EncryptedData 封装在 V2XSecData 中。

此方法能够为已经知数据加密密钥的接收方、尚未知道数据加密密钥的接收方或这两种类型接收方的组合创建EncryptedData。

在Sec-EncryptedData.request原语中，SDPF通过提供或不提供数据加密密钥类型参数向SSF指示是否使用短暂或静态数据加密密钥。如果SDPF请求短暂数据加密密钥，则SSF可以在相应的Sec-EncryptedData.confirm原语中返回该数据加密密钥的加密材料句柄，以便在需要时重用该数据加密密钥。

在Sec-EncryptedDataDecryption.confirm原语中，SSF可以为Data Encryption Key（数据加密密钥）CMH参数中的数据加密密钥提供加密材料句柄。这使得接收该原语的SDPF能够存储CMH以供后使用。加密算法及编码示例参见附录K.3。

D. 6.2 数据加密

数据加密按如下规则执行：

- 明文 P 具有有效编码的 V2XSecData 结构形式；
- 如 D. 3. 1 中所述，与 D. 5 中规定的对称加密算法相对应的对称数据加密密钥 k 可用；
- 用 k 加密 P 以获得密文 C。如果对称加密算法使用 nonce 随机数，那么对于每个新的明文，都应随机地和重新生成一个 nonce 随机数；
- 密文 C 被编码为 SymmetricCiphertext。

D. 6.3 对数据加密密钥的加密

可以用一个或多个公钥或对称密钥去加密数据加密密钥，并将它们放在单个加密的SPDU，接收方只要知道其中任何一个对应的解密密钥，就可以解密和得到数据加密密钥。本标准将每一个这样的加密密钥称作为接收方密钥，将拥有相应的解密密钥的所有者称作为接收方。

按如下方式对数据加密密钥进行加密：

- 对于每个接收方密钥，使用接收方密钥对数据加密密钥 k 进行加密，以获得被加密的数据加密密钥 ek。
 - 如果接收方密钥是非对称公钥，则加密处理流程取决于公钥的来源，详见 D. 4；
 - 如果接收方密钥是对称密钥，则按 D. 5 中所述用指定的对称密钥去加密数据加密密钥，并且 RecipientInfo 的类型为 SymmRecipientInfo。

附 录 E

(规范性附录)

V2X 设备与安全证书管理系统接口的数据格式

E.1 概述

本附录规定V2X设备与LTE-V2X安全证书管理系统（Security Credential Management System, SCMS）进行交互时采用的协议数据单元（protocol data unit, PDU），使用抽象语法记法一（Abstract Syntax Notation 1, ASN.1）对具体数据结构进行描述，采用正则八位字节编码规则（Canonical Octet Encoding Rules, COER）进行编码。附录中每小节详细描述了一类数据结构：

E.2列出了附录中引用自本标准其他章节的数据结构；

E.3定义了SCMS中PDU的数据结构，包括了SCMS中不同模块之间的接口（Interface）、不同模块与V2X设备之间的接口，这些接口交互的所有应用级数据的结构；

E.4定义了SCMS所有安全协议数据单元SPDU，它们基于6.4 SPDU数据结构进行约束定义；

E.5定义了参数化的ASN.1数据类型，这些数据类型用于构造E.4中的SPDU；

E.6定义了授权证书CA和注册机构RA之间的接口；

E.7定义了链接机构LA和异常行为管理机构MA之间的接口；

E.8定义了链接机构LA和注册机构RA之间的接口；

E.10定义了安全证书管理使用的AID十进制3627的相关SSP。

注：附录E的ASN.1数据结构中，需要特别说明的3个代称包括：ee表示V2X设备（End-Entity），aca表示授权证书机构（Authorization CA，包含了本标准中的假名证书机构PCA和应用证书机构ACA），ra表示注册机构（Registration Authority, RA，包含了假名注册机构PRA和应用注册机构ARA）。

E.2 引用的数据结构

附录E中引用了本标准其他章节出现数据结构，数据结构名称如下：

BasePublicEncryptionKey

BitmapSsp

BitmapSspRange

Certificate

CertificateId

CertificateType

CrlSeries

Duration

EccP256CurvePoint

EccP384CurvePoint

EcdsaP256Signature

EcdsaP384Signature

EncryptedData

GeographicRegion

HashAlgorithm
 HashedId3
 HashedId8
 HashedId10
 Hostname
 V2XSecData
 IValue
 LaId
 PKRecipientInfo
 Aid
 AidGroupPermissions
 AidSspRange
 PublicEncryptionKey
 SecuredCrl
 SequenceOfCertificate
 SequenceOfAidGroupPermissions
 SequenceOfAidSsp
 SignedData
 SignerIdentifier
 SspRange
 SubjectAssurance
 Time32
 Uint8
 ValidityPeriod
 VerificationKeyIndicator

E.3 SCMS中的PDU

E.3.1 概述

本小节对SCMS PDU的数据结构进行定义，各数据结构的序号是基于其在先前数据结构中出现的顺序。AcaRaInterfacePdu、LaMaInterfacePdu、LaRaInterfacePdu三个接口有关的pdu和spdu内容详见附录E.6、E.7、E.8，因为这3个接口属于scms内部接口，与V2X设备无关，因此单独成节。

E.3.2 ScmsPdu

```

ScmsPdu ::= SEQUENCE {
    version  Uint8 (1),
    content  CHOICE {
        aca-ee  AcaEeInterfacePdu,
        aca-la  AcaLaInterfacePdu,
        aca-ma  AcaMaInterfacePdu,
        aca-ra  AcaRaInterfacePdu,
        cert    CertManagementPdu,
    }
  
```

```

    eca-ee  EcaEeInterfacePdu,
    ee-ma   EeMaInterfacePdu,
    ee-ra   EeRaInterfacePdu,
    la-ma   LaMaInterfacePdu,
    la-ra   LaRaInterfacePdu,
    ma-ra   MaRaInterfacePdu,
    ...
  }
}

```

ScmsPdu作为V2X设备与SCMS进行数据交互的主数据结构，具体包括：

- 1) version: ScmsPdu 数据结构的当前版本号，本标准中版本号为 1；
- 2) aca-ee 定义了授权证书 CA 与 V2X 设备之间进行数据交互的接口数据结构，此接口为逻辑接口；
- 3) aca-la 定义了授权证书 CA 与 LA 之间进行数据交互的接口数据结构，本标准暂未定义此接口，因此定义为 NULL；
- 4) aca-ma 定义了授权证书 CA 与 MA 之间进行数据交互的接口数据结构，本标准暂未定义此接口，因此定义为 NULL；
- 5) aca-ra 定义了授权证书 CA 与注册机构 RA 之间进行数据交互的接口数据结构，即**错误！未找到引用源。**，详见附录 E. 6；
- 6) cert 定义了用于证书管理的接口数据结构；
- 7) eca-ee 定义了 ECA 与 V2X 设备之间进行数据交互的接口数据结构，此接口为逻辑接口；
- 8) ee-ma 定义了 V2X 设备与 MA 之间进行数据交互的接口数据结构，即**错误！未找到引用源。****错误！未找到引用源。**，本数据结构在本标准范围之外，因此将其定义为 NULL；
- 9) ee-ra 定义了 V2X 设备与注册机构 RA 之间进行数据交互的接口数据结构，即**错误！未找到引用源。****错误！未找到引用源。**；
- 10) la-ma 定义了 LA 与 MA 之间进行数据交互的接口数据结构，即**错误！未找到引用源。****错误！未找到引用源。**，详见附录 E. 7；
- 11) la-ra 定义了 LA 与注册机构 RA 之间进行数据交互的接口数据结构，即**错误！未找到引用源。****错误！未找到引用源。**，详见附录 E. 8；
- 12) ma-ra 定义了 MA 与注册机构 RA 之间进行数据交互的接口数据结构，本数据结构在本标准范围之外，因此将其定义为 NULL。

E. 3. 3 AcaEeInterfacePdu

```

AcaEeInterfacePdu ::= CHOICE {
    acaEeCertResponse  AcaEeCertResponse,
    ...
}

```

AcaEeInterfacePdu是授权证书CA与V2X设备之间进行交互的所有数据结构的主结构，授权证书CA与V2X设备之间只是一个逻辑接口，即授权证书CA与V2X设备之间没有直接交互，所有交互都是通过RA进行的。在此数据结构中：

acaEeCertResponse包括了授权证书CA对RaAcaCertRequestSpdu的响应，该响应通过RA发送给V2X设备使用。

E. 3. 4 AcaEeCertResponse

```
AcaEeCertResponse ::= SEQUENCE {
    version          Uint8 (1),
    generationTime   Time32,
    certificate       Certificate,
    privateKeyInfo   OCTET STRING (SIZE (32)) OPTIONAL,
    ...
}
```

该结构是对成功的授权证书申请请求的响应，该请求是通过RaAcaCertRequestSpdu发送的。在此结构中：

- a) version 包含了本结构的当前版本号，本标准中版本号为 1；
- b) generationTime 包含了 AcaEeCertResponse 的生成时间；
- c) certificate 包含了由授权证书 CA 生成的假名证书、应用证书或身份证书。证书的类型与对应的证书申请请求中的 type 字段一致，如果请求的类型不正确，则响应错误信息，而非此数据结构。
- d) privateKeyInfo
 - 1) 如果对应的证书签发请求 RaAcaCertRequest 中的标志位 flags.butterflyExplicit 被置位，则 privateKeyInfo 字段设置为 PRESENT 并包含完整私钥重构所需的 c。V2X 设备使用此私钥分量 c 重构 7.2.7 所述密钥扩展流程中显式证书所对应的完整私钥 si, j。
 - 2) 如果不是以上的情况，则 privateKeyInfo 应为 ABSENT。

注：如果使用密钥衍生功能在RaAcaCertRequest中填入了certEncKey，j值不会传递给授权证书CA。然而接收到AcaEeCertResponse的V2X设备只有知道j值时才能解密该响应。因此，RA应存储j值，以便将j值与对应的AcaEeCertResponse进行关联，并将j值编码进证书下载的文件名中。

E. 3. 5 AcaLaInterfacePdu

AcaLaInterfacePdu ::= NULL

本标准暂未定义此接口，因此定义为NULL。

E. 3. 6 AcaMaInterfacePdu

AcaMaInterfacePdu ::= NULL

本标准暂未定义此接口，因此定义为NULL。

E. 3. 7 AcaRaInterfacePdu

本数据结构将在E. 6 ACA-RA接口中具体描述。

E. 3. 8 CertManagementPdu

```
CertManagementPdu ::= CHOICE {
    compositeCrl          CompositeCrl,
    certificateChain       CertificateChain,
```

```
...
}
```

CertManagementPdu是SCMS所有模块的证书管理数据结构的主结构，它包括：

- a) CompositeCrl 包含了零个或更多个在附录 C.3.2 中定义的 SecuredCrl;
- b) certificateChain 包含了证书的集合;

E.3.9 CompositeCrl

```
CompositeCrl ::= SEQUENCE {
    crl          SEQUENCE SIZE(0..MAX) OF SecuredCrl,
    ...
}
```

CompositeCrl包含了一个复合CRL所需要的参数。在此结构中：

- a) Crl 是一个已被签名的 CRL 的列表，包含了不同的 cracaID 和 crlSeries 组合。这些 CRL 都各自被签名，并且不指定这些 CRL 的顺序。

E.3.10 CertificateChain

```
CertificateChain ::= SEQUENCE {
    others      SEQUENCE SIZE(0..MAX) OF Certificate,
    ...
}
```

此结构用于封装证书和CTL。在此结构中：

- a) others 包含了 SCMS 中有效的机构的证书。

E.3.11 EcaEeInterfacePdu

```
EcaEeInterfacePdu ::= CHOICE {
    eeEcaCertRequest  EeEcaCertRequest,
    ecaEeCertResponse EcaEeCertResponse,
    ...
}
```

EcaEeInterfacePdu数据结构是指ECA与V2X设备之间所有数据交互的主结构。在此结构中：

- a) eeEcaCertRequest 包含了 V2X 设备向 ECA 发送的注册证书申请请求。
- b) ecaEeCertResponse 包含了 ECA 返回给 V2X 设备的注册证书申请响应。

E.3.12 EeEcaCertRequest

```
EeEcaCertRequest ::= SEQUENCE {
    version          Uint8 (1),
    generationTime   Time32,
    type             CertificateType,
    tbsCert          ToBeSignedCertificate (WITH COMPONENTS {
        ...,
        id (WITH COMPONENTS {
            ...,
```

```

        linkageData ABSENT
    )),
    cracaId ('000000'H),
    crlSeries (0),
    appPermissions ABSENT,
    certIssuePermissions ABSENT,
    certRequestPermissions PRESENT,
    verifyKeyIndicator (WITH COMPONENTS {
        verificationKey
    })
)),
canonicalId IA5String OPTIONAL,
...
}

```

EeEcaCertRequest数据结构包含了向ECA申请EC时所需的参数。根据策略，ECA签发的EC中的内容可能与本申请中的内容不同。在本结构中：

- a) version 包含了 EeEcaCertRequest 的版本号，在本标准中版本号为 1。
 - b) generationTime 包含了 EeEcaCertRequest 的生成时间。
 - c) type 表明了请求的 V2X 设备是请求显式（explicit）证书还是其他证书。
 - d) tbsCert 包含了 ECA 用来生成 EC 的参数。
 - e) tbsCert.verifyKeyIndicator.verificationKey 包含了请求的 V2X 设备发送的公钥信息。
verifyKeyIndicator 字段选择 verificationKey，这使请求的 V2X 设备可以表明它正在申请哪种签名算法和曲线。如果响应此请求而发出的 EC 证书是显式，则此字段中的值用作 EC 证书中的验证公钥。
 - f) canonicalId 用于存放正则 ID，如何使用正则 ID 及其对应的正则公私钥不在本标准范围内。
- 注1：tbsCert.cracaId和tbsCert.crlSeries在V2X设备提交的EeEcaCertRequest中填充为'000000'H和0，但ECA在EeEcaCertResponse中将更改为相应的取值。
- 注2：V2X设备使用tbsCert.verifyKeyIndicator字段指示它是在请求显式注册证书还是其他注册证书。将运用策略来确定哪种证书类型适合相应情况（例如AID，其他V2X设备的信息，位置等），并且如果V2X设备申请的证书类型是SCMS策略所不允许的，则ECA将返回错误码给V2X设备。

E. 3. 13 EcaEeCertResponse

```

EcaEeCertResponse ::= SEQUENCE {
    version          Uint8 (1),
    requestHash      HashedId8,
    ecaCertChain     SequenceOfCertificate,
    certificate       Certificate,
    privateKeyInfo   OCTET STRING (SIZE(32)) OPTIONAL,
    ...
}

```

ECA使用此结构来响应V2X设备的EC申请请求。AAA提供其他初始化信息至V2X设备，比如RA的证书。AAA的交互数据规范不在本标准范围之内。在此结构中：

- a) version 包含结构的当前版本，在本标准中此处版本号为 1。
- b) requestHash 包含以下内容的哈希值：
 - 1) 如果 EcaEeCertResponse 响应 EeEcaCertRequestSpdu 请求，则 requestHash 是 EeEcaCertRequestSpdu 的哈希值。
 - 2) 如果 EcaEeCertResponse 响应 EeRaSuccessorEnrollmentCertRequestSpdu 请求，则 requestHash 是 EeRaSuccessorEnrollmentCertRequestSpdu 的哈希值。
- c) ecaCertChain 包含 ECA 当前的有效证书和证书链，证书链直至 Root CA（包括 Root CA）。
- d) certificate 包含了 ECA 生成的 EC，其类型应为相应请求中 type 字段指示的类型。
- e) privateKeyInfo 预留用于私钥重构值（如果 certificate.type 是 reserved），因本标准默认的 type 为 explicit，因此本字段默认为 ABSENT。

E. 3. 14 EeMaInterfacePdu

EeMaInterfacePdu ::= NULL

该数据结构在本标准范围之外，因此将其定义为 NULL。

E. 3. 15 EeRaInterfacePdu

```
EeRaInterfacePdu ::= CHOICE {
    eeRaCertRequest          EeRaCertRequest,
    raEeCertAck              RaEeCertAck,
    raEeCertInfo              RaEeCertInfo,
    eeRaDownloadRequest      EeRaDownloadRequest,
    eeRaSuccessorEnrollmentCertRequest EeEcaCertRequestSpdu,
    ...
}
```

EeRaInterfacePdu 是 V2X 设备和 RA 之间所有数据交互的主数据结构。在此结构中：

- a) eeRaCertRequest 包含了 V2X 设备发送给 RA 的授权证书申请请求。
- b) raEeCertAck 包含了 RA 接收到 EeRaCertRequestSpdu 后的确认信息。
- c) raEeCertInfo 包含了相应证书下载的必要信息。
- d) eeRaDownloadRequest 包含了 V2X 设备发送给 RA 的下载请求。
- e) eeRaSuccessorEnrollmentCertRequest 为 EeEcaCertRequestSpdu，是一个自签名的 EeEcaCertRequest，其格式与申请初始 EC 的格式相同，并使用 EeEcaCertRequest 中的公钥对应的私钥进行签名，再通过使用当前的旧注册证书签名，得到 eeRaSuccessorEnrollmentCertRequestSpdu，用于申请新的注册证书。

E. 3. 16 EeRaCertRequest

```
EeRaCertRequest ::= SEQUENCE {
    version          Uint8 (1),
    generationTime   Time32,
    type              CertificateType,
    tbsCert           ToBeSignedCertificate (WITH COMPONENTS {
        ...,
        cracaId ('000000'H),
```

```

    crlSeries (0),
    appPermissions PRESENT,
    certIssuePermissions ABSENT,
    certRequestPermissions ABSENT,
    verifyKeyIndicator (WITH COMPONENTS {
        verificationKey
    })
  }),
  additionalParams AdditionalParams OPTIONAL,
  ...,
  aaaAuthorizationInfoList SequenceOfAaaAuthorizationInfo OPTIONAL
}

```

此结构包含请求不同类型的授权证书所需的参数。在此结构中：

- a) version 包含结构的当前版本，在本标准中此处版本号为 1。
- b) generationTime 包含 EeRaCertRequest 的生成时间。
- c) type 指示申请的 V2X 设备是申请显式证书还是其他。
- d) tbsCert 包含授权证书 CA 用来生成授权证书的参数。
- e) tbsCert.verifyKeyIndicator.verificationKey 包含请求者发送的公钥信息。这使请求者可以指明他们正在请求哪种签名算法和曲线。
 - 1) 如果响应此请求而签发的证书是显式的，并且未使用 7.2.6 密钥衍生流程，则此字段中的值为该证书中的公钥。
 - 2) 如果使用了 7.2.6 密钥衍生流程，即如果在 additionalParams 字段中存在 original, unified, compactUnified 的选项之一，则本字段中的值将与 AdditionalParams 字段中的值组合使用。
- f) additionalParam 包含了相关参数，在使用了 7.2.6 密钥衍生流程时运用这些参数生成请求的假名证书，或者没使用密钥衍生机制时运用这些参数对证书进行加密。如果本字段存在，则字段 tbsCert.verifyKeyIndicator 将用于存放签名种子公钥（公钥因子），即 7.2.6 中的 A。
- g) aaaAuthorizationInfoList 包含了需要提供给 AAA 的与证书签发授权相关的参数。

注：在密钥衍生机制用于衍生加密密钥并加密证书的情况下，j 值不会传递给授权证书 CA。但是，接收到证书响应的 V2X 设备只有知道 j 值时才能解密该响应。因此，RA 将存储 j 值，以便将 j 值与相应的证书响应进行关联。

E.3.17 AdditionalParams

```

AdditionalParams ::= CHOICE {
    original      ButterflyParamsOriginal,
    unified       ButterflyExpansion,
    compactUnified ButterflyExpansion,
    encryptionKey PublicEncryptionKey,
    ...
}

```

AdditionalParams 结构包含了密钥衍生机制的参数。在此结构中：

- a) original 包含了 original 的密钥衍生机制中相关的参数，即 7.2.6 中的 P、 k_s 、 k_E

- b) unified 包含了 unified 的密钥衍生机制中相关的参数，即 7.2.6 中的 k_s
- c) compactUnified 包含了 compact unified 的密钥衍生机制中相关的参数，即 7.2.6 中的 k_s 。
- d) encryptionKey: 如果没有使用密钥衍生机制，则本字段包含了用于加密证书的公钥。

注1: 使用original的密钥衍生机制，加密对称密钥、加密种子密钥对，与签名对称密钥、签名种子密钥对是不相同的，即(p, P)与(a, A)不同， k_E 与 k_S 不同，

注2: unified或compact unified的密钥衍生机制，则加密对称密钥、加密种子密钥对，与签名对称密钥、签名种子密钥对是相同的，即(p, P)与(a, A)相同， k_E 与 k_S 相同。compact unified比unified更省略了7.2.6 步骤6中使用PCA的私钥对返回的PC进行签名的流程

E.3.18 ButterflyParamsOriginal

```
ButterflyParamsOriginal ::= SEQUENCE {
    signingExpansion      ButterflyExpansion,
    encryptionKey         PublicEncryptionKey,
    encryptionExpansion   ButterflyExpansion
}
```

此结构包含了密钥衍生机制中原始变体的参数。在此结构中：

- a) signingExpansion 包含签名对称密钥，即 7.2.6 中的 k_s 。
- b) encryptionKey 包含加密种子公钥（公钥因子），即 7.2.6 中的 P。
- c) encryptionExpansion 包含加密对称密钥，即 7.2.6 中的 k_E 。

E.3.19 ButterflyExpansion

```
ButterflyExpansion ::= CHOICE {
    aes128      OCTET STRING (SIZE(16)),
    ...,
    sm4         OCTET STRING (SIZE(16))
}
```

该结构包含用于密钥衍生函数中所使用的对称密钥。本标准支持的对称加密算法为SM4算法，本字段指定的16字节字符串用作密钥。

E.3.20 RaEeCertInfo

```
RaEeCertInfo ::= SEQUENCE {
    version          Uint8 (1),
    generationTime   Time32,
    currentI         IValue,
    requestHash      HashedId8,
    nextDLTime       Time32,
    ...
}
```

此结构用于创建与批量证书一起下载的信息文件。在使用了7.2.6 密钥衍生流程来生成证书时使用。

- a) version 包含结构的当前版本，在本标准中此处版本号为 1。

- b) generationTime 包含了 RaEeCertInfo 的生成时间。
- c) currentI 包含了与本批证书关联的 i 值。currentI 包含了与本批证书关联的 i 值，同时用于密钥衍生 lv 生成，本标准中以北京时间 2020-1-7, 4am 为起始时间计算 i 值（i 取值从 0 开始，即 2020-1-8 对应的 i 值为 0）。
- d) requestHash 包含对应的 EeRaCertRequestSpdu 或 EeRaCertRequestPlainSpdu 的哈希值（该值将作为假名证书下载文件的文件名或文件名的一部分，见 E. 4. 15）。
- e) nextDlTime: 在此时间之后，V2X 设备可以连接到 RA 以下载证书。

E. 3. 21 EeRaDownloadRequest

```
EeRaDownloadRequest ::= SEQUENCE {
    generationTime  Time32,
    filename         UTF8String (SIZE (0..255)),
    ...
}
```

EeRaDownloadRequest 结构包含 V2X 设备请求从 RA 下载证书所需的参数。在此结构中：

- a) generationTime 包含 EeRaDownloadRequest 的生成时间。
- b) filename 包含请求下载的文件名。针对基于密钥衍生机制的证书请求，V2X 设备需根据当前请求下载的时间计算出 i 值并与 requestHash 构成 filename；针对未基于密钥衍生机制的证书请求，V2X 设备仅需使用 requestHash 作为 filename。filename 的命名方式见附录 E. 4. 15 的相关描述。

E. 3. 22 LaMaInterfacePdu

本数据结构将在 E. 7 LA-MA 接口中具体描述。

E. 3. 23 LaRaInterfacePdu

本数据结构将在 E. 8 LA-RA 接口中具体描述。

E. 3. 24 MaRaInterfacePdu

MaRaInterfacePdu ::= NULL

本数据结构在本标准范围之外，因此将其定义为 NULL。

E. 3. 25 AaaAuthorizationInfo

```
AaaAuthorizationInfo ::= SEQUENCE {
    aaaAuthorizationInfo  Opaque,
    aaaUrl                 .....
                           UTF8String (SIZE (0..511)) OPTIONAL
}
```

SequenceOfAaaAuthorizationInfo ::= SEQUENCE OF AaaAuthorizationInfo

V2X设备使用AaaAuthorizationInfo向证书签发系统提供与证书签发授权相关的信息。这些授权信息由证书签发系统转发给AAA。AAA解析这些授权信息，确定是否能够向V2X设备签发证书，并将授权结果提供给证书签发系统。在此结构中：

- a) aaaAuthorizationInfo 包含有 AAA 对证书签发过程进行授权所需要的信息，该信息对证书签发系统是透明的。
- b) aaaUrl 为 AAA 的地址或标识信息。证书签发系统使用此信息确定应将授权信息发送给哪个 AAA。

E. 3. 26 RaEeCertAck

```
RaEeCertAck ::= SEQUENCE {
    version          Uint8 (1),
    generationTime   Time32,
    requestHash      HashedId8,
    nextDlTime       Time32,
    ...
}
```

此结构用于创建证书请求的确认。

- a) version 包含本结构的版本号，在本标准中版本号为 1。
- b) generationTime 包含 RaEeCertAck 的生成时间。
- c) requestHash 包含对应的 EeRaCertRequestSpdu 的哈希值。
- d) nextDlTime 在此时间之后，V2X 设备可以连接到 RA 下载证书。

E. 4 SCMS中的SPDU

E. 4. 1 概述

E. 4中的数据结构是对E. 3和E. 5中定义的PDU数据结构的V2XSecData封装。在E. 4中这些数据结构以字母顺序排序。

注：虽然某些SPDU在协议数据单元层面上没有明确进行保护，但其在传输过程中使用其他机制进行保护。

E. 4. 2 AcaEeCertResponseCubkSpdu

```
AcaEeCertResponseCubkSpdu ::= V2XSecData-Encrypted {
    ScmsPdu-Scoped {
        AcaEeInterfacePdu (WITH COMPONENTS {
            acaEeCertResponse
        })
    }
}
```

此SPDU结构用于将仅加密的AcaEeCertResponse返回给RA，采用了适于返回至原申请的V2X设备的形式。

授权证书CA将本结构用于compact-unified的密钥衍生机制。如果使用RaAcaCertRequest结构在RA与ACA之间进行交互，则RA通过在相应的RaAcaCertRequest中flags字段中置位usesCubk（1）位来表明使用compact-unified的密钥衍生机制。

AcaEeCertResponse由授权证书CA使用衍生的加密公钥进行加密。

E. 4. 3 AcaEeCertResponsePlainSpdu

```
AcaEeCertResponsePlainSpdu ::= V2XSecData-Unsecured {
    ScmsPdu-Scoped {
        AcaEeInterfacePdu (WITH COMPONENTS {
            acaEeCertResponse
        })
    }
}
```

该SPDU结构用于将不安全的AcaEeCertResponse返回至RA，采用了适于返回至原申请的V2X设备的形式。该结构适用于不需要向RA保护V2X设备的隐私的情况下，例如，当返回的证书并不是假名证书时。

E. 4. 4 AcaEeCertResponsePrivateSpdu

```
AcaEeCertResponsePrivateSpdu ::= V2XSecData-EncryptedSigned {
    ScmsPdu-Scoped {
        AcaEeInterfacePdu (WITH COMPONENTS {
            acaEeCertResponse
        })
    },
    SecurityMgmtAid
}
```

AcaEeCertResponsePrivateSpdu此SPDU结构用于将AcaEeCertResponse返回至RA，采用了适于返回至原申请的V2X设备的形式。该结构由被签名的SPDU组成，该被签名的SPDU又包含了一个被加密过的SPDU。

被加密的SPDU使用了提供给授权证书CA的加密密钥用于响应。确定此密钥的方法如下：

- 如果来自 V2X 设备的原始 EeRaCertRequest 表示了单一的加密密钥用于响应，即如果请求中存在 additionalParams.encryptionKey 字段，则使用该密钥对响应进行加密。
- 如果来自 V2X 设备的原始 EeRaCertRequest 中表示，对响应加密密钥使用 originalunified 密钥衍生机制，即在请求中提供了 AdditionalParams.original 字段，则将使用从 AdditionalParams 中的参数衍生的加密扩展密钥 $Q_{i,j}$ （见 7.2.6）对响应进行加密。
- 如果来自 V2X 设备的原始 EeRaCertRequest 中表示，对响应加密密钥使用 unified 密钥衍生机制，即在请求中提供了 AdditionalParams.unified 字段，则将使用从 AdditionalParams 中的参数衍生的签名扩展密钥 $B_{i,j}$ （见 7.2.6）对响应进行加密。

本结构中签名生成V2XSecData的encryptedData的证书，应当与签发AcaEeCertResponse中的certificate证书的授权证书CA的证书相同。如果不同，则无效。

E. 4. 5 CertificateChainSpdu

```
CertificateChainSpdu ::= V2XSecData-Unsecured {
```

```

ScmsPdu-Scoped {
    CertManagementPdu (WITH COMPONENTS {
        certificateChain
    })
}

```

此结构是用于发送不安全的CertificateChain的SPDU。

E. 4. 6 CompositeCrlSpdu

```

CompositeCrlSpdu ::= V2XSecData-Unsecured {
    ScmsPdu-Scoped {
        CertManagementPdu (WITH COMPONENTS {
            compositeCrl
        })
    }
}

```

此结构是用于发送不安全的CompositeCrl的SPDU。

E. 4. 7 EcaEeCertResponseSpdu

```

EcaEeCertResponseSpdu ::= V2XSecData-Signed {
    ScmsPdu-Scoped {
        EcaEeInterfacePdu (WITH COMPONENTS {
            ecaEeCertResponse
        })
    },
    SecurityMgmtAid
}

```

此结构是用于发送已签名的EcaEeCertResponse的SPDU。为了使签名有效，签名证书应包含等于SecurityMgmtAid的AID和相应的SSP（COER编码的ScmsSsp结构，且选择EcaSsp）。

E. 4. 8 EeEcaCertRequestSpdu

```

EeEcaCertRequestSpdu ::= V2XSecData-SignedCertRequest {
    ScmsPdu-Scoped {
        EcaEeInterfacePdu (WITH COMPONENTS {
            eeEcaCertRequest
        })
    },
    SignerSelf
}

```

此结构是用于发送已签名的EeEcaCertRequest的SPDU。V2X设备应使用EC申请中的公钥（存放于EeEcaCertRequest中tbsCert.verifyKeyIndicator字段）对应的私钥对此结构进行签名。

E. 4. 9 EeRaCertRequestPlainSpdu

```

EeRaCertRequestPlainSpdu ::= V2XSecData-Unsecured {
    ScmsPdu-Scoped {
        EeRaInterfacePdu (WITH COMPONENTS {
            eeRaCertRequest
        })
    }
}

```

此结构是用于发送不安全（未签名或加密）的EeRaCertRequest的SPDU，V2X设备在已经使用其他安全手段进行保护时，使用该数据结构，例如：OAuth2.0机制进行认证等。

E. 4. 10 EeRaCertRequestSpdu

```

EeRaCertRequestSpdu ::= V2XSecData-SignedEncryptedCertRequest {
    ScmsPdu-Scoped {
        EeRaInterfacePdu (WITH COMPONENTS {
            eeRaCertRequest
        })
    },
    SignerSingleCert
}

```

此结构是用于发送先签名后加密的EeRaCertRequest的SPDU。V2X设备使用其注册证书签名此结构。V2X设备再使用RA证书中的encryptionKey对签过名的结构进行加密。

E. 4. 11 EeRaDownloadRequestPlainSpdu

```

EeRaDownloadRequestPlainSpdu ::= V2XSecData-Unsecured {
    ScmsPdu-Scoped {
        EeRaInterfacePdu (WITH COMPONENTS {
            eeRaDownloadRequest
        })
    }
}

```

此结构是用于发送不安全的EeRaDownloadRequest的SPDU。

E. 4. 12 EeRaDownloadRequestSpdu

```

EeRaDownloadRequestSpdu ::= V2XSecData-SignedEncrypted {
    ScmsPdu-Scoped {
        EeRaInterfacePdu (WITH COMPONENTS {
            eeRaDownloadRequest
        })
    },
    SecurityMgmtAid
}

```

此结构是用于发送先签名后加密的EeRaDownloadRequest的SPDU。V2X设备使用其注册证书签名此结构。V2X设备再使用RA证书中的encryptionKey对签过名的结构进行加密。

E. 4. 13 EeRaSuccessorEnrollmentCertRequestSpdu

```
EeRaSuccessorEnrollmentCertRequestSpdu ::=
  V2XSecData-SignedEncryptedCertRequest { ScmsPdu-Scoped {
    EeRaInterfacePdu (WITH COMPONENTS {
      eeRaSuccessorEnrollmentCertRequest
    })
  },
  SignerSingleCert
}
```

此结构是用于发送先签名后加密的eeEcaCertRequest的SPDU。V2X设备使用其注册证书签名此结构。V2X设备再使用RA证书中的encryptionKey对签过名的结构进行加密。

E. 4. 14 RaEeCertAckSpdu

```
RaEeCertAckSpdu ::= V2XSecData-Signed {
  ScmsPdu-Scoped {
    EeRaInterfacePdu (WITH COMPONENTS {
      raEeCertAck
    })
  },
  SecurityMgmtAid
}
```

此结构是用于对已收到的EeRaCertRequestSpdu进行确认的SPDU。

E. 4. 15 RaEeCertInfoSpdu

```
RaEeCertInfoSpdu ::= V2XSecData-Unsecured {
  ScmsPdu-Scoped {
    EeRaInterfacePdu (WITH COMPONENTS {
      raEeCertInfo
    })
  }
}
```

此结构是用于创建未签名的.info文件的SPDU，该.info文件将与批量证书一起放入zip文件中。

zip文件内的PC证书下载文件是未压缩的。zip文件的生成应遵循PKWare Zip文件格式规范版本6.3.6。与证书关联的i值和j值定义如下：这两个值被编码为以最小长度十六进制表示的ASCII字符串（例如，数值10表示为a而不是0a）。

——i 值是证书中 iCert 的 ASCII 十六进制表示形式；

——j 值是 0 到 jmax-1 之间的数值的 ASCII 十六进制表示形式，其中 jmax 是每周向终端签发证书的数量。

下载文件包含两种类型：基于密钥衍生机制的证书请求的下载文件和未基于密钥衍生机制的证书请求的下载文件。

针对基于密钥衍生机制的证书请求的下载文件，文件名为h-i.zip的形式，其中

——h 是已提交的证书请求 EeRaCertRequestSpdu 或 EeRaCertRequestPlainSpdu 的 HashedId8 的 16 字节 ASCII 十六进制表示形式，在 RaEeCertAckSpdu.raEeCertInfo 的 requestHash 字段中返回；

——i 即为 i 值。

针对未基于密钥衍生机制的证书请求的下载文件，文件名为h.zip的形式，其中

——h 是已提交的证书请求 EeRaCertRequestSpdu 或 EeRaCertRequestPlainSpdu 的 HashedId8 的 16 字节 ASCII 十六进制表示形式，在 RaEeCertAckSpdu.raEeCertInfo 的 requestHash 字段中返回。

证书下载.zip文件的内容被组织成一个目录，该目录中包含一周的证书和元数据，文件总数为 jmax+1，其中

——根目录下第一个文件是名为 x.info 的 info 文件，文件内容是 C-OER 编码的 RaEeCertInfoSpdu，x 代表该批证书的 i 值（本标准不使用 ACPC）；

——第二个文件开始是证书文件，每个证书文件内容是 C-OER 编码的 AcaRaCertResponse.acaResponse，命名格式为 x_y 且不带任何扩展名，这里 x 是 i 值，y 是 j 值，例如，iCert=30，jmax=60 的证书命名方式如下：1E_0, 1E_1, ..., 1E_3B, 1E_3C。

E.4.16 RaEeEnrollmentCertAckSpdu

```
RaEeEnrollmentCertAckSpdu ::= V2XSecData-Signed {
    ScmsPdu-Scoped {
        EeRaInterfacePdu (WITH COMPONENTS {
            raEeCertInfo
        })
    },
    SecurityMgmtAid
}
```

此结构是用于发送已签名的RaEeCertInfo的SPDU。

E.4.17 SecurityMgmtAid

```
SecurityMgmtAid ::= Aid (3627)
```

此AID应为十进制3627，用于标识本标准中定义的安全证书管理应用。

E.5 参数化类型 (parameterized types)

E.5.1 概述

E.5中的数据结构是用于定义E.4的SPDU结构的参数化类型。这些结构以字母顺序排列。

E.5.2 V2XSecData-Encrypted

```
V2XSecData-Encrypted {Tbe} ::= V2XSecData (WITH COMPONENTS {
    ...,
```

```

content (WITH COMPONENTS {
    encryptedData (CONSTRAINED BY {
        --encryption of-- Tbe
    })
})
})

```

此结构定义了一个参数化类型，基于V2XSecData的子类型创建一个被加密的数据。在此结构中首先对Tbe进行加密，然后将得到的密文作为encryptedData字段的输入。

E. 5.3 V2XSecData-EncryptedOpen

```

V2XSecData-EncryptedOpen ::= V2XSecData (WITH COMPONENTS {
    ...,
    content (WITH COMPONENTS {
        encryptedData
    })
})

```

此结构定义了一个参数化类型，基于V2XSecData的子类型创建一个被加密的数据。此结构与V2XSecData-Encrypted的不同之处在于它不指定加密数据的内容。

E. 5.4 V2XSecData-EncryptedSigned

```

V2XSecData-EncryptedSigned {Tbes, Aid} ::= V2XSecData-Signed {
    V2XSecData-Encrypted {
        Tbes
    },
    Aid
}

```

此结构定义了一个参数化类型，基于V2XSecData的子类型创建一个先被加密后被签名的数据。

E. 5.5 V2XSecData-Signed

```

V2XSecData-Signed {Tbs, Aid} ::= V2XSecData (WITH COMPONENTS {
    ...,
    content (WITH COMPONENTS {
        ...,
        signedData (WITH COMPONENTS {
            ...,
            tbsData (WITH COMPONENTS {
                ...,
                payload (WITH COMPONENTS {
                    ...,
                    data (WITH COMPONENTS {
                        ...,

```

```

        content (WITH COMPONENTS {
            unsecuredData (CONTAINING Tbs)
        })
    }),
    headerInfo (WITH COMPONENTS {
        ...,
        aid (Aid),
        generationTime ABSENT,
        expiryTime ABSENT,
        generationLocation ABSENT,
        p2pcdLearningRequest ABSENT,
        missingCrlIdentifier ABSENT,
        encryptionKey ABSENT
    })
    }),
    signer (SignerSingleCert)
    })
    })
    })

```

此结构定义了一个参数化类型，基于V2XSecData的子类型创建一个被签名的数据。

E. 5. 6 V2XSecData-SignedCertRequest

```

V2XSecData-SignedCertRequest {Tbscr, Signer} ::=
V2XSecData (WITH COMPONENTS {
    ...,
    content (WITH COMPONENTS {
        ...,
        signedCertificateRequest (CONTAINING
            SignedCertificateRequest (WITH COMPONENTS {
                ...,
                tbsRequest (Tbscr),
                signer (Signer)
            })
        })
    })
    })
    })

```

此结构定义了一个参数化类型，基于V2XSecData的子类型创建一个被签名的证书申请。

E. 5. 7 V2XSecData-SignedEncrypted

```

V2XSecData-SignedEncrypted {Tbse, Aid} ::= V2XSecData-Encrypted {
    V2XSecData-Signed {
        Tbse,

```

```

        Aid
    }
}

```

此结构定义了一个参数化类型，基于V2XSecData的子类型创建一个先被签名再被加密的数据。

E. 5. 8 V2XSecData-SignedEncryptedCertRequest

```

V2XSecData-SignedEncryptedCertRequest {Tbstecr, Signer} ::=
V2XSecData-Encrypted {
    V2XSecData-SignedCertRequest {
        Tbstecr,
        Signer
    }
}

```

此结构定义了一个参数化类型，基于V2XSecData的子类型创建一个先被签名再被加密的证书申请。

E. 5. 9 V2XSecData-Unsecured

```

V2XSecData-Unsecured {Tbu} ::= V2XSecData (WITH COMPONENTS {
    content (WITH COMPONENTS {
        ...,
        unsecuredData (CONTAINING Tbu)
    })
})

```

此结构定义了一个参数化类型，基于V2XSecData的子类型创建一个无安全处理的数据。

E. 5. 10 ScmsPdu-Scoped

```

ScmsPdu-Scoped {Pdu} ::= ScmsPdu (WITH COMPONENTS {
    ...,
    content (CONSTRAINED BY {
        Pdu
    })
})

```

此结构定义了一个参数化类型，基于ScmsPdu的子类型创建一个限定范围的数据。

E. 5. 11 ScopedCertificateRequest

```

ScopedCertificateRequest ::= ScmsPdu (
    ScmsPdu-Scoped {
        AcaRaInterfacePdu (WITH COMPONENTS {
            raAcaCertRequest
        })
    } |
    ScmsPdu-Scoped {
        EcaEeInterfacePdu (WITH COMPONENTS {

```

```

        eeEcaCertRequest
    })
} |
ScmsPdu-Scoped {
    EeRaInterfacePdu (WITH COMPONENTS {
        eeRaCertRequest
    })
} |
ScmsPdu-Scoped {
    EeRaInterfacePdu (WITH COMPONENTS {
        eeRaSuccessorEnrollmentCertRequest
    })
}
)

```

此结构将所有证书申请请求结构，定义为ScmsPdu的限定范围版本。

E. 5. 12 SignedCertificateRequest

```

SignedCertificateRequest ::= SEQUENCE {
    hashId      HashAlgorithm,
    tbsRequest  ScopedCertificateRequest,
    signer      SignerIdentifier,
    signature   Signature
}

```

该结构定义了已被签名的证书申请请求的格式。在此结构中：

- a) hashId 是当前请求所使用的哈希算法。
- b) tbsRequest 包含了由发送者签名的证书申请请求信息。
- c) signer 表示签名实体的标识符。
- d) signature 包含了请求发送者的签名。

E. 5. 13 SignerSelf

```

SignerSelf ::= SignerIdentifier(WITH COMPONENTS {
    self
})

```

本结构定义了没有证书的签名者，用于表示自签名。

E. 5. 14 SignerSingleCert

```

SignerSingleCert ::= SignerIdentifier(WITH COMPONENTS {
    certificate (SequenceOfCertificate (SIZE (1)))
})

```

本结构定义了证书链长度只有1的签名者。

E. 6 ACA-RA接口

E. 6.1 概述

E. 7中的数据结构适用于ACA-RA接口。这些结构按照E. 3. 1中的方法进行排序，例外是相关的SPDU数据结构，他们在E. 7的最后。

E. 6.2 AcaRaInterfacePdu

```
AcaRaInterfacePdu ::= CHOICE {
    raAcaCertRequest    RaAcaCertRequest,
    acaRaCertResponse   AcaRaCertResponse,
    ...
}
```

这是ACA和RA之间所有交互数据的数据结构，其中：

- a) raAcaCertRequest: RA 代表 V2X 设备向授权证书 CA 提交的授权证书请求。
- b) acaRaCertResponse: 授权证书 CA 对 RaAcaCertRequest 的响应。

E. 6.3 RaAcaCertRequest

```
RaAcaCertRequest ::= SEQUENCE {
    version            Uint8 (1),
    generationTime     Time32,
    flags              RaAcaCertRequestFlags,
    linkageInfo         EncryptedIndividuallV OPTIONAL,
    certEncKey          PublicEncryptionKey OPTIONAL,
    tbsCert             ToBeSignedCertificate (WITH COMPONENTS {
        ...,
        cracaId ('000000'H),
        crlSeries (0),
        appPermissions PRESENT,
        certIssuePermissions ABSENT,
        certRequestPermissions ABSENT
    }),
    ...
}
```

本结构包含了请求单个授权证书所需参数。其中：

- a) version: 本结构的当前版本号，默认为 1。
- b) generationTime: RaAcaCertRequest 的生成时间。
- c) flags: 与使用密钥衍生机制相关的标志位，并向授权证书 CA 提供如何生成响应的相关指令。
- d) EncryptedIndividuallV 为已加密的用于生成证书的链接值。本字段如果存在，则 tbsCert.id 字段应选择 LinkageData 类型，其中 iCert 字段应设置为实际的 i-period 值，而 linking-value 字段设置为虚拟值，授权证书 CA 将使用实际的链接值进行替换。发送至授权证书 CA 的链接值由 LA 加密。该结构体见 E. 8. 11。

- e) certEncKey 包含了用于加密证书的公钥。如果 certEncKey 字段存在，则授权证书 CA 的响应应取决于标志位 flags.useCubk。
- f) tbsCert 包含所请求证书的参数。如果字段 tbsCert.verifyKeyIndicator 为 verificationKey，则证书的类型为显式。如果字段 tbsCert.verifyKeyIndicator 为 reconstructionValue，则证书的类型为其他。

E. 6. 4 RaAcaCertRequestFlags

RaAcaCertRequestFlags ::=

BIT STRING {butterflyExplicit (0), useCubk (1)} (SIZE (8))

此结构用于将有关生成证书时要执行的操作的信息从RA传递到ACA。

E. 6. 5 AcaRaCertResponse

AcaRaCertResponse ::= SEQUENCE {

version Uint8 (1),

generationTime Time32,

requestHash HashedId8,

acaResponse AcaResponse,

...

}

AcaResponse ::= CHOICE {

plain AcaEeCertResponsePlainSpdu,

private AcaEeCertResponsePrivateSpdu,

cubk AcaEeCertResponseCubkSpdu,

...

}

本结构为响应RaAcaCertRequestSpdu所需的参数，其中：

- a) version 包含本结构的当前版本号，默认为 1。
- b) generationTime 为 AcaRaCertResponse 的生成时间。
- c) requestHash 包含响应的 RaAcaCertRequestSpdu 的哈希值。
- d) plain 包含的 V2X 设备的授权证书，没有被加密或签名。仅当相应的 RaAcaCertRequest 中的字段 certEncKey 不存在时，才应使用此选择。
- e) private 包含的 V2X 设备的授权证书，其先被加密再被签名，用来保护 V2X 设备的隐私免受 RA 侵害。仅当相应的 RaAcaCertRequest 中存在 certEncKey 字段且 flags.useCubk 没有置位时，才应使用此选择。
- f) cubk 包含的 V2X 设备的授权证书，是只加密的形式。仅当相应的 RaAcaCertRequest 中 flags.useCubk 被置位时，才应使用此选择。

E. 6. 6 RaAcaCertRequestSpdu

RaAcaCertRequestSpdu ::= V2XSecData-SignedCertRequest {

ScmsPdu-Scoped {

AcaRaInterfacePdu (WITH COMPONENTS {

raAcaCertRequest

```

    })
  },
  SignerSingleCert
}

```

此结构是用于发送被签名的RaAcaCertRequest的SPDU，为了使签名有效，签名证书应包含等于SecurityMgmtAid的AID和相应的SSP（COER编码的ScmsSsp结构，且选择RaSsp）。

E. 6.7 AcaRaCertResponseSpdu

```

AcaRaCertResponseSpdu ::= V2XSecData-Signed {
  ScmsPdu-Scoped {
    AcaRaInterfacePdu (WITH COMPONENTS {
      acaRaCertResponse
    })
  },
  SecurityMgmtAid
}

```

此结构是用于发送被签名的AcaRaCertResponse的SPDU。为了使签名有效，签名证书应包含等于SecurityMgmtAid的AID和相应的SSP（COER编码的ScmsSsp结构，且选择AcaSsp）。

E. 7 LA-MA接口

E. 7.1 概述

0中的数据结构适用于LA-MA接口。这些结构按照E. 3.1中的方法进行排序，例外是相关的SPDU数据结构，他们在0的最后。

E. 7.2 LaMaInterfacePdu

```

LaMaInterfacePdu ::= CHOICE{
  maLaLinkageSeedRequest MaLaLinkageSeedRequest,
  laMaLinkageSeedResponse LaMaLinkageSeedResponse,
  ...
}

```

LaMaInterfacePdu是LA与MA之间所有数据交互的母结构体。在此结构体中：

- a) maLaLinkageSeedRequest 包含了 Ma 向 La 发起的链接种子请求的消息体内容。
- b) laMaLinkageSeedResponse 包含了 La 接收到 Ma 的链接种子请求消息后做出的响应。

E. 7.3 MaLaLinkageSeedRequest

```

MaLaLinkageSeedRequest ::= SEQUENCE{
  maId OCTET STRING (SIZE(0..255)),
  individualLVs SEQUENCE (SIZE(0..MAX)) OF IndividualLV,
  groupLVs SEQUENCE (SIZE(0..MAX)) OF GroupLV
}

```

此结构体用于设置Ma到La的链接种子请求消息的内部内容。其中：

- a) maId 为 Ma 的唯一主机名，表示着发送方的地址。
- b) individualLVs 包含了 Ma 请求需要获取链接种子的个体链接值，其大小限制为 0-max，当只需要请求组链接种子时候，可以将数量设置为 0。
- c) groupLVs 包含了 Ma 请求需要获取组链接种子的组链接值，其大小限制为 0-max，当只需要请求个体链接种子时候，可以将数量设置为 0。

E. 7. 4 IndividualLV

```
IndividualLV ::= SEQUENCE {
    iValue      IValue,
    lv          LinkageValue
}
```

此结构体用于设置Ma请求La获取个体链接种子的个体链接值参数。其中：

- a) iValue 为该个体链接值对应的 i 值。
- b) lv 为需要向 La 获取链接种子的个体链接值。

E. 7. 5 GroupLV

```
GroupLV ::= SEQUENCE {
    iValue      IValue,
    j           Uint32,
    lv          LinkageValue
}
```

此结构体用于设置Ma请求La获取组链接种子的组链接值参数。其中：

- a) iValue 为该组链接值对应的 i 值。
- b) j 为该组链接值对应的 j 值。
- c) lv 为需要向 La 获取组链接种子的组链接值。

E. 7. 6 LaMaLinkageSeedResponse

```
LaMaLinkageSeedResponse ::= SEQUENCE{
    requestHash      HashedId8,
    laId             LaId,
    individualLSInfos SEQUENCE(SIZE(0..MAX)) OF IndividualLSInfo,
    groupLSInfos      SEQUENCE(SIZE(0..MAX)) OF GroupLSInfo
}
```

此结构体用于设置La根据Ma的链接种子请求的相关信息，响应对应的链接种子内容的构体。其中：

- a) requestHash 为 Ma 请求获取链接种子结构体的 hashId8。
- b) laId 为 La 唯一标识符，撤销证书时使用。
- c) individualLSInfos 为根据 Ma 请求发送的个体链接值获取到的对应的个体链接种子，个数与请求的个体链接值个数相同。
- d) groupLSInfos 为根据 Ma 请求发送的组链接值获取到的对应的组链接种子，个数与请求的组链接值个数相同。

E. 7. 7 IndividualLSInfo

```

IndividualLSInfo ::= SEQUENCE {
    i      IValue,
    lv     LinkageValue,
    ls     LinkageSeed
}

```

此结构体用于设置Ma请求的个体链接值对应的个体链接种子。其中：

- a) i 为 Ma 发送的个体链接值对应的 i 值。
- b) lv 为 Ma 发送的个体链接值。
- c) ls 为该个体链接值对应的链接种子。

E. 7. 8 GroupLSInfo

```

GroupLSInfo ::= SEQUENCE {
    i      IValue,
    j      Uint32,
    lv     LinkageValue,
    ls     LinkageSeed
}

```

此结构体用于设置Ma请求的组链接值对应的组链接种子。其中：

- a) i 为 Ma 发送的组链接值对应的 i 值。
- b) j 值为 Ma 发送的组链接值对应的 j 值
- c) lv 为 Ma 发送的组链接值。
- d) ls 为该组链接值对应的链接种子。

E. 7. 9 MaLaLinkageSeedRequestSpdu

```

MaLaLinkageSeedRequestSpdu ::= V2XSecData-SignedEncrypted {
    ScmsPdu-Scoped {
        LaMaInterfacePdu (WITH COMPONENTS {
                                maLaLinkageSeedRequest
                            })
    },
    SecurityMgmtAid
}

```

此结构用于Ma请求La获取链接种子结构体，其中签名证书选择Ma的证书，加密证书选择接收者La证书，签名使用的Aid为默认使用SecurityMgmtAid。

E. 7. 10 LaMaLinkageSeedResponseSpdu

```

LaMaLinkageSeedResponseSpdu ::= V2XSecData-SignedEncrypted {
    ScmsPdu-Scoped {
        LaMaInterfacePdu (WITH COMPONENTS {
                                laMaLinkageSeedResponse
                            })
    },
}

```

```

        SecurityMgmtAid
    }

```

此结构用于La响应Ma获取链接种子结构体，其中签名证书选择La的证书，加密证书选择接收者Ma证书，签名使用的Aid为默认使用SecurityMgmtAid。

E.8 LA-RA接口

E.8.1 概述

E.8 中的数据结构适用于LA-RA接口。这些结构按照E.3.1中的方法进行排序，例外是相关的SPDU数据结构，他们在E.8 的最后。

E.8.2 LaRaInterfacePdu

```

LaRaInterfacePdu ::= CHOICE{
    praLaLinkageValueRequest PraLaLinkageValueRequest,
    laPraLinkageValueResponse LaPraLinkageValueResponse,
    ...
}

```

LaRaInterfacePdu是LA与RA之间所有数据交互的母结构体。在此结构体中：

- praLaLinkageValueRequest 包含了 Pra 向 La 发起的链接值请求的消息体内容。
- laPraLinkageValueResponse 包含了 La 接收到 Pra 的链接值请求消息后做出的响应。

E.8.3 PraLaLinkageValueRequest

```

PraLaLinkageValueRequest ::= CHOICE{
    individualLVRequest PraLaIndividualLinkageValueRequest,
    groupLVRequest PraLaGroupLinkageValueRequest
}

```

该结构包含了个体的获取结构和组链接值的获取结构，其中：

- individualLVRequest 包含了 Pra 向 La 发起的个体链接值请求的消息体内容。
- groupLVRequest 包含了 Pra 向 La 发起的组链接值请求的消息体内容。

E.8.4 PraLaIndividualLinkageValueRequest

```

PraLaIndividualLinkageValueRequest ::= SEQUENCE {
    header          PraLaRequestHeader,
    jMax            UInt8,
    numberOfFreshInd  UInt32 OPTIONAL,
    continuationsInd SEQUENCE OF LinkageChainId OPTIONAL
}

```

该结构包含了Pra请求获取个体链接值的参数内容，其中：

- header 为请求头，组链接值和个体链接值公用参数。
- jMax 为 Pra 需要为每条链每个周期 i 获取到的链接值的个数。

- c) numberOfFreshInd 为 Pra 需要申请的新的个体链接链的条数（用于之前没有申请过的设备，或之前的链因为某种原因不能使用的情况）。
- d) continuationsInd 为旧的个体链对应的链接标识符，LA 根据链接标识符以获取到对应的链接种子链。

E.8.5 LinkageChainId

```
LinkageChainId ::= EncryptedData (WITH COMPONENTS {
    recipients (SIZE (1) INTERSECTION (WITH COMPONENT (
        RecipientInfo (WITH COMPONENTS {
            certRecipInfo
        }))))),
    ciphertext
}
```

该结构用于描述链接标识符结构，其中该结构仅使用EncryptedData结构体，且内部固定选择加密类型为证书加密，其中加密证书为LA证书，加密数据原文为该链对应的初始链接种子ls(0)。

E.8.6 PraLaGroupLinkageValueRequest

```
PraLaGroupLinkageValueRequest ::= SEQUENCE {
    header          PraLaRequestHeader,
    jMax            Uint32,
    groupIdentifier OCTET STRING (SIZE(4))
}
```

该结构包含了Pra请求获取组链接值的参数内容，其中：

- a) header 为请求头，组链接值和个体链接值请求的公用参数。
- b) jMax 为 Pra 需要为每条链每个周期 i 获取到的链接值的个数。
- c) groupIdentifier 为组链标识符，由请求方提供，LA 会为每个组链标识符标识一条链，如果该组链标识符之前没有申请获取组链接值，则 LA 会为该组链接标识符生成一条新链，如果改组链标识符之前已经在 LA 申请过组链接值，则 LA 会继续使用该链为 Pra 生成组链接值。

E.8.7 PraLaRequestHeader

```
PraLaRequestHeader ::= SEQUENCE {
    iMin            Uint16,
    iMax            Uint16,
    praId           Hostname,
    pcaId           Hostname
}
```

该结构包含了Pra请求获取组接值的请求头参数内容，其中：

- a) iMin 为 Pra 本次请求需要生成的链接值对应的最小 i 值。
- b) iMax 为 Pra 本次请求需要生成的链接值对应的最大 i 值。
- c) praId 为 Pra 的服务地址。
- d) pcaId 为 Pca 的服务地址，La 通过该值判断使用哪个 PCA 公钥证书进行加密链接值。

E. 8. 8 LaPraLinkageValueResponse

```

LaPraLinkageValueResponse ::= SEQUENCE {
    iMin                Uint16,
    iMax                Uint16,
    individuallVs       SEQUENCE (SIZE (0..MAX)) OF IndividualLVResponseLinkageChain,
    groupLVs            SEQUENCE (SIZE (0..MAX)) OF GroupLVResponseLinkageChain
}

```

该结构体包含了La通过Pra请求的获取链接值参数生成的链接值的内容，其中：

- a) iMin 为 Pra 请求的 iMin。
- b) iMax 为 Pra 请求的 iMax。
- c) individuallVs 为 La 为 Pra 生成的加密个体链接值。
- d) groupLVs 为 La 为 Pra 生成的加密组链接值。

E. 8. 9 IndividualLVResponseLinkageChain

```

IndividualLVResponseLinkageChain ::= SEQUENCE {
    jMax                Uint8,
    values              SEQUENCE OF
                        SEQUENCE OF EncryptedIndividualLV,
    linkageChainId      LinkageChainId
}

```

该结构包含了Pra请求获取组接值的请求头参数内容，其中：

- a) jMax 为 Pra 请求的 jMax。
- b) value 为从 iMin-iMax 顺序、且每个 i 下都包含顺序的 jMax 个加密的个体链接值，j 的取值为顺序的 0 - jMax-1。
- c) linkageChainId 为该批次的个体链接值对应的链接标识符。

E. 8. 10 GroupLVResponseLinkageChain

```

GroupLVResponseLinkageChain ::= SEQUENCE {
    jMax                Uint32,
    values              SEQUENCE OF
                        .....
                        ..... SEQUENCE OF EncryptedGroupLV,
    groupIdentifier      OCTET STRING (SIZE(4))
}

```

该结构包含了Pra请求获取组接值的请求头参数内容，其中：

- a) jMax 为 Pra 请求的 jMax。
- b) value 为从 iMin-iMax 顺序、且每个 i 下都包含顺序的 jMax 个加密的组链接值，j 的取值为顺序的 0 - jMax-1。
- c) groupIdentifier 为该批次的组链接值对应的链接标识符，该值通过 Pra 请求获取。

E. 8. 11 EncryptedIndividualLV

```

EncryptedIndividualLV ::= SEQUENCE {

```

```

version      Uint8(1),
laId         LaId,
enc-lv       EncryptedData (WITH COMPONENTS {
  recipients (SIZE (1) INTERSECTION (WITH COMPONENT (
    RecipientInfo (WITH COMPONENTS {
      certRecipInfo
    }))))),
                                ciphertext
  })
}

```

该结构包含了La相应Pra请求链接值中的加密个体链接值参数，其中：

- a) version 为当前结构体对应的版本，默认为 1。
- b) laId 为 La 唯一标识符。
- c) enc-lv 为加密的个体链接值，使用 PCA 公钥证书加密，PCA 证书根据 Pra 请求的 Pca 服务地址获取，加密数据的原文为 IndividualLV。

E. 8. 12 EncryptedGroupLV

```

EncryptedGroupLV ::= SEQUENCE {
  version      Uint8(1),
  laId         LaId,
  enc-lv       EncryptedData (WITH COMPONENTS {
    recipients (SIZE (1) INTERSECTION (WITH COMPONENT (
      RecipientInfo (WITH COMPONENTS {
        certRecipInfo
      }))))),
                                ciphertext
  })
}

```

该结构包含了La相应Pra请求链接值中的加密组链接值参数，其中：

- a) version 为当前结构体对应的版本，默认为 1。
- b) laId 为 La 唯一标识符。
- c) enc-lv 为加密的组链接值，使用 PCA 公钥证书加密，PCA 证书根据 Pra 请求的 Pca 服务地址获取，加密数据的原文为 GroupLV。

E. 8. 13 PraLaIndividualLinkageValueRequestSpdu

```

PraLaIndividualLinkageValueRequestSpdu ::= V2XSecData-SignedEncrypted {
  ScmsPdu-Scoped {
    LaRaInterfacePdu (WITH COMPONENTS {
      praLaLinkageValueRequest (WITH COMPONENTS {
        ..... individualLVRequest
      })
    })
  }
}

```

```

    },
    SecurityMgmtAid
}

```

此结构用于Pra-La请求个体链接值的结构体，其中签名证书选择Pra的证书，加密证书选择接收者La证书，签名使用的Aid为默认使用SecurityMgmtAid。

E. 8. 14 PraLaGroupLinkageValueRequestSpdu

```

PraLaGroupLinkageValueRequestSpdu ::= V2XSecData-SignedEncrypted {
    ScmsPdu-Scoped {
        LaRaInterfacePdu (WITH COMPONENTS {
            praLaLinkageValueRequest (WITH COMPONENTS {
                ..... groupLVRequest
            })
        })
    },
    SecurityMgmtAid
}

```

此结构用于Pra-La请求组链接值的结构体，其中签名证书选择Pra的证书，加密证书选择接收者La证书，签名使用的Aid为默认使用SecurityMgmtAid。

E. 8. 15 LaPraLinkageValueResponseSpdu

```

LaPraLinkageValueResponseSpdu ::= V2XSecData-SignedEncrypted {
    ScmsPdu-Scoped {
        LaRaInterfacePdu (WITH COMPONENTS {
            laPraLinkageValueResponse
        })
    },
    SecurityMgmtAid
}

```

此结构用于La响应Pra请求个体链接值和组链接值的结构体，其中签名证书选择La的证书，加密证书选择接收者Pra证书，签名使用的Aid为默认使用SecurityMgmtAid。

E. 9 GBA流程相关接口

E. 9. 1 概述

本节定义GBA架构下，V2X设备与AAA、ECA/PRA/ARA证书机构之间所交互的请求及响应消息格式，以便完成EC/PC/AC安全证书的申请、更新等操作。涉及C1、C2、C3接口，按照业务流程顺序排序。

E. 9. 2 V2X设备→AAA发送的请求消息

V2X设备向ECA/PRA/ARA发送的证书请求消息，请求消息类型为：HTTP GET / HTTP/1.1，HTTP消息体内可携带加密的数据信息。

Host填写为：证书Server的FQDN

Authorization中username填写为：B-TID信息

HTTP request body为：加密的请求数据信息

其他字段请参考RFC 2617。

其中，B-TID：引导事务标识，终端用户在GBA引导过程中生成的用户临时标识。

E.9.3 AAA→ECA/PRA/ARA转发的请求消息

AAA向ECA/PRA/ARA转发V2X设备发送的证书请求消息。HTTP消息中应扩展HTTP头：X-Key-Material，用于携带B-TID信息。采用巴科斯BNF定义如下：

```
X-Key-Material    = "X-Key-Material" ":" Key-info
Key-info          = 1#(B-TID)
```

E.9.4 ECA/PRA/ARA→AAA发送的请求消息

ECA/PRA/ARA向AAA发送的用户信息请求消息。请求消息类型为：HTTP GET / HTTP/1.1，消息中应扩展HTTP头：X-User-Info，用于携带B-TID及证书Server的FQDN信息。采用巴科斯BNF定义如下：

```
X-User-Info      = "X-User-Info" ":" User-Info
User-Info        = 1#(B-TID| [ServerFQDN])
B-TID            = "B-TID" "=" B-TID
ServerFQDN       = "ServerFQDN" "=" ServerFQDN
```

在PRA/ARA向AAA发起授权请求的情况下，HTTP消息体内携带加密的用户身份的隐私信息。

E.9.5 AAA→ECA/PRA/ARA发送的响应消息

接收到ECA/PRA/ARA的查询请求后，AAA回复的成功响应消息。响应消息类型为：HTTP 200 OK，消息中应扩展HTTP头：X-Key-Material，用于携带提供给GBA应用层会话密钥。采用巴科斯BNF定义如下：

```
X-Key-Material    = "X-Key-Material" ":" Key-info
Key-info          = 1#(Key-V2X-Enc| [Key-V2X-Int] | [Key-V2X-Auth] | [Key-E2E-Sec] | [Key-ExpiryTime])
Key-V2X-Enc       = "Key-V2X-Enc" "=" base64Binary (Key-V2X-Enc)
Key-V2X-Int       = "Key-V2X-Int" "=" base64Binary (Key-V2X-Int)
Key-V2X-Auth      = "Key-V2X-Auth" "=" base64Binary (Key-V2X-Auth)
Key-V2X-E2E-Sec   = "Key-V2X-E2E-Sec" "=" base64Binary (Key-V2X-E2E-Sec)
Key-ExpiryTime    = "Key-ExpiryTime" "=" Key-ExpiryTime
```

其中，Key-V2X-Enc：证书请求/响应消息加密密钥；

Key-V2X-Int：证书请求/响应消息完整性保护密钥

Key-V2X-Auth：V2X设备-CA服务器双向身份认证密钥

Key-V2X-E2E-Sec：端到端安全通道建立密钥

Key-ExpiryTime：密钥超期时间，数值为自UTC 1900 年1月1日0点 以来的秒数；

应用层会话密钥生成规则，请参考附录F。

E.9.6 ECA/PRA/ARA→V2X设备发送的响应消息

证书请求处理后，ECA/PRA/ARA向V2X设备发送的成功响应消息。响应消息类型为：HTTP 200 OK，消息体内可携带加密的数据信息。

响应消息类型为：HTTP 200 OK。

HTTP 200 OK Response body为加密的响应数据信息。

其他字段的填写，请参考RFC 2617。

E. 9.7 GBA认证授权系统→V2X设备转发的响应消息

AAA向V2X设备转发ECA/PRA/ARA发送的HTTP 200 OK成功响应消息，定义与E. 8. 6相同。

E. 10 安全证书管理AID的SSP

E. 10.1 ScmsSsp

```
ScmsSsp ::= CHOICE {
    root      RootCaSsp,
    ma        MaSsp,
    pg        PgSsp,
    ica       IcaSsp,
    eca       EcaSsp,
cra         CraSsp,
    dcm       DcmSsp,
    aca       AcaSsp,
    la        LaSsp,
    ra        RaSsp,
    ee        EeSsp,
    ...
}
```

本节规定了安全证书管理应用AID十进制3627的相关SSP，该SSP仅存在于6. 1. 1定义的安全证书管理系统内各实体的证书中。

以EcaSsp为例，使用在注册证书机构ECA证书的ToBeSignedCertificate结构内（见6. 5. 7），其中的appPermissions字段对应的SequenceOfAidSsp（见6. 5. 27）中，应包含一组下述的AidSsp（AID和SSP的组合）：

- a) AidSsp（见6. 5. 27）中 aid 字段对应的 Aid，取值为十进制 3627；
- b) AidSsp 中 ssp 字段对应的 ServiceSpecificPermissions（见6. 5. 28）选择 opaque，对应的 OCTET STRING 的取值为 ScmsSsp（选择 EcaSsp 时）的 COER 编码。

注：因SPDU结构中（见6. 4），仅在HeaderInfo结构体（见6. 4. 7）内支持标识该SPDU所属的AID，并不支持标识SSP，因此当前未定义安全证书管理应用（AID十进制3627）中V2X设备申请证书、SCMS返回证书至V2X设备等各类具体消息所对应的SSP。

E. 10.2 RootCaSsp

```
RootCaSsp ::= SEQUENCE {
    version  Uint8 (1),
    ...
}
```

RootCaSsp定义了Root CA进行安全证书管理（AID十进制3627）时的SSP，当前仅定义了1个参数：版本号。

E. 10.3 MaSsp

```
MaSsp ::= SEQUENCE {
    version      Uint8 (1),
    relevantAids SequenceOfAid,
    ...
}
```

MaSsp定义了策略生成MA进行安全证书管理（AID十进制3627）时的SSP，当前定义参数包括参数：

- a) version-版本号，默认为1；
- b) relevantAid-表示该 MA 被授权能够接收的异常行为报告的 AID（当前仅包括异常行为管理 AID 十进制 3629）。包含此 SSP 的证书是 MA 证书，V2X 设备上报异常行为报告时使用 AID 十进制 3629。

E. 10.4 PgSsp

```
PgSsp ::= SEQUENCE {
    version  Uint8 (1),
    ...
}
```

PgSsp定义了策略生成PG进行安全证书管理（AID十进制3627）时的SSP，当前仅定义了1个参数：版本号。

E. 10.5 IcaSsp

```
IcaSsp ::= SEQUENCE {
    version  Uint8 (1),
    ...
}
```

IcaSsp定义了ICA进行安全证书管理（AID十进制3627）时的SSP，当前仅定义了1个参数：版本号。

E. 10.6 EcaSsp

```
EcaSsp ::= SEQUENCE {
    version  Uint8 (1),
    ...
}
```

EcaSsp定义了ICA进行安全证书管理（AID十进制3627）时的SSP，当前仅定义了1个参数：版本号。

E. 10.7 AcaSsp

```
AcaSsp ::= SEQUENCE {
    version  Uint8 (1),
    ...
}
```

AcaSsp定义了授权证书CA（6.1.1定义的PCA和ACA）进行安全证书管理（AID十进制3627）时的SSP，当前仅定义了1个参数：版本号。

E. 10.8 CraSsp

```
CraSsp ::= SEQUENCE {
    version  Uint8 (1),
    ...
}
```

CraSsp定义了CRA进行安全证书管理（AID十进制3627）时的SSP，当前仅定义了1个参数：版本号。

E. 10.9 DcmSsp

```
DcmSsp ::= SEQUENCE {
    version  Uint8 (1),
    ...
}
```

DcmSsp定义了DCM进行安全证书管理（AID十进制3627）时的SSP，当前仅定义了1个参数：版本号。

E. 10.10 LaSsp

```
LaSsp ::= SEQUENCE {
    version  Uint8 (1),
    laId     Uint16,
    ...
}
```

LaSsp定义了LA进行安全证书管理（AID十进制3627）时的SSP，当前定义参数包括参数：

- a) version-版本号；
- b) laId-为La 唯一标识符。

E. 10.11 RaSsp

```
RaSsp ::= SEQUENCE {
    version  Uint8 (1),
    ...
}
```

RaSsp定义了RA进行安全证书管理（AID十进制3627）时的SSP，当前仅定义了1个参数：版本号。

E. 10.12 EeSsp

```
EeSsp ::= SEQUENCE {
    version  Uint8 (1),
    ...
}
```

}

EeSsp定义了V2X设备的EC证书进行安全证书管理应用（AID十进制3627）时的SSP，当前仅定义了1个参数：版本号。

附 录 F
(规范性附录)
GBA 机制应用层会话密钥产生及使用方法

F.1 应用层会话密钥种类

本附录对GBA架构下注册证书、假名证书、应用证书处理流程中使用到的多种应用层会话密钥的种类进行规定，以便V2X设备、GBA认证授权系统、证书机构之间协商使用一致的会话密钥。
本标准使用的GBA应用层会话密钥如表F.1。

表 F.1 GBA 应用层会话密钥

GBA 应用层会话密钥	作用及功能	字符串
K1	证书请求/响应消息加密	V2X_Enc
K2	证书请求/响应消息完整性保护	V2X_Int
K3	V2X 设备-CA 服务器双向身份认证	V2X_Auth
K4	端到端安全通道建立	V2X_E2E_Sec
K5	V2X 设备用户身份隐私保护	V2X_ID_Sec

F.2 应用层会话密钥生成

K1~K5密钥产生方式如下：
 $K1 \sim K5 = KDF(Ks_int_NAF, \text{字符串}, B-TID, \text{UE标识}, \text{应用服务标识})$ 。其中，KDF是密钥生成算法，应采用国家密码管理局规定的商用密码算法实现，生成长度为128比特的会话密钥。Ks_int_NAF是基于GBA_U方式产生，在USIM与NAF间共享的GBA密钥；字符串参照上述表格选取；B-TID是GBA引导事务标识；UE标识是V2X设备的标识IMPI；应用服务标识为CA服务器的FQDN域名，该域名应确保全局唯一。

F.3 应用层会话密钥使用

在采用GBA应用层会话密钥K1/K5对数据做机密性保护时，应使用SM4算法，采用CBC模式实现。
在采用GBA应用层会话密钥K2对数据做完整性保护时，应使用HMAC-SM3算法实现。
在需要对数据同时做机密性和完整性保护的情况下，可采用SM4算法CCM模式，使用GBA应用层会话密钥K1实现。
在采用GBA应用层会话密钥K3做双向身份认证时，应使用挑战应答方式实现。
在采用GBA应用层会话密钥K4建立端到端安全通道时，应使用SM4算法，采用CBC或CCM模式实现。

附 录 G
(资料性附录)
证书生命周期及更新场景

G.1 证书生命周期

不同级别证书具有不同的生命周期，在证书的更新时，需要考虑彼此间的生命周期才能清楚定义相关的场景。

表G.1为现有证书生命周期摘要，证书使用年限为假设值，其他证书待补充。

表 G.1 证书生命周期摘要

证书类别	发行 CA	期限	使用周期	申请更新时间	更新有效期开始	并发有效证书数（正在使用[+旧版]）	示例大小（以字节为单位）（证书大小不固定）	注记
RSU 注册证书	ECA	6 years	6 years	任意时间	6 years	1	87	
RSU 假名证书	PCA	1 week + 1 hr	1 week	任意时间	1 week	1 + 1 (for just 1 hour)	89	
根证书	Self	15 years	8 years	终止前三个月	8 years	1 + 2	211	
注册中心	ICA	3 years + 1 wk	3 years	终止前三个月	3 years	1 + 1 (for just 1 week)	217	
OBU 假名证书	PCA	1 week + 1 hr	1 week	任意时间	1 week	20 + 20 (for just 1 hour)	86	
OBU 鉴别证书	PCA	1 month + 1 hr	1 month	任意时间	1 month	1 + 1 (for just 1 hour)	89	
OBU 注册证书	ECA	6 years	6 years	任意时间	6 years	1	87	
ICA	Root CA	13 years	4 years	终止前三个月	4 years	1 + 3	195	

PCA	ICA	4 years	1 year	终止前 三个月	1 year	1 + 3	216	
MA	Root CA	4 years + 1 wk	4 years	终止前 三个月	4 years	1 + 1 (for just 1 week)	205	
…其他								

G.2 证书更新场景

更新场景说明如图G.1所示。

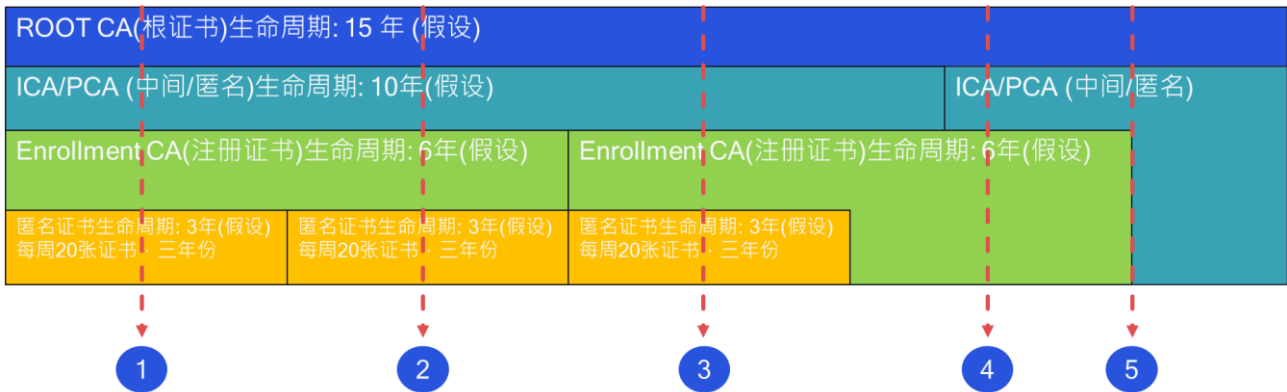


图 G.1 证书更新场景

场景一：当终端假名证书存储量低于一半，注册证书签署终端证书更新请求，执行以下检查：

- a) 注册证书有效期没有问题
- b) ICA/PCA 证书有效期大于欲签发的证书生命周期
- c) Root CA 证书有效期大于欲签发的证书生命周期

上述皆符合并签发终端证书。

场景二：当终端假名证书存储量低于一半，注册证书签署终端证书更新请求，执行以下检查：

- a) 注册证书有效期没有问题(若注册证书有效期低于假名证书发放周期，则应触发注册证书更新机制)
- b) ICA/PCA 证书有效期大于欲签发的证书生命周期
- c) Root CA 证书有效期大于欲签发的证书生命周期

上述皆符合并签发终端证书。

场景三：当终端假名证书存储量低于一半，注册证书签署终端证书更新请求，执行以下检查：

- a) 注册证书过期，需重新申请注册证书后，才能签署终端证书请求
- b) ICA/PCA 证书有效期大于欲签发的证书生命周期
- c) Root CA 证书有效期大于欲签发的证书生命周期

上述皆符合并签发终端证书。

场景四：当终端假名证书存储量低于一半，注册证书签署终端证书更新请求，执行以下检查：

- a) 注册证书有效期没有问题
- b) ICA/PCA 证书有效期可能低于欲签发的证书生命周期，需先进行 ICA/PCA 证书更新
- c) Root CA 证书有效期大于欲签发的证书生命周期

上述皆符合并签发终端证书

场景五：当终端假名证书存储量低于一半，注册证书签署终端证书更新请求，执行以下检查：

- a) 注册证书有效期没有问题
- b) ICA/PCA 证书有效期大于欲签发的证书生命周期
- c) Root CA 证书有效期可能低于欲签发的证书生命周期，则应重新签发根证书后，才进行证书之签署

上述皆符合并签发终端证书

附 录 H

（资料性附录）

密钥衍生流程的一种算法建议

H.1 概述

本附录所述算法是为了配合7.2.6 密钥衍生流程而提出的一种算法建议，其中需要使用的密钥衍生函数如K.2所述，该算法与7.2.6 结合后的密钥衍生流程如K.3所述。本附录中使用的对称加密算法、非对称加密算法为：

- 对称加密算法使用 GB/T 32907-2016 《信息安全技术 SM4 分组密码算法》。
- 非对称加密算法使用 GB/T 32918.2-2016 《信息安全技术 SM2 椭圆曲线公钥密码算法 第2部分：数字签名算法》。

H.2 密钥衍生函数

H.2.1 符号表达说明

以下符号表达说明适用于**错误!未找到引用源。**H：

- a) 有一个公认的“基点”，表示为 G ；
- b) 椭圆曲线的阶数用 l 表示；
- c) 对于一个比特串 b 和数字 n ， b^n 表示通过将比特串 b 重复 n 次而形成的比特串；
- d) 对于比特串 x ， x^{INT} 表示 x 转换为整数，例如：如果 $x = 0101$ ，则 $x^{INT} = 5$ ；
- e) 对于比特串 x 和数字 n ， $x + n$ 是 $x^{INT} + n$ 产生的比特串的简写，例如：如果 $x = 0100$ ，则 $x + 1 = 0101$ ， $x + 2 = 0110$ ， $x + 3 = 0111$ ，依此类推；
- f) 对于比特串 x 和 y ， $x \oplus y$ 表示它们的按位异或，而 $x || y$ 表示串联的比特串，例如：如果 $x = 0110$ 且 $y = 1010$ ，则 $x \oplus y = 1100$ 且 $x || y = 01101010$ ；
- g) 对于 128 比特字符串 k 和 m ， $Symm(k, m)$ 表示使用对称加密算法得到的 128 比特密文， m 为被加密的明文数据， k 为对称密钥，使用 128 比特的分组大小；
- h) 对于数字 m 和 n ， $m \bmod n$ 表示对具有模数 n 的 m 进行模运算的结果，例如：如果 $m = 9$ 且 $n = 2$ ，则 $m \bmod n = 1$ 。

H.2.2 密钥衍生函数

2个密钥衍生函数：签名函数 f_s 和加密函数 f_E 。2个函数的参数如下：

- 对称加密算法 $Symm$ ，具有 128 比特的输入和输出以及 128 比特的密钥，采用 SM4 分组密码算法，ECB/NoPadding 工作模式。
 - 一个 128 比特的密钥，分别表示为 f_s 函数的 k_s ，和 f_E 函数的 k_E 。
 - l 是一个 256 比特的整数，表示进行密钥衍生的椭圆曲线的基点的阶。
- 两个函数的输入为 (i^{INT}, j^{INT}) 两个整数，其范围是 $(0, 2^{32}-1)$ 。
- 两个函数的输出为一个 256 比特的整数 o ，范围为 $(0, l)$ 。
- 两个函数过程如下：

- a) 将输入 i^{INT} 和 j^{INT} 转换为 32 比特的比特串 i 和 j 。

- b) b) 拼接下列 128 位的比特串 x_S (用于 f_S) 或 x_E (用于 f_E)。
- 1) $x_S = (0^{32} || i || j || 0^{32})$
 - 2) $x_E = (1^{32} || i || j || 0^{32})$
- c) 创建如下的临时输出 y_S (用于 f_S) 或 y_E (用于 f_E)。输出为 $3 \times 128 = 384$ 比特的字符串。
- 1) $y_S = (\text{Symm}(k_S, x_S+1) \oplus (x_S+1)) || (\text{Symm}(k_S, x_S+2) \oplus (x_S+2)) || (\text{Symm}(k_S, x_S+3) \oplus (x_S+3))$
 - 2) $y_E = (\text{Symm}(k_E, x_E+1) \oplus (x_E+1)) || (\text{Symm}(k_E, x_E+2) \oplus (x_E+2)) || (\text{Symm}(k_E, x_E+3) \oplus (x_E+3))$

这里，“ $x+i$ ”表达了“将八位位组字符串 x 转换为整数，该整数加 i ，然后将加了 i 之后的整数转换回八位位组字符串”。即：

$$\begin{aligned} x_S &= (0^{32} || i || j || 0^{32}) = (0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_b || i || j || 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_b) \\ x_S + 1 &= (0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_b || i || j || 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001_b) \\ x_S + 2 &= (0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_b || i || j || 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010_b) \\ x_S + 3 &= (0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_b || i || j || 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0011_b) \\ x_E &= (1^{32} || i || j || 0^{32}) = (1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_b || i || j || 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_b) \\ x_E + 1 &= (1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_b || i || j || 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001_b) \\ x_E + 2 &= (1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_b || i || j || 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010_b) \\ x_E + 3 &= (1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_b || i || j || 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0011_b) \end{aligned}$$

- d) f_S 和 f_E 的最终输出如下：
- 1) $f_S(k_S, i^{INT}, j^{INT}) = y_S^{INT} \bmod 1$
 - 2) $f_E(k_E, i^{INT}, j^{INT}) = y_E^{INT} \bmod 1$

H.3 使用建议函数后的密钥衍生流程

将H.2 建议的函数与7.2.6 密钥衍生流程配合后，详细流程如表H.1所示。

密钥衍生流程		
V2X 设备	PRA	PCA
1) 生成： a) 1 个 SM4 密钥 k_S ，即签名对称密钥； b) 1 个 SM4 密钥 k_E ，即加密对称密钥； c) 1 对 SM2 密钥对 $(a, A = a * G)$ ，即签名密钥对，签名私钥	3) 对于每个 $(i INT, j INT)$ ，计算： a) 签名扩展公钥 $B_{i,j}$ ， $B_{i,j} = A + f_S(k_S, i INT, j INT) * G$ ； b) 加密扩展公钥 $Q_{i,j}$ ， $Q_{i,j} = P + f_E(k_E, i INT, j INT) * G$ ；	5) 生成 1 对 SM2 密钥 $(c, C=c * G)$ ，用于向 PRA 隐藏假名证书中的完整公钥，计算完整公钥 $S_{i,j} = B_{i,j} + C$ ； 6) 使用完整公钥 $S_{i,j}$ 构造 ToBeSignedCertificate，签发显式证书 $PC_{i,j}$ ，使用 $Q_{i,j}$ 加密 $(PC_{i,j}, c)$ 得到密文 $CT_{i,j}$ ，PCA 对该密

因子 a 和公钥因子 A; d) 1 对 SM2 密钥对 $(p, P = p * G)$, 即加密种子密钥对, 加密私钥因子 p 和公钥因子 P。 2) 将 (kS, kE, A, P) 随 $EeRaCertRequest$ 发送至 PRA。	4) 对于每个 $(iINT, jINT)$, 将生成的 (Bi,j, Qi,j) 随 $RaAcaCertRequest$ 发送至 PCA。	文再次进行签名得到 $SCTi,j$, 并将 $SCTi,j$ 发送至 PRA。
	7) 对于某个 V2X 设备, PRA 整理并打包其在某个 i 阶段所有的 $SCTi,j$, 以及每一条 $SCTi,j$ 对应的 j, 待该 V2X 设备发起假名证书下载请求后返回。	
8) 对于每个 $(iINT, jINT)$, 计算: a) 签名扩展私钥, $bi,j = (a + fS(kS, iINT, jINT)) \bmod l$ b) 加密扩展私钥, $qi,j = (p + fE(kE, iINT, jINT)) \bmod l$ 9) 对于每个 $(iINT, jINT)$, 验证 $SCTi,j$ 中的 PCA 签名: a) 如果验签成功, 使用 qi,j 解密 CTi,j 得到 (PCi,j, c), 计算该 PCi,j 中的公钥对应的完整私钥 $si,j = (bi,j + c) \bmod l$ b) 如果验证失败, 退出		

附录 I
(规范性附录)
链接值相关定义

I.1 链接值生成算法

本附录定义的链接值默认使用单种子算法。
证书链接值的计算生成主要采用对称加密算法和hash算法相结合，既可以提高计算效率，又有不可逆的性质，且每个链接值都可以由链接种子推出，但由链接值却无法推出链接种子，放置在证书中既可以保证假名证书的有效批量撤销，又不会泄露同组假名证书的信息。链接值生成的算法过程如图I.1所示：

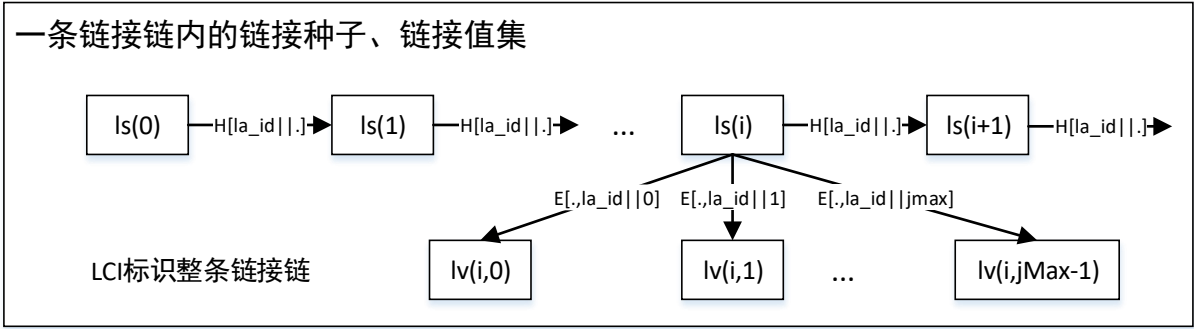


图 I.1 链接值生成算法过程

令la_id为某个LA相关联的32位标识字符串，每组假名证书内的链接值生成算法具体描述如下（个体与组链接值生成算法一致，对称算法SM4ECB举例，可支持AES128ECB；摘要算法以SM3举例，可支持SHA256；||表示拼接，0（xxx-bit）表示xxx个0）：

- a) 为每组假名证书生成一个 128bit 的随机数，作为该组假名证书第 i=0 个周期对应的初始链接种子 ls(0)；
- b) 对于 i>=1，通过以下方式计算第 i 个周期的链接种子 ls(i)：
 - 1) 计算 la_id (16-bit) || ls(i-1) (128-bit) || 0 (112-bit)；
 - 2) 对得到数据进行 SM3 计算，取前 128bit 作为 ls(i)。
 - 3) 在每个周期 i 内，对于每个 0 ≤ j < jMax（其中，jMax 表示每个 i 周期内，签发的证书最大数量），通过以下方式计算第 j 个链接值 lv(i, j)：
 - 4) 拼接 SM4 明文输入 input: la_id (16-bit) || j (32-bit) || 0 (80-bit)；
 - 5) 对输入数据使用 ls(i) 进行 SM4/ECB/NoPadding 加密得到 output；
 - 6) 使用 input XOR output 取前 72bit 作为 LA 对于该设备第 i 阶段的第 j 张假名证书的链接值。

I.2 基于个体链接值的证书撤销验证算法

设备方获取到假名证书类型对应CRL后，可以通过CRL中的链接种子和i值等相关信息进行计算，判断需要检测的假名证书是否被撤销。下面是基于个体链接值的证书撤销验证算法：

- a) 检验方会取出 CRL 中 individual 对应的所有 iMaxGroup。
- b) 将 iMaxGroup 的 iMax 依次与假名证书中对应的 iCert 值进行比较；
- c) 当 $iMax < iCert$ 则认为该证书不会被此链接种子撤销继续与下一个 iMax 进行对比；
- d) 当 $iMax \geq iCert$ 时，对比 iRev 和 iCert 的值；
- e) 当 $iCert > iRev$ 时，则认为该链接种子与该证书的撤销无关，并向后计算链接种子，计算公式为 $SM3(la_id(16bit) || ls(128bit) || 0(112bit))$ 并取前 128bit 作为 iRev+1 的链接种子，以此类推，其中 ls 取自 iMaxGroup 里的 singleSeeds：
 - 1) 当计算到 $iRev = iCert$ 时，则认为该链接种子有可能与该证书的撤销有关；
 - 2) 根据 i 值依次计算 0 至 jMax-1 的明文输入 input: $la_id(16bit) || j(32bit) || 0(80bit)$
 - 3) 使用明文输入数据和链接种子通过 SM4/ECB/NoPadding 对称算法计算 (ls, input) 得到密文输出 output。
 - 4) 使用 input XOR output 取前 72bit 作为个体链接值与当前证书的链接值进行比较；
 - 5) 若有相同，则认为该证书被撤销，若无相同，则重复步骤 b，直至循环完所有的 iMaxGroup。

1.3 基于组链接值的证书撤销验证算法

设备方获取到假名证书类型对应CRL后，可以通过CRL中的组链接种子和i值等相关信息进行计算，判断需要检测的假名证书是否被撤销。下面是基于组链接种子的证书撤销验证算法：

- a) 检验方会将 CRL 中 groupsSingleSeed 对应所有的 GroupSingleSeedCrlEntry。
- b) 将 GroupSingleSeedCrlEntry 中的 iMax 依次与假名证书中对应的 iCert 值进行比较；
- c) 当 $iMax < iCert$ 则认为该证书不会被此组链接种子撤销继续与下一个 iMax 进行对比；
- d) 当 $iMax \geq iCert$ 时，对比 iRev 和 iCert 的值；
- e) 当 $iCert > iRev$ 时，则认为该组链接种子与该证书的撤销无关，并向后计算组链接种子，计算公式为 $SM3(la_id(16bit) || ls(128bit) || 0(112bit))$ 并取前 128bit 作为 iRev+1 的组链接种子，以此类推：
 - 1) 当计算到 $iRev = iCert$ 时，则认为该组链接种子有可能与该证书的撤销有关；
 - 2) 根据 i 值和证书中组链接值的 j 值计算对应的明文输入数据 input: $la_id(16bit) || j(32bit) || 0(80bit)$ 。
 - 3) 使用明文输入数据和链接种子通过 SM4/ECB/NoPadding 计算 (ls, input)，得到密文输出数据 output。
 - 4) 使用 input XOR output 取出前 72bit 作为组链接值与当前证书的链接值进行比较；
 - 5) 若相同，则认为该证书被撤销，若无相同，则重复第 b 步，直至循环完所有的 GroupSingleSeedCrlEntry。

附 录 J
(规范性附录)
可信 CA 证书列表及互信认证流程

J.1 概述

本附录规范了V2X设备实现多PKI系统交叉认证过程中使用的数据结构和CTL列表下载说明。

J.2 引用的数据结构

本附录所引用的本标准其他章节定义的数据结构的名称如下：

Certificate
IssuerIdentifier
SequenceOfCertificate
Signature

J.3 根证书可信列表

J.3.1 RootCtl

```
RootCtl ::= SEQUENCE {  
    toBeSignedRootCtl    ToBeSignedRootCtl,  
    signature             Signature  
}
```

该数据结构包含：

- toBeSignedRootCtl：列表的内容；
- signature：由 toBeSignedRootCtl 中的 issuer 字段标识的签名者对 toBeSignedRootCtl 计算签名的方法和签名值。

J.3.2 ToBeSignedRootCtl

```
ToBeSignedRootCtl ::= SEQUENCE {  
    version                Uint8 (1),  
    issuer                 IssuerIdentifier,  
    series                 Uint16,  
    issueDate              Time32,  
    nextRootCtl            Time32,  
    pkiCtlInfoList         SequenceOfPkiCtlInfo  
}
```

该数据结构包含：

- version：根可信证书列表的版本号，本标准中为 1；
- issuer：根可信证书列表的签发者；

- series: 根可信证书列表的序列号;
- issueDate: 根可信证书列表的发布时间;
- nextRootCtl: 预计下次根可信证书列表的发布时间;
- pkiCtlInfoList: 可信 CA 证书列表的内容。

J.3.3 PkiCtlInfo

```
PkiCtlInfo ::= SEQUENCE {
    rootCertificate      Certificate,
    caListUrl            TF8String (SIZE (0..511)) OPTIONAL
}
```

SequenceOfPkiCtlInfo ::= SEQUENCE OF PkiCtlInfo

该数据结构包含:

- rootCertificate: 特定 PKI 系统中用于验证该 PKI 系统的根 CA 证书;
- caListUrl: 下载该 PKI 系统可信 CA 证书列表的地址。

J.3.4 下载根可信证书列表请求和响应参数说明

该接口用于V2X设备下载根可信证书列表。

根可信证书列表申请请求类型: HTTP GET或HTTPS GET

请求中包含版本信息、序列号、时间等参数;

响应中包含根可信证书列表等参数。

J.4 可信CA证书列表

J.4.1 PkiCtl

```
PkiCtl ::= SEQUENCE {
    toBeSignedPkiCtl    ToBeSignedPkiCtl,
    signature            Signature
}
```

该数据结构包含:

- toBeSignedPkiCtl: PKI 可信证书列表的内容;
- signature: 由 toBeSignedPkiCtl 中的 issuer 字段标识的签名者对 toBeSignedPkiCtl 计算签名的方法和签名值。

J.4.2 ToBeSignedPkiCtl

```
ToBeSignedPkiCtl ::= SEQUENCE {
    version              Uint8 (1),
    issuer               IssuerIdentifier,
    series               Uint16,
    issueDate            Time32,
    nextPkiCtl           Time32,
    pkiCertInfoList      SequenceOfPkiCertInfo
}
```

- 该数据结构包含：
- version：PKI 系统证书可信列表的版本号，本标准中为 1；
 - issuer：PKI 系统证书可信列表的签发者；
 - series：PKI 系统证书可信列表的序列号；
 - issueDate：PKI 系统证书可信列表的发布时间；
 - nextPkiCtl：预计下次 PKI 系统证书可信列表的发布时间；
 - pkiCtlInfoList：PKI 系统证书可信列表的内容。

J. 4. 3 PkiCertInfo

```
PkiCertInfo ::= SEQUENCE {  
    certificate          SequenceOfCertificate,  
    crlUrl               UTF8String (SIZE (0..511)) OPTIONAL,  
    maUrl                UTF8String (SIZE (0..511)) OPTIONAL  
}
```

```
SequenceOfPkiCertInfo ::= SEQUENCE OF PkiCertInfo
```

- 该数据结构包含：
- certificate：特定 PKI 系统中的可信证书；
 - crlUrl：下载与此可信证书相关的 CRL 列表的地址。
 - maUrl：上载与此可信证书相关的异常行为报告的地址。

J. 4. 4 下载PKI可信证书列表请求和响应参数说明

该接口用于V2X设备下载根可信证书列表。
PKI可信证书列表申请请求类型：HTTP GET或HTTPS GET
请求中包含版本信息、序列号、时间等参数；
响应中包含PKI可信证书列表等参数。

附 录 K
(资料性附录)
算法编码示例

K.1 概述

本附录提供算法编码示例。

K.2 签名算法编码示例

签名证书数据:

签名证书数据:

```
<Certificate>
  <version>3</version>
  <type><explicit/></type>
  <issuer>
    <sm3AndDigest>21 22 23 24 25 26 27 28</sm3AndDigest>
  </issuer>
  <toBeSigned>
    <id>
      <linkageData>
        <iCert>10</iCert>
        <linkage-value>08 08 08 08 08 08 08 08 08</linkage-value>
      </linkageData>
    </id>
    <cracaId>09 09 09</cracaId>
    <crlSeries>2</crlSeries>
    <validityPeriod>
      <start>2323200</start>
      <duration>
        <hours>200</hours>
      </duration>
    </validityPeriod>
    <verifyKeyIndicator>
      <verificationKey>
        <ecsigSm2>
          <compressed-y-0>
            1E 5F 89 A5 74 08 2C 8B 36 11 FB A8 D0 65 F3 08
            B4 8F 64 EE EF 71 30 3C 75 BA 8E 95 D5 BE 89 42
          </compressed-y-0>
        </ecsigSm2>
      </verificationKey>
    </verifyKeyIndicator>
  </toBeSigned>
</Certificate>
```

```

        </ecsigSm2>
        </verificationKey>
    </verifyKeyIndicator>
</toBeSigned>
<signature>
    <sm2Signature>
        <r>
            04 04 04 04 04 04 04 04 04 04 04 04 04 04 04 04
            04 04 04 04 04 04 04 04 04 04 04 04 04 04 04 04
        </r>
        <s>
            05 05 05 05 05 05 05 05 05 05 05 05 05 05 05 05
            05 05 05 05 05 05 05 05 05 05 05 05 05 05 05 05
        </s>
    </sm2Signature>
</signature>
</Certificate>签名证书数据OER编码:
80 03 00 83 08 21 22 23 24 25 26 27 28 00 80 00
00 0a 08 08 08 08 08 08 08 08 09 09 09 00 02
00 23 73 00 84 00 c8 80 84 21 82 1e 5f 89 a5 74
08 2c 8b 36 11 fb a8 d0 65 f3 08 b4 8f 64 ee ef
71 30 3c 75 ba 8e 95 d5 be 89 42 84 40 04 04 04
04 04 04 04 04 04 04 04 04 04 04 04 04 04 04 04
04 04 04 04 04 04 04 04 04 04 04 04 04 05 05 05
05 05 05 05 05 05 05 05 05 05 05 05 05 05 05 05
05 05 05 05 05 05 05 05 05 05 05 05 05 05
签名证书对应的私钥:
17 9e 18 ae e3 38 72 e1 60 5e 99 e8 bc 61 20 f7
95 e7 c8 f8 a9 ce 60 7a 4a 71 78 68 23 53 a1 69
签名证书对应的公钥:
X:
1e 5f 89 a5 74 08 2c 8b 36 11 fb a8 d0 65 f3 08
b4 8f 64 ee ef 71 30 3c 75 ba 8e 95 d5 be 89 42
Y:
77 84 ef 22 09 41 de eb cf 5b 3e 34 65 98 0a 30
91 67 34 6f fe 05 0a e1 00 7c 2d 2f 3f 10 57 74
待签名数据值:
34 34 34 34 34 34 34 34 34 34 34 34 34 34 34 34
34 34 34 34 34 34 34 34 34 34 34 34 34 34 34 34
签名证书哈希值:
e3 8c 7c b2 ec ed b9 28 9b 6e 71 ad 30 b6 26 2c
4a 57 85 73 b9 a4 4d e8 ab 65 9a 6b cd 96 7d 77

```

签名证书HashID8值:

ab 65 9a 6b cd 96 7d 77

组成ToBeSignedData数据结构:

```
<ToBeSignedData>
  <payload>
    <data>
      <protocolVersion>3</protocolVersion>
      <content>
        <unsecuredData>
          34 34 34 34 34 34 34 34 34 34 34 34 34 34 34
          34 34 34 34 34 34 34 34 34 34 34 34 34 34 34
        </unsecuredData>
      </content>
    </data>
  </payload>
  <headerInfo>
    <psid>32</psid>
    <generationTime>2323203</generationTime>
  </headerInfo>
</ToBeSignedData>
```

ToBeSignedData编码值:

40 03 80 20 34 34 34 34 34 34 34 34 34 34 34 34
 34 34 34 34 34 34 34 34 34 34 34 34 34 34 34 34
 34 34 34 34 40 01 20 00 00 00 00 00 23 73 03

ZA值:

ff 25 43 65 9c 0b 93 c3 9f e3 b4 00 73 98 df 9f
 b4 0c f4 5d e3 65 e7 c9 af 16 5d 42 68 35 0b b9

Hash (ZA || Data input) 值:

7a b1 2a 19 b6 45 df c6 36 e2 0e a7 ae b0 e5 5e
 f3 f3 68 38 87 b2 e0 76 a8 eb 32 94 90 4e e5 d4

签名结果值:

R:

DD F9 07 5D D2 D9 47 05 A1 2F 42 5E 51 CB 27 4C
 B8 F4 AE 65 B3 1C 7C DD CD 31 C2 BC 3C 55 28 81

S:

20 86 11 DE 09 A7 CC 4E 48 76 E6 9E F2 E7 C0 6A
 85 39 A4 67 F2 0B 0E A7 FE EB 2A FE 9D 59 5C C4

SPDU数据结构:

```
<V2XSecData>
  <protocolVersion>3</protocolVersion>
  <content>
    <signedData>
```

```

<hashId><sm3/></hashId>
<tbsData>
  <payload>
    <data>
      <protocolVersion>3</protocolVersion>
      <content>
        <unsecuredData>
          34 34 34 34 34 34 34 34 34 34 34 34 34 34 34 34
          34 34 34 34 34 34 34 34 34 34 34 34 34 34 34 34
        </unsecuredData>
      </content>
    </data>
  </payload>
  <headerInfo>
    <psid>32</psid>
    <generationTime>2323203</generationTime>
  </headerInfo>
</tbsData>
<signer>
  <digest>AB 65 9A 6B CD 96 7D 77</digest>
</signer>
<signature>
  <sm2Signature>
    <r>
      DD F9 07 5D D2 D9 47 05 A1 2F 42 5E 51 CB 27 4C
      B8 F4 AE 65 B3 1C 7C DD CD 31 C2 BC 3C 55 28 81
    </r>
    <s>
      20 86 11 DE 09 A7 CC 4E 48 76 E6 9E F2 E7 C0 6A
      85 39 A4 67 F2 0B 0E A7 FE EB 2A FE 9D 59 5C C4
    </s>
  </sm2Signature>
</signature>
</signedData>
</content>
</V2XSecData>
SPDU数据OER编码结果值:
03 81 02 40 03 80 20 34 34 34 34 34 34 34 34 34
34 34 34 34 34 34 34 34 34 34 34 34 34 34 34 34
34 34 34 34 34 34 34 40 01 20 00 00 00 00 00 23
73 03 80 ab 65 9a 6b cd 96 7d 77 84 40 dd f9 07
5d d2 d9 47 05 a1 2f 42 5e 51 cb 27 4c b8 f4 ae

```

```

65 b3 1c 7c dd cd 31 c2 bc 3c 55 28 81 20 86 11
de 09 a7 cc 4e 48 76 e6 9e f2 e7 c0 6a 85 39 a4
67 f2 0b 0e a7 fe eb 2a fe 9d 59 5c c4

```

K.3 加密算法编码示例

加密证书数据：

```

<Certificate>
  <version>3</version>
  <type><explicit/></type>
  <issuer>
    <sm3AndDigest>21 22 23 24 25 26 27 28</sm3AndDigest>
  </issuer>
  <toBeSigned>
    <id>
      <linkageData>
        <iCert>10</iCert>
        <linkage-value>08 08 08 08 08 08 08 08 08</linkage-value>
      </linkageData>
    </id>
    <cracaId>09 09 09</cracaId>
    <crlSeries>2</crlSeries>
    <validityPeriod>
      <start>2323200</start>
      <duration>
        <hours>200</hours>
      </duration>
    </validityPeriod>
    <encryptionKey>
      <supportedSymmAlg><sm4Ccm/></supportedSymmAlg>
      <publicKey>
        <ecencSm2>
          <compressed-y-1>
            FF 2F DD 71 5A C1 A3 41 B4 A2 03 2A B1 16 92 B2
            DE EB 2E 77 2F 59 5E 9F 65 F1 21 5F B8 7E 6D CB
          </compressed-y-1>
        </ecencSm2>
      </publicKey>
    </encryptionKey>
    <verifyKeyIndicator>
      <verificationKey>

```

```

        <ecsigSm2>
            <compressed-y-0>
                1E 5F 89 A5 74 08 2C 8B 36 11 FB A8 D0 65 F3 08
                B4 8F 64 EE EF 71 30 3C 75 BA 8E 95 D5 BE 89 42
            </compressed-y-0>
        </ecsigSm2>
    </verificationKey>
</verifyKeyIndicator>
</toBeSigned>
<signature>
    <sm2Signature>
        <r>
            04 04 04 04 04 04 04 04 04 04 04 04 04 04 04 04
            04 04 04 04 04 04 04 04 04 04 04 04 04 04 04 04
        </r>
        <s>
            05 05 05 05 05 05 05 05 05 05 05 05 05 05 05 05
            05 05 05 05 05 05 05 05 05 05 05 05 05 05 05 05
        </s>
    </sm2Signature>
</signature>
</Certificate>
加密证书的OER编码:
80 03 00 83 08 21 22 23 24 25 26 27 28 01 80 00
00 0a 08 08 08 08 08 08 08 08 08 09 09 09 00 02
00 23 73 00 84 00 c8 01 82 21 83 ff 2f dd 71 5a
c1 a3 41 b4 a2 03 2a b1 16 92 b2 de eb 2e 77 2f
59 5e 9f 65 f1 21 5f b8 7e 6d cb 80 84 21 82 1e
5f 89 a5 74 08 2c 8b 36 11 fb a8 d0 65 f3 08 b4
8f 64 ee ef 71 30 3c 75 ba 8e 95 d5 be 89 42 84
40 04 04 04 04 04 04 04 04 04 04 04 04 04 04 04
04 04 04 04 04 04 04 04 04 04 04 04 04 04 04 04
04 05 05 05 05 05 05 05 05 05 05 05 05 05 05 05
05 05 05 05 05 05 05 05 05 05 05 05 05 05 05 05
05
加密证书的签名私钥:
17 9e 18 ae e3 38 72 e1 60 5e 99 e8 bc 61 20 f7
95 e7 c8 f8 a9 ce 60 7a 4a 71 78 68 23 53 a1 69
加密证书的签名私钥对应公钥:
X:
1e 5f 89 a5 74 08 2c 8b 36 11 fb a8 d0 65 f3 08
b4 8f 64 ee ef 71 30 3c 75 ba 8e 95 d5 be 89 42

```

Y:

77 84 ef 22 09 41 de eb cf 5b 3e 34 65 98 0a 30
91 67 34 6f fe 05 0a e1 00 7c 2d 2f 3f 10 57 74

加密证书的加解密私钥:

ef 20 26 9f db 6f ba 2d 3b f5 52 e1 87 3b c3 c6
4f 6f ae d9 19 67 d7 9d 3d 53 f5 f0 5b bc b9 34

加密证书的加解密私钥对应公钥:

X:

ff 2f dd 71 5a c1 a3 41 b4 a2 03 2a b1 16 92 b2
de eb 2e 77 2f 59 5e 9f 65 f1 21 5f b8 7e 6d cb

Y:

1c 20 d1 47 63 ca 3b 98 24 9e 3f 92 ad a4 bf 6c
58 f7 ed 14 6d d1 d6 7b f9 5c ef c9 3b 51 68 fd

加密证书摘要值(SM3):

31 10 49 f5 05 c4 aa e4 09 f4 78 cc 55 3d 08 0e
2a 79 7d 1b 5b d9 9b 02 96 f3 2c 9d ed e1 26 0f

待加密的明文数据:

a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 a0 a1 a2 a3 a4 a5
a6 a7 a8 a9

待加密的明文数据有效OER编码:

03 80 14 a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 a0 a1 a2
a3 a4 a5 a6 a7 a8 a9

SM4对称密钥值:

ca 44 ef 8d f3 25 ab b3 8d ac 37 43 dd 32 43 df

对称加密IV值:

72 43 52 3C 65 17 8B 96 68 37 A3 6F

SM4CCM对数据进行对称加密结果:

fb 78 40 15 24 ca 9c 2d 68 3b c4 e9 5d dc 71 b8
28 07 77 81 aa 8e 3f e8 1c e4 de 21 38 76 49 19
59 ee 87 63 e2 21 55

加密报文数据:

<V2XSecData>

<protocolVersion>3</protocolVersion>

<content>

<encryptedData>

<recipients>

<certRecipInfo>

<recipientId>96 F3 2C 9D ED E1 26 0F</recipientId>

<encKey>

<ecencSm2>

<v>

```

        <compressed-y-1>
            A1 BF D8 0D 29 69 93 E5 D3 14 21 F9 11 D6 1E A1
            EE 2D 4E B0 54 3F 35 FF 3C BB DD 90 2F F5 22 2F
        </compressed-y-1>
    </v>
    <t>
        1A 03 AB 79 33 87 DC 65 9F 93 BB 11 F4 D0 ED CF
        77 14 AA 2C 08 AA F1 B8 07 D1 7E 9E 8D CE 1A 89
    </t>
    <c>E0 E6 AE 0D A8 2D 3C C4 3E 3D 85 80 A7 99 DA B6</c>
</ecencSm2>
</encKey>
</certRecipInfo>

</recipients>
< ciphertext>
    <sm4ccm>
        <nonce>72 43 52 3C 65 17 8B 96 68 37 A3 6F</nonce>
        <ccmCiphertext>
            FB 78 40 15 24 CA 9C 2D 68 3B C4 E9 5D DC 71 B8
            28 07 77 81 AA 8E 3F E8 1C E4 DE 21 38 76 49 19
            59 EE 87 63 E2 21 55
        </ccmCiphertext>
    </sm4ccm>
</ ciphertext>
</encryptedData>
</content>
</V2XSecData>加密报文数据OER编码:
03 82 01 01 82 96 f3 2c 9d ed e1 26 0f 82 51 83
a1 bf d8 0d 29 69 93 e5 d3 14 21 f9 11 d6 1e a1
ee 2d 4e b0 54 3f 35 ff 3c bb dd 90 2f f5 22 2f
1a 03 ab 79 33 87 dc 65 9f 93 bb 11 f4 d0 ed cf
77 14 aa 2c 08 aa f1 b8 07 d1 7e 9e 8d ce 1a 89
e0 e6 ae 0d a8 2d 3c c4 3e 3d 85 80 a7 99 da b6
81 34 72 43 52 3c 65 17 8b 96 68 37 a3 6f 27 fb
78 40 15 24 ca 9c 2d 68 3b c4 e9 5d dc 71 b8 28
07 77 81 aa 8e 3f e8 1c e4 de 21 38 76 49 19 59
ee 87 63 e2 21 55

```
